

# Building LLMs from Scratch

## A Practical Guide from First Principles

\*Generated February 2026 from the YouTube playlist\*

---

## About This Book

---

This manuscript is a faithful written record of the video series "**Building LLMs from Scratch**".

Every concept, code example, and explanation is derived directly from the video transcripts. Nothing has been added, inferred, or invented. Where transcripts were unclear, the text says so explicitly.

Each paragraph includes a citation timestamp that references the exact moment in the source video.

**Videos covered:** 43 of 43 total

---

# Table of Contents

---

- Chapter 1: Series Introduction - Building Large Language Models From Scratch
- Chapter 2: Large Language Models (LLM) Basics
- Chapter 3: Pre-training and Fine-tuning LLMs
- Chapter 4: Introduction to Transformers
- Chapter 5: The Evolution and Capabilities of GPT Models
- Chapter 6: Stages of Building an LLM from Scratch
- Chapter 7: Tokenization from Scratch
- Chapter 8: The GPT Tokenizer: Byte Pair Encoding
- Chapter 9: Creating Input-Target Data Pairs using Python DataLoader
- Chapter 10: Token Embeddings
- Chapter 11: Positional Encoding
- Chapter 12: The Data Preprocessing Pipeline for Large Language Models
- Chapter 13: Introduction to the Attention Mechanism in Large Language Models (LLMs)
- Chapter 14: Simplified Self-Attention Mechanism Without Trainable Weights
- Chapter 15: Coding the Self-Attention Mechanism with Key, Query, and Value Matrices
- Chapter 16: Causal Self-Attention Mechanism
- Chapter 17: Multi-Head Attention Part 1 - Basics and Python Code
- Chapter 18: Multi-Head Attention with Weight Splits
- Chapter 19: Birds Eye View of the LLM Architecture
- Chapter 20: Layer Normalization in the LLM Architecture
- Chapter 21: GELU Activation and Feed-Forward Networks in LLMs
- Chapter 22: Shortcut Connections in the LLM Architecture
- Chapter 23: Coding the Transformer Block
- Chapter 24: Assembling the GPT Model Architecture
- Chapter 25: Generating Text from GPT-2 Output
- Chapter 26: Measuring LLM Performance with Loss Functions
- Chapter 27: Evaluating LLM Performance on a Real Dataset
- Chapter 28: Training the Large Language Model
- Chapter 29: Temperature Scaling in Large Language Models
- Chapter 30: Top-K Sampling in Large Language Models
- Chapter 31: Saving and Loading LLM Model Weights using PyTorch

- Chapter 32: Loading Pre-trained GPT-2 Weights
  - Chapter 33: Introduction to Large Language Model Fine-Tuning
  - Chapter 34: Dataloaders in LLM Classification Finetuning
  - Chapter 35: Coding the Model Architecture for LLM Classification Fine-tuning
  - Chapter 36: Coding a Fine-Tuned LLM Spam Classification Model
  - Chapter 37: Introduction to LLM Instruction Fine-tuning | Loading Dataset | Alpaca Prompt format
  - Chapter 38: Data Batching in LLM Instruction Fine-tuning
  - Chapter 39: Dataloaders in Instruction Fine-tuning
  - Chapter 40: Instruction Fine-tuning: Loading Pre-trained LLM Weights
  - Chapter 41: LLM Fine-tuning Training Loop
  - Chapter 42: Evaluating Fine-Tuned LLMs Using Ollama
  - Chapter 43: Build LLMs from Scratch: A 20-Minute Summary
-

# Chapter 1: Series Introduction - Building Large Language Models From Scratch

---

*Source video: Lecture 1: Building LLMs from scratch: Series introduction*

*Duration: 16 minutes | Playlist position 1*

## Course Philosophy and Motivation

This series aims to teach the fundamentals of building large language models (LLMs) from scratch, without assuming prior knowledge. The instructor, Dr. Raj dander, emphasizes a "nuts and bolts" approach to understanding the core concepts that make LLMs powerful. The motivation behind this series stems from the transformative impact of LLMs and generative AI across various industries, leading to increased job opportunities in this field. [\[0:05\]](#)

A common pitfall observed in learning about LLMs is the direct jump to applications and running code without a foundational understanding. This series intends to bridge that gap by providing a comprehensive, step-by-step guide to building an LLM from the ground up. Upon completion, participants will have successfully built their own LLM, fostering confidence and a deeper comprehension of the subject. [\[0:05\]](#)

The creation of this series involves significant effort, including the preparation of detailed lecture notes that will be shared with the audience. All video content will be made available completely free of charge. The instructor will share his personal journey in learning LLMs, discuss the limitations of existing online resources, and outline the unique approach of this playlist. [\[0:05\]](#)

## Evolution of Natural Language Processing

To appreciate the current state of LLMs, the series looks back at the early days of natural language processing (NLP). An example provided is ELIZA, one of the first chatbots developed around 50 to 60 years ago, which simulated a therapist. A sample interaction demonstrates ELIZA's limited conversational capabilities, where the dialogue quickly becomes circular and unproductive. [\[4:12\]](#)

In contrast, modern LLMs like ChatGPT provide highly useful and specific responses. When asked for resources to learn about large language models, ChatGPT offers a comprehensive list including books, online courses, and research papers. This stark difference highlights the significant advancements in NLP and LLMs over the decades. [\[4:45\]](#)

The current era is characterized by the power and sophistication of LLMs such as GPT. While familiarity with ChatGPT is not assumed, participants will learn about it as they build their own version throughout the playlist. This historical perspective serves to underscore the

remarkable progress in the field and motivate the learning journey. [4:45]

## **Open Source vs. Closed Source Models and Generative AI**

The landscape of LLMs is dynamic, with major players like Meta releasing open-source models such as Llama 3.1. Open-source models make their architecture publicly available, allowing for greater transparency and accessibility. This contrasts with closed-source models, like those from OpenAI, where details about the architecture and weights are often kept proprietary. [4:45]

A graph illustrates the trend of decreasing performance gaps between open-source and closed-source models. While closed-source models like GPT-4 initially led, open-source alternatives are rapidly catching up, with Llama 3.1 demonstrating comparable performance. This signifies that essential information for learning and building LLMs is increasingly available through open-source initiatives. [4:45]

Generative AI extends beyond just text, encompassing video, audio, and 3D models. Examples of AI-generated realistic videos are presented to showcase the breadth of capabilities within generative AI. The series will clarify the distinctions between generative AI and LLMs, positioning LLMs as a subset of generative AI focused on language. [4:45]

The economic impact of generative AI is substantial, evidenced by the rapid growth of the job market. Viua's own AI application, featuring functionalities like an MCQ generator powered by an LLM, demonstrates the practical applications being developed. The projected growth of the generative AI job market highlights the increasing demand for skills in this domain. [4:45]

## **Addressing the Need for Foundational Learning**

The search for resources to learn LLMs reveals a common issue: many courses focus on application development rather than building models from scratch. Titles like "Build LLM Apps" or "Master LLM Concepts" often do not delve into the fundamental architecture or the "nuts and bolts" required for in-depth understanding. [4:45]

Similarly, YouTube searches for "LLM from scratch" often yield playlists with short, fragmented videos that do not provide the comprehensive, deep technical instruction needed. Even advanced courses, like those by Andrej Karpathy, can be too complex for beginners, starting mid-concept and lacking the foundational approach desired. [4:45]

The instructor found a valuable resource in a book by Sebastian Raschka, which serves as the reference material for this series. The plan is to convert the entire content of this book into a series of detailed video lectures, aiming for 50-60 videos that cover every aspect from the very basics. This approach ensures that nothing is assumed and all concepts are thoroughly

explained. [4:45]

The lecture notes will be extensive, potentially reaching 200-300 pages, and the videos will be designed for individuals serious about transitioning into the LLM field and understanding the technology correctly. This contrasts with tools like LangChain, which are useful for building LLM applications but require a prior understanding of how LLMs themselves are constructed. [4:45]

## **Course Objectives and Outline**

The primary goal of this course is to equip learners with the confidence to discuss LLM concepts in detail, particularly in job interviews. By building an LLM from scratch, participants will gain a deeper understanding than those who only deploy applications. This foundational knowledge will enable them to confidently answer questions about core components like query, key, value, or positional embeddings. [4:45]

The course will follow a structured outline, converting the table of contents from the reference book into video lectures. The instructor encourages audience engagement by requesting comments to gauge interest and maintain motivation for creating subsequent videos. All content will be released for free to benefit the wider community. [4:45]

The next lecture will cover an introduction to large language models, detailing the topics visible on the board. [4:45]

---

\*End of Chapter 1. Source: <https://www.youtube.com/watch?v=Xpr8D6LeAtw>\*

---

## Chapter 2: Large Language Models (LLM) Basics

---

*Source video: Lecture 2: Large Language Models (LLM) Basics*

*Duration: 33 minutes | Playlist position 2*

### What is a Large Language Model?

A Large Language Model (LLM) is a neural network designed to understand, generate, and respond to human-like text. At its core, an LLM is a neural network, a system of interconnected neurons inspired by the human brain, stacked in layers to process information and produce an output. These deep neural networks have found applications in various fields, including image detection, text generation, and autonomous driving. LLMs specifically leverage this architecture for text-related tasks. [\[0:05\]](#)

The ability of an LLM to understand, generate, and respond to human-like text can be demonstrated through interactions with models like ChatGPT. For instance, a user can ask ChatGPT to plan a relaxing day, specifying preferences like reading a book. The LLM then generates a detailed plan, including activities like morning meditation and reading time, mimicking a human-like conversation. [\[5:37\]](#)

In essence, LLMs are deep neural networks trained on vast amounts of data. This training enables them to perform specific tasks such as understanding, generating, and responding to human-like text, often in a manner that is indistinguishable from human communication. [\[5:37\]](#)

### Why "Large" Language Model?

The term "large" in Large Language Model refers to the immense number of parameters these models possess. Unlike earlier language models, LLMs typically contain billions, and sometimes even trillions, of parameters. For example, GPT-3, an earlier version of GPT, ranged from 125 million parameters in its "small" version to 175 billion parameters in its largest configuration. GPT-4 has an even greater number of parameters. [\[5:37\]](#)

This increase in model size is evident when comparing different versions of GPT. GPT-1, GPT-2, and GPT-3 show a significant rise in parameters, with GPT-3 having 175 billion parameters compared to GPT-2's 1.5 billion. This trend of increasing parameters correlates with improved performance. [\[5:37\]](#)

A broader view of model size over time, from 1950 to 2022, illustrates this exponential growth. While models in the mid-20th century had parameters in the tens or hundreds, by 2020, models reached hundreds of millions, billions, and even close to a trillion parameters. The orange symbols on the graph, representing LLMs, consistently show the largest

parameter counts, underscoring why they are termed "large." [5:37]

The "language model" part of the terminology signifies that these models exclusively deal with language. They are designed for a wide array of natural language processing (NLP) tasks, including question answering, translation, and sentiment analysis, but do not process other modalities like images or video. [5:37]

## **LLMs vs. Earlier NLP Models**

While Natural Language Processing (NLP) as a field predates LLMs, earlier NLP models were typically designed for very specific tasks, such as language translation or sentiment analysis. In contrast, modern LLMs are versatile and can perform a wide range of NLP tasks using the same underlying architecture. For instance, a GPT model trained for text completion can also perform well in language translation. [13:28]

A key difference lies in their application capabilities. Earlier language models struggled with tasks like drafting an email based on custom instructions. Modern LLMs like ChatGPT, however, can perform such tasks with ease, even incorporating details like emojis. This broad applicability and versatility distinguish LLMs from their predecessors. [13:28]

## **The Secret Sauce: Transformer Architecture**

The remarkable capabilities of LLMs are largely attributed to the Transformer architecture. Introduced in a 2017 paper titled "Attention Is All You Need," this architecture revolutionized the field of AI. The Transformer architecture, though complex with components like input embedding, multi-head attention, and positional encoding, is the foundation upon which modern LLMs are built. [13:28]

The "Attention Is All You Need" paper has garnered over 100,000 citations, highlighting its significant impact. While the paper is dense, its concepts, including dot product attention, key-query-value mechanisms, and multi-head attention, are crucial for understanding LLMs. Subsequent lectures in this series will delve into these details. [13:28]

## **Distinguishing AI, ML, DL, LLM, and Generative AI**

Understanding the relationship between various AI-related terminologies is essential. Artificial Intelligence (AI) is the broadest category, encompassing any machine exhibiting human-like intelligence. Machine Learning (ML) is a subset of AI where systems learn from data and adapt based on user interaction. [13:28]

Deep Learning (DL) is a further subset of ML that specifically utilizes neural networks. While ML can involve various algorithms like decision trees, DL is characterized by its reliance on neural network architectures. For example, predicting heart disease using a decision tree is ML but not DL, whereas image detection using a convolutional neural



network is DL. [13:28]

Large Language Models (LLMs) are a specialized application within Deep Learning, focusing exclusively on text processing. Generative AI, on the other hand, is a broader concept that combines LLMs with Deep Learning to create new content across various modalities, including text, images, sound, and video. Therefore, LLMs are a subset of DL, and Generative AI can be seen as a combination of LLM capabilities and broader DL techniques. [13:28]

The hierarchy can be visualized as  $AI > ML > DL > LLM$ , with Generative AI encompassing LLMs and other DL applications for content creation. [13:28]

## Applications of LLMs

LLMs have a wide range of applications, broadly categorized into five main areas. Firstly, they excel at content creation, enabling the generation of original poems, books, articles, and other forms of text that may not have existed previously. [24:00]

Secondly, LLMs serve as powerful chatbots and virtual assistants. Companies are increasingly using them for customer service, automating interactions for tasks like booking flights or managing bank accounts. This playlist aims to equip viewers with the skills to develop their own chatbots. [24:00]

Thirdly, LLMs are highly effective for machine translation, allowing for quick and accurate translation of text between languages, including some regional languages, though support for less common languages is an ongoing area of research. [24:00]

Fourthly, as mentioned, LLMs are adept at new text generation, producing creative content like poems, books, and news articles. Finally, LLMs can perform sentiment analysis, identifying the emotional tone of a given text, which is useful for applications like hate speech detection on social media platforms. [24:00]

To illustrate these applications, a portal was created featuring tools like a YouTube generator, MCQ generator, text summarizer, and worksheet generator for educators. For instance, an LLM can generate a lesson plan on gravity aligned with specific curriculum standards or create multiple-choice questions with answers and explanations for a topic like World War II. [24:00]

The potential applications of LLMs are vast, limited only by imagination. Understanding the foundational concepts, such as the Transformer architecture and its components like key-query-value and positional encoding, is crucial for developing impactful LLM applications. Superficial knowledge gained from simply running code is insufficient; a deep understanding of the underlying mechanisms is key. [24:00]

\*End of Chapter 2. Source: <https://www.youtube.com/watch?v=3dWzNZXA8DY>\*

---

## Chapter 3: Pre-training and Fine-tuning LLMs

---

*Source video: Lecture 3: Pretraining LLMs vs Finetuning LLMs*

*Duration: 28 minutes | Playlist position 3*

### Stages of Building Large Language Models

Building a large language model (LLM) involves two primary stages: pre-training and fine-tuning. Pre-training is the initial phase where the model learns from a vast and diverse dataset. Fine-tuning is a subsequent stage where the pre-trained model is further trained on a more specific, narrower dataset to adapt it for particular tasks or domains. [0:06]

The previous lecture covered the basics of LLMs, their origins, the factors contributing to their effectiveness, and the distinctions between LLMs, Generative AI, deep learning, machine learning, and artificial intelligence. It also touched upon various applications of LLMs. [0:06]

### Understanding Pre-training

Pre-training is essentially training a model on a large and diverse dataset. This process enables LLMs to interact effectively, understand human language, and respond accurately. For instance, GPT-3, a precursor to GPT-4, had 175 billion parameters and was trained on an extensive corpus of data. [0:06]

The data used for GPT-3's training included approximately 410 billion words from Common Crawl, a vast open repository of internet data. Additionally, 20 billion words were sourced from WebText2, a dataset comprising Reddit submissions, blog posts, Stack Overflow articles, and code. Books contributed around 67 billion words, and Wikipedia provided 3 billion words, totaling about 300 billion tokens or words for GPT-3's training. [0:06]

LLMs are trained on a task called word completion, where they predict the next word in a sequence. For example, given the sentence "The lion is in the...", the LLM would predict "forest." [6:18]

A surprising outcome of this pre-training approach is that models trained solely for word completion demonstrate capabilities in a wide range of other tasks without explicit training. These tasks include translation, answering multiple-choice questions, text summarization, and sentiment detection. [6:18]

This emergent capability distinguishes LLMs from traditional Natural Language Processing (NLP) models, which require separate training for each specific task like translation or sentiment analysis. Pre-trained LLMs offer a single model capable of performing multiple tasks effectively. [6:18]

For example, while GPT-4 was not specifically trained to generate multiple-choice questions, it can perform this task effectively due to its extensive pre-training. Companies leverage this by building applications, such as an MCQ generator, that call upon pre-trained models like GPT-4. [6:18]

In essence, pre-training involves training an LLM on a massive amount of data, equipping it with the ability to perform a wide array of tasks. [6:18]

## **The Need for Fine-tuning**

Fine-tuning becomes necessary when a specific application requires responses tailored to a particular company or domain, rather than generic information. For instance, an airline company developing a chatbot needs it to provide specific pricing and flight information relevant to their services, which a generic pre-trained model might not possess. [6:18]

Similarly, an educational company aiming to generate high-quality questions might need to fine-tune a pre-trained model to better suit their specific educational content and standards. [6:18]

Fine-tuning is a refinement process that occurs on a much narrower dataset. For example, a bank like JP Morgan can take a pre-trained model and train it further on its own proprietary data to ensure that the model's responses are specific to the bank's operations and information. [6:18]

Fine-tuning is crucial for building applications specific to a particular task or domain. While general users might find pre-trained models like GPT-4 sufficient, companies deploying LLMs in real-world applications often require fine-tuning on their datasets. [6:18]

Examples of fine-tuning include SK Telecom's development of a Korean-language chatbot for customer service, which significantly improved summarization quality and intent recognition accuracy. Another example is Harvey, an AI legal tool for attorneys, which was fine-tuned on legal case history to provide specialized legal knowledge. [6:18]

JP Morgan Chase also developed its own AI-powered LLM suite, fine-tuned with their internal data, to provide specific answers for their employees. These examples highlight the necessity of fine-tuning for specialized applications in telecommunications, legal, and finance sectors. [6:18]

While using foundational models directly is suitable for students' general purposes, fine-tuning is essential for developing more advanced, industry-specific applications. [6:18]

## **The Pre-training and Fine-tuning Schematic**

The process of building an LLM can be visualized in three steps. The first step involves gathering a large dataset, which can include internet text, books, media, and research articles. This dataset can contain billions or even trillions of words. [0:06]

The second step is the actual pre-training of the LLM on this massive dataset. This requires significant computational power and resources, with the pre-training cost for GPT-3 estimated at \$4.6 million. The result of this stage is a foundational model, also known as a base model. [0:06]

The third step is fine-tuning this pre-trained foundational model on a specific, labeled dataset to create specialized applications. These applications can include personal assistants, translation bots, summarization tools, or classification agents. [0:06]

Pre-training typically uses unlabeled data in an unsupervised learning manner, often referred to as autoregression, where the model predicts the next word in a sequence. In contrast, fine-tuning predominantly utilizes labeled data. [0:06]

For example, to build a spam classification model, a labeled dataset of emails marked as spam or not spam is required for fine-tuning. Similarly, for specialized tasks like Harvey, the legal AI tool, the fine-tuning process would involve providing legal case studies with corresponding answers. [0:06]

The term "pre-training" is used because there is a subsequent fine-tuning component. If fine-tuning were not a consideration, the initial training phase might simply be called "training." [0:06]

## Steps in Building an LLM

The process of building an LLM can be summarized in three key steps. First, train on a large corpus of text data, often referred to as "raw text," which means text without any predefined labels. [0:06]

Second, this initial training stage is called pre-training, resulting in a pre-trained LLM, also known as a base or foundational model. This model, like GPT-4, is capable of text completion and a variety of other tasks such as sentiment analysis and question answering. [0:06]

Third, after obtaining the pre-trained LLM, it is further trained on labeled data through a process called fine-tuning. This fine-tuning enables specific applications like those seen in the legal sector (Harvey), telecommunications (SK Telecom), and finance (JP Morgan Chase). [0:06]

Within fine-tuning, there are typically two categories: instruction fine-tuning and fine-tuning for classification tasks. Instruction fine-tuning involves providing instruction-answer pairs as labeled data, such as English-to-French translation examples or customer support

query-response pairs. [0:06]

Fine-tuning for classification tasks requires labeled data where each data point is associated with a specific category, like classifying emails as spam or not spam using a dataset of labeled emails. [0:06]

Pre-training generally does not require labeled data, whereas fine-tuning typically does. [0:06]

In summary, building an LLM involves pre-training on a large, diverse corpus of data, which requires substantial computational power and financial investment. This is followed by fine-tuning on a labeled dataset to adapt the model for specific tasks like classification, summarization, translation, or building custom chatbots. Companies developing specialized LLM applications invariably use fine-tuning, as relying solely on foundational models is often insufficient for production-level needs. [0:06]

The next lecture will introduce the basics of Transformers and briefly discuss the "Attention Is All You Need" paper. [0:06]

---

\*End of Chapter 3. Source: <https://www.youtube.com/watch?v=-bsa3fCNGg4>\*

---

## Chapter 4: Introduction to Transformers

---

*Source video: Lecture 4: What are transformers?*

*Duration: 40 minutes | Playlist position 4*

### The Transformer Architecture: A Foundation for LLMs

The secret sauce behind large language models (LLMs) and their popularity is the Transformer architecture. Most modern LLMs rely on this deep neural network architecture, first introduced in the 2017 paper titled "Attention Is All You Need." This paper has garnered over 100,000 citations, highlighting its significant impact on the field. The GPT architecture, foundational to models like ChatGPT, is heavily based on this paper, though not identical.

[\[0:08\]](#)

The original Transformer was proposed for machine translation tasks, such as converting English to German or French. Text completion, a primary function of GPT, was not the initial focus. The Transformer mechanism introduced in the paper led to significant advancements in translation tasks, and later, it was discovered that architectures derived from it could be used for a wide array of other applications. [\[0:08\]](#)

### A Simplified View of the Transformer

A simplified schematic of the Transformer architecture can be understood through eight key steps. For translation from English to German, the process begins with the input English text. This text is then pre-processed through tokenization, where sentences are broken down into smaller units, often words or subwords, and assigned unique numerical IDs. [\[0:08\]](#)

Following tokenization, these token IDs are passed to the encoder. The encoder's primary role is to perform vector embedding. This process converts the tokens into numerical vectors that capture the semantic meaning and relationships between words. For instance, words like "dog" and "puppy" would have vectors that are closer in the vector space than "dog" and "car," reflecting their semantic similarity. [\[0:08\]](#)

These generated vector embeddings are then fed into the decoder. The decoder also receives the partial output text that has been generated so far, as it produces the translation one word at a time. The decoder's task is to predict the next word in the sequence based on both the input embeddings and the previously generated output words. This process continues until the final translation is complete. [\[0:08\]](#)

The Transformer architecture, in its full form, includes feed-forward layers with weights and parameters that are optimized during training, similar to any neural network. The simplified eight-step process illustrates how input text is transformed into vector embeddings by the

encoder and then used by the decoder, along with partial output, to generate the final translated text word by word. [0:08]

## Encoder and Decoder Blocks

The Transformer architecture is predominantly composed of two main blocks: the encoder and the decoder. The encoder's purpose is to convert the input text into meaningful embedding vectors. The decoder, on the other hand, takes these embedding vectors and the partial output generated so far to produce the final output text. [0:08]

It is important to note that the GPT architecture differs from the original Transformer as it does not have an encoder; it solely utilizes the decoder. Conversely, models like BERT primarily use the encoder component. [0:08]

## The Self-Attention Mechanism

A crucial component of the Transformer architecture is the self-attention mechanism. This mechanism allows the model to weigh the importance of different words and tokens relative to each other within a sequence. It enables the modeling of dependencies between words regardless of their distance in the input or output sequences, addressing the challenge of long-range dependencies. [21:49]

Self-attention calculates an "attention score" that indicates which words should receive more focus when predicting the next word. This capability is vital for accurate next-word prediction, as it allows the model to consider context from distant parts of the text. The paper "Attention Is All You Need" emphasizes this mechanism, as it is central to the Transformer's effectiveness. [21:49]

## Variations: BERT and GPT

Later variations of the Transformer architecture include BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformers). BERT operates by predicting hidden or masked words within a sentence, processing the input bidirectionally (from left to right and right to left). This bidirectional nature makes BERT particularly adept at tasks like sentiment analysis, as it can capture nuances by considering the entire sentence context. [0:08]

GPT models, in contrast, are generative and predict the next word in a sequence, operating in a left-to-right manner. They receive incomplete text and learn to generate one word at a time. GPT models consist only of a decoder, whereas BERT models primarily use an encoder. [0:08]

## Transformers vs. Large Language Models (LLMs)



It is important to distinguish between Transformers and LLMs. While many modern LLMs are based on the Transformer architecture, not all Transformers are LLMs. Transformers can also be applied to other domains, such as computer vision tasks like image recognition and classification, leading to models like Vision Transformers (ViT). [0:08]

Conversely, not all LLMs are necessarily built using the Transformer architecture. LLMs can also be based on recurrent neural networks (RNNs) or convolutional architectures, which were used for sequence modeling and text completion tasks before the advent of Transformers. Therefore, the terms "Transformer" and "LLM" should not be used interchangeably, as they represent different concepts with overlapping but distinct applications. [0:08]

---

\*End of Chapter 4. Source: <https://www.youtube.com/watch?v=NLn4eetGmf8>\*

---

## Chapter 5: The Evolution and Capabilities of GPT Models

---

*Source video: Lecture 5: How does GPT-3 really work?*

*Duration: 48 minutes | Playlist position 5*

### Historical Progression of GPT Research

The journey of Generative Pre-trained Transformer (GPT) models began with the seminal 2017 paper "Attention Is All You Need," which introduced the Transformer architecture and its core self-attention mechanism. This innovation significantly improved the ability to capture long-range dependencies in text, leading to better next-word prediction compared to previous recurrent neural network (RNN) and long short-term memory (LSTM) models. The original Transformer architecture included both an encoder and a decoder. [0:05]

Following this, the first GPT paper in 2018 introduced the concept of generative pre-training. This approach focused on unsupervised learning, leveraging large amounts of unlabeled text. Instead of relying on scarce labeled data, the model was trained to predict the next word within a given sentence, using the sentence itself as both the training and testing data. This method aimed to improve language understanding by training on a diverse corpus of unlabeled text. [0:05]

The GPT architecture, developed after the original Transformer paper, notably omitted the encoder block, relying solely on the decoder. This generative pre-training phase, where the model learns to predict the next word in an unsupervised manner, was a key innovation. OpenAI also published a blog post in June 2018 highlighting this approach, emphasizing the combination of Transformer architecture and unsupervised pre-training to achieve state-of-the-art results on language tasks. [0:05]

In 2019, the GPT-2 paper, "Language Models Are Unsupervised Multitask Learners," was released. This iteration utilized a larger dataset and a generative pre-trained network, introducing models with up to 1.5 billion parameters. GPT-2 marked a significant step in large language model development, with its largest version featuring approximately 1 billion parameters, demonstrating impressive results at the time. [0:05]

The year 2020 saw the release of GPT-3, a model with a staggering 175 billion parameters. GPT-3 demonstrated remarkable capabilities beyond its core training objective of next-word prediction, performing tasks such as translation, sentiment analysis, and question answering. This massive scale led to widespread exploration and recognition of its potential. Subsequently, GPT-3.5 gained commercial traction, and the current iteration, GPT-4, represents the latest advancement in this lineage. [0:05]

### Zero-Shot vs. Few-Shot Learning

A key concept in understanding GPT models is the distinction between zero-shot and few-shot learning. Zero-shot learning refers to a model's ability to generalize to entirely new tasks without any prior specific examples. In contrast, few-shot learning involves the model learning from a minimal number of examples provided as input. [0:05]

The GPT-3 paper provides clear illustrations of these concepts. In zero-shot learning, the model is given only a description of the task, such as translating English to French, without any supporting examples. For instance, translating the word "cheese" to French without prior translation examples demonstrates zero-shot capability. [0:05]

One-shot learning, a variation, provides a single example alongside the task description. For example, showing the model how "sea otter" translates to French before asking it to translate "cheese" illustrates one-shot learning. Few-shot learning extends this by providing a few examples, such as multiple English-to-French translations, to guide the model before it performs the target translation. [0:05]

The authors of the GPT-3 paper claimed it was a few-shot learner, meaning it performed tasks effectively when provided with a few examples. While GPT-3 could perform zero-shot tasks, its responses were often more accurate in a few-shot setting. When queried, GPT-4 identified itself as a few-shot learner, capable of performing tasks better with examples, but also possessing zero-shot capabilities. GPT-4 suggested that providing examples enhances accuracy, making it a strong few-shot learner, while still being able to handle tasks without prior examples. [0:05]

### **Data Utilization in GPT-3**

GPT-3 was trained on a massive dataset comprising approximately 300 billion tokens. The primary source was the Common Crawl dataset, contributing 410 billion tokens, which constituted about 60% of the total training data. Common Crawl is an open repository of web crawl data, spanning 17 years and containing over 250 billion pages. [0:05]

Additionally, GPT-3 utilized data from WebText2, an enhanced version of the original WebText corpus, which includes Reddit submissions from 2015 to 2020. This contributed 19 billion tokens, making up 22% of the training data. The remaining 18-19% of the data came from books and Wikipedia. For simplicity, a token can be considered equivalent to a word, though tokenization is a more complex process. [0:05]

The total pre-training cost for GPT-3 was estimated at \$4.6 million. This significant expense is attributed to the immense computational power required to process such vast amounts of data and optimize the model's 175 billion parameters. [0:05]

### **Pre-training vs. Fine-tuning and Open vs. Closed Source Models**

Pre-trained models like GPT-3 are often referred to as base or foundational models, which can then be further refined through a process called fine-tuning. Fine-tuning involves training the pre-trained model on a smaller, specific dataset to adapt it for particular tasks or domains, often required for production-level applications. While pre-training requires vast amounts of unlabeled data, fine-tuning uses labeled data relevant to the target organization or task.

[\[0:05\]](#)

Many pre-trained large language models (LLMs) are available as open-source, allowing general use without fine-tuning for tasks like text generation or analysis. However, a distinction exists between open-source and closed-source models. Closed-source models, such as GPT-4, offer access primarily through an interface, with their internal parameters and weights not publicly disclosed.

[\[0:05\]](#)

While closed-source models historically offered superior performance, the gap is narrowing. Recent open-source models, like Meta's Llama 3.1 with 405 billion parameters, demonstrate performance comparable to, and in some cases exceeding, models like GPT-4. For students, interacting with readily available models like GPT-3 and GPT-4 is generally sufficient and does not necessitate fine-tuning or direct access to model weights.

[\[0:05\]](#)

## **The GPT Architecture and Autoregression**

The GPT architecture is characterized by its simplicity, consisting solely of a decoder block, unlike the original Transformer which included both an encoder and a decoder. This decoder-only structure is a key differentiator. In GPT-3, this architecture is scaled up, featuring 96 Transformer layers and contributing to its 175 billion parameters.

[\[0:05\]](#)

GPT models are trained on next-word prediction tasks, a process that underpins their autoregressive nature. In this training paradigm, a sentence is divided into an input sequence and a target output, which is the next word in the sequence. This process is unsupervised because the sentence itself provides the "labels" (the subsequent words).

[\[0:05\]](#)

The autoregressive property stems from how the model uses its previous predictions as input for future predictions. For instance, if the model predicts "is" as the next word, that word then becomes part of the input for predicting the subsequent word. This iterative process, where the output of one step informs the input of the next, defines its autoregressive behavior.

[\[0:05\]](#)

Visually, this can be seen through iterations. In iteration one, a single word is input, and the model predicts the next word. This predicted word is then included in the input for iteration two, and so on. This continuous cycle of prediction and incorporation of previous outputs into new inputs is fundamental to how GPT models generate text.

[\[0:05\]](#)

## Emergent Behavior in GPT Models

Emergent behavior refers to the ability of a model to perform tasks it was not explicitly trained for. Despite being trained solely on next-word prediction, GPT models exhibit capabilities in areas like language translation, question answering, and even generating multiple-choice questions. This phenomenon was surprising to researchers, as the models developed these diverse skills organically during the pre-training phase. [0:05]

OpenAI noted that underlying language models could perform tasks without specific training, observing that performance on tasks like multiple-choice questions improved as the language model itself improved. This suggests that the vast scale and unsupervised training allow models to develop a broad understanding of language and reasoning, leading to emergent capabilities. The exact mechanisms behind emergent behavior remain an active area of research. [0:05]

This emergent behavior enables GPT models to perform a wide array of tasks, including text summarization, essay grading, and lesson plan creation, far beyond their initial training objective. The exploration of emergent behavior in LLMs presents significant opportunities for future research and development. [0:05]

---

\*End of Chapter 5. Source: <https://www.youtube.com/watch?v=xbaYCf2FHSY>\*

---

## Chapter 6: Stages of Building an LLM from Scratch

---

*Source video: Lecture 6: Stages of building an LLM from Scratch*

*Duration: 20 minutes | Playlist position 6*

### Roadmap for the LLM Building Series

This lecture outlines the structure and content of the upcoming series on building Large Language Models (LLMs) from scratch. The series will be divided into three distinct stages, moving from foundational concepts and data preparation to pre-training and finally, fine-tuning for specific applications. The goal is to provide a detailed understanding of the entire LLM development process, covering both theoretical underpinnings and practical implementation. [\[0:19\]](#)

The material presented in this series is based on the book "Building a Large Language Model from Scratch" by Sebastian Raschka. The series aims to dedicate multiple lectures to each stage to ensure a thorough grasp of the underlying mechanisms. [\[0:57\]](#)

### Stage 1: Building Blocks of an LLM

Stage one focuses on the essential components required to build an LLM. This includes data pre-processing and sampling, understanding the attention mechanism, and comprehending the LLM architecture. Key topics within data preparation and sampling include tokenization, vector embedding, and constructing data batches. [\[1:26\]](#)

The attention mechanism will be explored in detail, covering concepts such as multi-head attention, masked multi-head attention, positional encoding, input embeddings, and output embeddings. Following this, the series will delve into constructing the LLM architecture, including how to stack layers and the placement of attention heads. The primary objective of stage one is to establish a fundamental understanding of how LLMs operate. [\[1:59\]](#)

Within data preparation and sampling, tokenization will be covered first, explaining how sentences are broken down into individual units. Subsequently, vector embedding will be discussed, focusing on transforming words into high-dimensional vectors to capture semantic meaning, ensuring that words with similar meanings are positioned closer together in the vector space. Positional encoding will also be addressed, highlighting the importance of word order in a sentence. Finally, the process of constructing data batches from large datasets for efficient computation during training will be implemented. [\[2:24\]](#)

The attention mechanism section will provide a comprehensive understanding of its components within the Transformer model, including multi-head attention and masked multi-head attention. This stage concludes with the construction of the LLM architecture,

laying the groundwork for the subsequent stages. [\[3:00\]](#)

## **Stage 2: Pre-training the Foundational Model**

Stage two is dedicated to the pre-training of the LLM. This involves writing code to train the model on a large, unlabeled dataset, resulting in a foundational model. The process includes iterating through epochs, computing gradients, and updating parameters. Text generation for visual inspection and model evaluation will also be performed. [\[3:24\]](#)

Key activities in stage two include text generation, evaluation of training and validation losses, and implementing the LLM training function. Furthermore, functions for saving and loading model weights will be developed to avoid redundant training and computational costs. The stage will also involve loading pre-trained weights from OpenAI into the custom LLM model. [\[3:47\]](#)

The primary goal of stage two is to build a foundational model through pre-training on unlabeled data. This foundational model serves as a base for further customization and application-specific development. [\[4:11\]](#)

## **Stage 3: Fine-tuning for Specific Applications**

Stage three focuses on fine-tuning the pre-trained LLM for specific applications. This involves training the model further on smaller, labeled datasets tailored to particular tasks. The series will demonstrate the creation of two applications: a text classifier and a personal assistant chatbot. [\[4:27\]](#)

An example of a text classifier application involves training the LLM to distinguish between spam and non-spam emails using labeled data. This requires providing the pre-trained model with specific examples of spam and non-spam content to refine its classification capabilities. [\[4:47\]](#)

The second application to be built is a chatbot capable of answering queries based on instructions and inputs. This will also be achieved through fine-tuning the LLM on a dataset designed for conversational interactions. [\[5:03\]](#)

## **Importance of All Stages**

It is crucial for aspiring LLM engineers to understand all three stages, as neglecting stage one and stage two can lead to a lack of confidence in the underlying mechanisms. This series aims to cover every concept in detail across all stages, ensuring a comprehensive learning experience for students, professionals, founders, and managers. [\[5:22\]](#)

The vision for this playlist is to become a highly detailed resource for LLM development. The instructor plans to incorporate feedback from viewers to refine the content over

approximately three to four months, making it a valuable reference for a wide audience. [\[5:43\]](#)

## Recap of Key Concepts

Large Language Models have significantly advanced the field of Natural Language Processing (NLP), enabling sophisticated text generation, understanding, and translation. Unlike previous NLP approaches that required separate algorithms for each task, LLMs exhibit emergent properties, performing well on a variety of tasks such as multiple-choice questions, text summarization, emotion classification, and language translation, even if not explicitly trained for them. [\[6:10\]](#)

Modern LLMs are typically trained in two main steps: pre-training on vast amounts of unlabeled data to create a foundational model, and fine-tuning on smaller, labeled datasets for specific applications. Pre-training requires substantial resources, including large datasets, significant compute power, and considerable financial investment, as exemplified by GPT-3's pre-training cost of \$4.6 million. [\[6:37\]](#)

Fine-tuning is essential for deploying LLMs in production environments, allowing companies to adapt pre-trained models to their specific data and requirements. Unlike the unlabeled, auto-regressive nature of pre-training data, fine-tuning utilizes labeled datasets (e.g., spam/not spam) to improve performance on specialized tasks. Fine-tuned LLMs generally outperform purely pre-trained models on specific tasks. [\[7:05\]](#)

While APIs and interfaces like GPT-4 or Perplexity are suitable for general assistance, building production-ready applications requires fine-tuning on proprietary data. The Transformer architecture, with its core attention mechanism, is fundamental to the power of modern LLMs. The attention mechanism allows models to selectively focus on relevant parts of the input sequence when generating output, considering context from previous sentences as well. [\[7:35\]](#)

It is important to distinguish between the original Transformer architecture, which included both an encoder and a decoder, and the Generative Pre-trained Transformer (GPT) architecture, which solely utilizes the decoder. GPT models, starting from GPT in 2018, followed by GPT-2 and GPT-3 (with 175 billion parameters), and leading to GPT-4, have exclusively used the decoder-only structure. [\[8:03\]](#)

Despite being trained primarily for next-word prediction, LLMs demonstrate remarkable emergent properties, including text classification, translation, and summarization capabilities, which were not explicitly part of their training objective. This effectiveness of the pre-training stage enables LLMs to be impactful across a wide array of applications. [\[8:27\]](#)



The upcoming lectures will transition into the hands-on aspects, starting with stage one: data preparation and sampling. This will involve working with text data, loading datasets, counting characters, and tokenizing data, with the introduction of Jupiter notebooks for parallel coding. The series aims to blend theoretical explanations with practical coding implementations.

[8:47]

\*End of Chapter 6. Source: <https://www.youtube.com/watch?v=z9fgKz1Drlc>\*

---

## Chapter 7: Tokenization from Scratch

---

*Source video: Lecture 7: Code an LLM Tokenizer from Scratch in Python*

*Duration: 69 minutes | Playlist position 7*

### Introduction to Tokenization

Tokenization is a fundamental pre-processing step in building large language models (LLMs). It involves breaking down text into smaller units, known as tokens, which can then be converted into numerical representations that neural networks can process. This process is crucial before the data is used for pre-training LLMs. [0:05]

The overall process of building an LLM can be divided into three stages: understanding the basic mechanisms (including data preparation), pre-training the foundational model, and fine-tuning for specific applications. Tokenization falls under the data preparation aspect of the first stage. [0:05]

At its core, tokenization prepares input text for LLMs by converting raw text into a sequence of tokens and then into numerical token IDs. This is a necessary precursor to vector embedding, where these IDs are transformed into numerical vectors. [0:05]

The tokenization process can be broadly broken down into three steps: splitting text into individual word and subword tokens, converting these tokens into token IDs, and finally encoding these token IDs into vector representations. This lecture will focus on the first two steps. [0:05]

### Step 1: Tokenizing Text into Words

The first step in tokenization is to split the input text into individual words or tokens. For example, the sentence "This is an example" would be tokenized into "This", "is", "an", and "example". [0:05]

The second step involves converting each of these tokens into a unique numerical identifier, known as a token ID. For instance, "This" might become ID 1, "is" ID 2, "an" ID 3, and "example" ID 4. These token IDs are then used for further processing, such as creating token embeddings. [0:05]

For demonstration purposes, this lecture uses the book "The Verdict" by Edith Wharton, published in 1908, as the training dataset. This book is freely available and serves as a manageable example for understanding tokenization. [0:05]

The process begins by loading the text file into Python. The `open()` function is used to read the file, and the content is stored in the `raw_text` variable. The total number of characters and the first 100 characters are then printed to verify successful loading. [0:05]

```
# Example of loading and reading a text file
with open("verdict.txt", "r") as f:
    raw_text = f.read()

print(f"Total characters: {len(raw_text)}")
print(f"First 100 characters: {raw_text[:100]}")
```

[0:05]

The goal is to tokenize all characters in the book into individual words that can later be converted into embeddings for LLM training. While LLMs are typically trained on millions of documents, a single book is sufficient for educational purposes to demonstrate the concept.

[0:05]

To split the text into tokens, Python's `re` (regular expression) module is utilized. The `re.split()` function can split text based on specified patterns, such as whitespace characters.

[0:05]

Initially, splitting by whitespace (`\s`) separates words but keeps punctuation attached to them (e.g., "hello,"). To separate punctuation marks like commas and periods as distinct tokens, they can be included in the `re.split()` pattern. [0:05]

Whitespace characters themselves can also be included as tokens in the split result. To remove these redundant whitespace tokens, a list comprehension with `item.strip()` can be used to filter them out, ensuring that only meaningful tokens remain. [0:05]

The decision to remove or keep whitespace tokens depends on the application. For text where spacing is significant, like Python code, whitespace should be preserved. However, for simplicity and memory efficiency in this example, whitespace tokens are removed. [0:05]

The tokenization scheme can be further refined by including more punctuation and special characters in the `re.split()` pattern, such as colons, semicolons, question marks, quotation marks, and brackets. This ensures that these characters are also treated as individual tokens.

[0:05]

The refined tokenization process involves splitting the text based on a set of delimiters (whitespace, punctuation) and then filtering out any remaining whitespace tokens. This results in a list of meaningful tokens from the text. [0:05]

```
import re

# Example text
text = "Hello, world! This is a test."

# Split by whitespace, comma, and period
tokens_with_punctuation = re.split(r'[,\s.!]', text)

# Filter out empty strings resulting from split
```

```
tokens = [token for token in tokens_with_punctuation if token]

print(tokens)
```

[0:05]

This tokenization scheme can then be applied to the entire `raw_text` of the book. The resulting list, stored in the `preprocessed` variable, contains all the individual tokens from the book. For the "Verdict" book, this results in approximately 4690 tokens. [0:05]

## Step 2: Converting Tokens to Token IDs

The second crucial step in tokenization is converting the obtained tokens into numerical token IDs. This is necessary because LLMs, being neural networks, operate on numerical data.

[0:05]

This conversion process involves creating a vocabulary, which is a list of all unique tokens present in the dataset, sorted alphabetically. Each unique token in this vocabulary is then assigned a unique integer ID, starting from 0. [0:05]

For example, if the tokens are "the", "quick", "brown", "fox", the vocabulary might be ["brown", "fox", "quick", "the"]. Assigning IDs would result in "brown": 0, "fox": 1, "quick": 2, "the": 3. [0:05]

The vocabulary size for the "Verdict" book, after processing, is determined to be 1130 unique tokens. This vocabulary acts as a dictionary mapping each token to its corresponding ID.

[0:05]

Python's `set()` function can be used to find unique tokens, and sorting them alphabetically creates the vocabulary. The `enumerate()` function is then used to efficiently assign integer IDs to each token in the sorted vocabulary. [0:05]

```
# Assuming 'preprocessed' is the list of tokens from Step 1
unique_tokens = set(preprocessed)
vocabulary = sorted(list(unique_tokens))
vocab_size = len(vocabulary)

# Create a mapping from token to token ID
token_to_id = {token: i for i, token in enumerate(vocabulary)}

print(f"Vocabulary size: {vocab_size}")
print(f"First 50 vocabulary items: {vocabulary[:50]}")
```

[0:05]

This process creates a mapping from tokens to token IDs. For the reverse process, converting token IDs back to tokens, an inverse mapping (ID to token) is also needed, which will be used in the decoder. [0:05]

## Implementing a Tokenizer Class

To encapsulate the tokenization and de-tokenization logic, a `Tokenizer` class is implemented. This class includes two primary methods: `encode` and `decode`. [0:05]

The `__init__` method initializes the tokenizer with a vocabulary, which is a mapping from string tokens to integer IDs (`str_to_int`). It also creates the reverse mapping (`int_to_str`) for decoding. [0:05]

The `encode` method takes raw text, applies the same pre-processing steps (splitting into tokens and removing whitespace), and then converts these tokens into their corresponding IDs using the `str_to_int` mapping. [0:05]

The `decode` method performs the reverse operation. It takes a list of token IDs, converts them back into tokens using the `int_to_str` mapping, and then joins these tokens to reconstruct the original text. It also includes logic to handle spacing around punctuation for better readability. [0:05]

```
class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i: token for token, i in vocab.items()}

    def encode(self, text):
        # Pre-processing: split text into tokens
        tokens = re.split(r'[, \s.!?\'":\-\;\(\)\[\]\{\}<>]', text)
        tokens = [token for token in tokens if token] # Remove empty strings

        # Convert tokens to token IDs
        token_ids = [self.str_to_int.get(token, -1) for token in tokens] # Use -1 for unknown tokens
        return token_ids

    def decode(self, token_ids):
        # Convert token IDs back to tokens
        tokens = [self.int_to_str.get(id, '[UNK]') for id in token_ids]

        # Join tokens and handle spacing around punctuation
        decoded_text = ""
        for i, token in enumerate(tokens):
            if i > 0 and token not in ['.', ',', '!', '?', ';', ':', ')', ']', '}', '']:
                decoded_text += " "
            decoded_text += token

        # Basic cleanup for spaces before punctuation
        decoded_text = re.sub(r'\s+([.,!?:;])', r'\1', decoded_text)
        return decoded_text

# Example usage:
# Assuming 'token_to_id' is the vocabulary dictionary created earlier
# tokenizer = SimpleTokenizerV1(token_to_id)
```

```
# encoded_text = tokenizer.encode("This is a sample text.")
# decoded_text = tokenizer.decode(encoded_text)
# print(decoded_text)
```

[0:05]

Testing the encode method with a passage from the book successfully converts text into token IDs. The decode method then correctly reconstructs the original text from these IDs, demonstrating the tokenizer's functionality. [0:05]

## Handling Unknown Words and Special Tokens

A challenge arises when the input text contains words not present in the tokenizer's vocabulary. In the current implementation, attempting to encode such text results in an error because the word cannot be mapped to a token ID. [0:05]

To address this, LLMs often employ special context tokens. Two important ones are the "unknown" token (<UNK>) and the "end of text" token (<EOS>). [0:05]

The <UNK> token is assigned an ID and used to represent any word not found in the vocabulary. This prevents errors and allows the model to process text with out-of-vocabulary words. [0:05]

The <EOS> token is used to mark the end of a text segment. When dealing with multiple independent documents or text sources, <EOS> tokens are inserted between them. This helps the LLM distinguish between different pieces of context and process them more effectively. [0:05]

The vocabulary is augmented by adding these special tokens. For instance, <EOS> might be assigned ID 1130 and <UNK> ID 1131, increasing the vocabulary size accordingly. [0:05]

A modified tokenizer class (SimpleTokenizerV2) is implemented to handle unknown words. During encoding, if a token is not found in the vocabulary, it is replaced with the <UNK> token. [0:05]

```
class SimpleTokenizerV2:
    def __init__(self, vocab, eos_token="<EOS>", unk_token="<UNK>"):
        self.eos_token = eos_token
        self.unk_token = unk_token

        # Add special tokens to vocabulary if not already present
        if eos_token not in vocab:
            vocab[eos_token] = len(vocab)
        if unk_token not in vocab:
            vocab[unk_token] = len(vocab)

        self.str_to_int = vocab
        self.int_to_str = {i: token for token, i in vocab.items()}
```

```

def encode(self, text):
    # Pre-processing: split text into tokens
    tokens = re.split(r'[\s.!?"':\-\;\(\)\[\]\{\}<>]', text)
    tokens = [token for token in tokens if token]

    # Convert tokens to token IDs, using UNK for unknown words
    token_ids = [self.str_to_int.get(token, self.str_to_int[self.unk_token]) for token in tokens]
    return token_ids

def decode(self, token_ids):
    tokens = [self.int_to_str.get(id, self.unk_token) for id in token_ids]

    decoded_text = ""
    for i, token in enumerate(tokens):
        if token == self.eos_token:
            continue # Skip EOS token during decoding for simple text reconstruction
        if i > 0 and token not in ['.', ',', '!', '?', ';', ':', ')', ']', '}', '']:
            decoded_text += " "
        decoded_text += token

    decoded_text = re.sub(r'\s+([\.,!?:;])', r'\1', decoded_text)
    return decoded_text

# Example usage with special tokens:
# Assuming 'token_to_id' is the vocabulary dictionary
# tokenizer_v2 = SimpleTokenizerV2(token_to_id.copy()) # Use a copy to not modify original
# text_with_unknown = "Hello, do you like tea?"
# encoded_ids = tokenizer_v2.encode(text_with_unknown)
# print(f"Encoded IDs: {encoded_ids}")
# decoded_text = tokenizer_v2.decode(encoded_ids)
# print(f"Decoded Text: {decoded_text}")

```

[0:05]

When encoding text containing unknown words like "hello", the tokenizer now replaces it with the <UNK> token ID instead of raising an error. Similarly, <EOS> tokens can be incorporated into the input sequence. [0:05]

While this implementation uses <UNK> and <EOS> tokens, it's noted that advanced LLMs like GPT-3 and GPT-4 often use a more sophisticated tokenization method called Byte Pair Encoding (BPE). BPE breaks words down into subword units, which inherently handles unknown words without needing a dedicated <UNK> token. [0:05]

Other special tokens sometimes used in LLMs include Beginning of Sequence (<BOS>), End of Sequence (<EOS>), and Padding (<PAD>). Padding tokens are used to ensure all sequences in a batch have the same length for efficient parallel processing. However, GPT models primarily rely on the <EOS> token for managing different text sources. [0:05]

The lecture concludes by summarizing the key concepts: tokenization involves splitting text into tokens and converting them to token IDs. The implementation covered creating a vocabulary, building an encode and decode method, and handling unknown words and special tokens like <UNK> and <EOS>. The next lecture will delve into Byte Pair Encoding.

[0:05]

\*End of Chapter 7. Source: <https://www.youtube.com/watch?v=rsy5Ragmso8>\*

---



## Chapter 8: The GPT Tokenizer: Byte Pair Encoding

---

*Source video: Lecture 8: The GPT Tokenizer: Byte Pair Encoding*

*Duration: 53 minutes | Playlist position 8*

### Introduction to Tokenization and Byte Pair Encoding

The process of tokenization is crucial for large language models (LLMs), and a sophisticated method known as Byte Pair Encoding (BPE) is commonly employed by modern LLMs like GPT-2 and GPT-3. Understanding BPE is essential for comprehending how these models process text. [\[0:05\]](#)

Previously, a simpler tokenization scheme was implemented where sentences were converted into tokens, and then these tokens were arranged to form a vocabulary, assigning a unique numerical ID to each token. This approach treated each word, along with special characters, as a distinct token. [\[1:15\]](#)

This lecture delves into a more advanced tokenization scheme: Byte Pair Encoding (BPE). This method is significant because it was used to train influential LLMs such as GPT-2, GPT-3, and the original model behind ChatGPT. A prior understanding of basic tokenization, as covered in the previous lecture, will greatly aid in grasping the concepts presented here. [\[1:35\]](#)

### Types of Tokenization Algorithms

There are primarily three types of tokenization algorithms: word-based, subword-based, and character-based. [\[2:18\]](#)

In word-based tokenization, each word in a sentence is treated as a single token. For instance, the sentence "The fox chased the dog" would be tokenized into "The", "fox", "chased", "the", and "dog". Similarly, "My hobby is playing cricket" would yield tokens "My", "hobby", "is", "playing", and "cricket". [\[2:35\]](#)

A major drawback of word-based tokenization is the handling of out-of-vocabulary (OOV) words ? words not present in the model's vocabulary. If a user inputs a word not seen during training, the model struggles to process it, potentially leading to errors. Another issue is that semantically similar words, like "boy" and "boys," are treated as entirely separate tokens, failing to capture their shared root and meaning. [\[3:00\]](#)

Character-based tokenization, on the other hand, treats each individual character as a token. For the sentence "My hobby is playing cricket," the tokens would be 'M', 'y', ' ', 'h', 'o', 'b', 'b', 'y', and so on. This approach results in a very small vocabulary size, as the number of characters in a language is finite (e.g., around 256 for English). [\[4:00\]](#)

The primary advantage of character-based tokenization is the elimination of the out-of-vocabulary problem, as any word can be broken down into known characters. However, it suffers from a significant disadvantage: the loss of word meaning. Breaking words into individual characters obscures their semantic relationships and root structures. Additionally, character-based tokenization leads to much longer token sequences compared to the original text, as a single word like "dinosaur" becomes eight separate tokens ('d', 'i', 'n', 'o', 's', 'a', 'u', 'r'). [4:45]

## Subword Tokenization and Byte Pair Encoding

Subword-based tokenization aims to strike a balance between word-based and character-based approaches. It retains frequently occurring words as single tokens while breaking down rare words into smaller, meaningful subword units. This strategy helps in capturing root words and common subword structures. [6:00]

The core idea is that common words are kept whole, while less frequent words are split. For example, "boy" might remain a single token if it appears frequently, but "boys" could be split into "boy" and "s," recognizing that "boy" is a common root and "s" often signifies plurality. This approach allows the model to understand relationships between words like "token," "tokens," and "tokenizing" by identifying the common root "token." It also helps in recognizing common suffixes like "ization" in words like "tokenization" and "modernization." [6:45]

Byte Pair Encoding (BPE) is a prominent subword tokenization algorithm. Originally introduced in 1994 as a data compression algorithm, BPE works by iteratively identifying the most frequent pair of consecutive bytes (or characters/tokens) in a dataset and replacing them with a new, unused byte (or token). This process is repeated until a desired level of compression or vocabulary size is achieved. [7:30]

## Practical Demonstration of Byte Pair Encoding

Consider a simple dataset of words: "old," "older," "finest," and "lowest." To prepare for BPE, each word is augmented with a special end-of-word token, denoted as `</w>`, resulting in "old</w>," "older</w>," "finest</w>," and "lowest</w>". [9:45]

The first step involves creating a frequency table of all characters and the `</w>` token. For instance, 'e' appears 16 times, 's' appears 13 times, and `</w>` appears 23 times. [10:30]

In the first iteration of BPE, the most frequent consecutive pair is identified. In this example, 'e' and 's' form the pair 'es', appearing 13 times across "finest</w>" and "lowest</w>". This pair 'es' is then merged into a new token. The counts for 'e' and 's' are adjusted, and 'es' is added to the vocabulary. [11:15]

In the second iteration, the algorithm looks for the next most frequent pair using the newly formed tokens. The pair 'es' and 't' form 'est', appearing 13 times in "finest</w>" and "lowest</w>". 'est' is merged into a new token, and the counts for 'es' and 't' are updated. This process continues, merging 'est' with </w> to form 'est</w>', which also appears 13 times. This step is crucial as it captures the common suffix 'est' and the end-of-word marker, helping to differentiate words like "highest" from "estimate." [12:30]

Further iterations might merge 'o' and 'l' into 'ol', and then 'ol' and 'd' into 'old', capturing the root word "old" which appears in both "old" and "older." The process stops when a predefined condition is met, such as a specific vocabulary size or number of iterations. The resulting vocabulary contains subword tokens like "old" and "est</w>", which are neither full words nor individual characters but meaningful units that capture root meanings and word structures. [14:30]

The final vocabulary derived from this process might include tokens like "old", "est</w>", and others, effectively representing the input words as sequences of these subword units. This subword representation allows the model to handle variations of words and capture semantic similarities that word-based or character-based tokenization would miss. [16:00]

## Advantages of Byte Pair Encoding

BPE offers several advantages over simpler tokenization methods. Firstly, it effectively handles out-of-vocabulary words by breaking them down into known subword units or characters, thus preventing errors. Secondly, it reduces the overall vocabulary size compared to word-based tokenization, making models more computationally efficient. For instance, GPT-2 uses a BPE tokenizer with approximately 50,000 tokens, a significant reduction from the potential hundreds of thousands of words in a typical English vocabulary. [17:30]

Thirdly, BPE captures the root meanings of words and common subword structures, enabling the model to better understand semantic relationships and linguistic patterns. This is a significant improvement over both word-based and character-based tokenization, which fail to preserve these nuances. [18:00]

## Implementing BPE with tiktoken

Implementing BPE from scratch can be complex. For practical purposes, libraries like tiktoken, developed by OpenAI, provide efficient BPE implementations. tiktoken is used by OpenAI models for tokenizing text. [19:00]

After installing tiktoken, a tokenizer instance can be created using tiktoken.get\_encoding("gpt2"). This instance provides encode and decode methods, similar to custom tokenizers. The encode method converts text into token IDs, while the decode method converts token IDs back into text. [20:30]

When encoding a sentence like "Hello do you like Tea <|endoftext|> In the sunlight Terraces of some unknown place," the BPE tokenizer correctly handles the text, including the special <|endoftext|> token and the unfamiliar words "sunlight Terraces" and "some unknown place." The tokenizer breaks down these unfamiliar words into subword units or characters, assigning them token IDs without encountering errors. [21:30]

The vocabulary size for the GPT-2 BPE tokenizer is 50,257, with the token ID 50,256 representing the <|endoftext|> token. This manageable vocabulary size, combined with the ability to handle unknown words and capture semantic meaning, makes BPE a powerful tokenization strategy for LLMs. [22:30]

The BPE tokenizer's ability to handle unknown words stems from its decomposition into subwords or characters. Even completely random sequences like "a k w i r w i e r" can be encoded into token IDs because the tokenizer can break them down into known subword units or individual characters present in its vocabulary. Decoding these IDs reconstructs the original sequence. [24:00]

This lecture has provided an intuitive understanding of Byte Pair Encoding, its implementation using tiktoken, and its advantages in handling unknown words and capturing linguistic meaning. The next steps in the LLM progression will involve data sampling, batch sizes, and context length before moving on to token embeddings. [25:00]

---

\*End of Chapter 8. Source: <https://www.youtube.com/watch?v=fKd8s29e-l4>\*

---

## Chapter 9: Creating Input-Target Data Pairs using Python DataLoader

---

*Source video: Lecture 9: Creating Input-Target data pairs using Python DataLoader*

*Duration: 55 minutes | Playlist position 9*

### Understanding Input-Target Pairs for LLMs

In the previous lecture, we explored Byte Pair Encoding (BPE) for subword tokenization, a technique used by models like GPT-2, GPT-3, and GPT-4. This lecture focuses on the data preprocessing stage, specifically creating input-target pairs, which is a crucial step before moving to vector embeddings and model training. [0:05]

Unlike classification or regression tasks where input-output pairs are often straightforward (e.g., images of cats/dogs as input, labels as output; house area as input, price as output), creating these pairs for large language models (LLMs) involves a specific technique. This lecture will detail this process. [0:05]

For LLMs, input-target pairs are generated by breaking down a sentence into sequential chunks. The blue-marked tokens represent the input to the LLM, and the red-marked token represents the target or the output the LLM must predict. This process is iterative: the output of one iteration becomes part of the input for the next. [0:05]

For example, if the sentence is "llms learn to predict one word at a time," in the first iteration, "llm" might be the input, and "learn" the target. In the second iteration, "llms learn" becomes the input, and "to" becomes the target. This pattern continues, with the LLM always predicting the next word in the sequence. [0:05]

The number of words provided as input is known as the "context length." The output length is always one word, representing the next word to be predicted. Words following the target word are masked, meaning the LLM cannot access them during prediction. [0:05]

This method is called "autoregressive" because the output of one step feeds into the next. It's also considered self-supervised or unsupervised learning because the sentence structure itself provides the labels, eliminating the need for manual annotation. [0:05]

### Implementing Input-Target Pairs in Python

To begin coding, we'll use the "The Verdict" short story as our dataset. This text will be tokenized using the BPE tokenizer from the previous lecture. The tokenizer converts text into token IDs. After encoding the entire text, we obtain a sequence of token IDs. [0:05]

For demonstration purposes, the first 50 tokens can be removed to focus on a more interesting passage. The length of the encoded text indicates the total number of tokens, and the vocabulary size (e.g., 5145) represents the number of unique tokens and their corresponding IDs. [0:05]

The simplest way to create input-target pairs is to define two arrays:  $x$  for inputs and  $y$  for targets.  $y$  is essentially  $x$  shifted by one position. For example, if  $x$  is [1, 2, 3, 4], then  $y$  would be [2, 3, 4, 5]. [0:05]

The "context size" determines how many tokens are included in the input. If the context size is four, the input might be tokens 1-4, and the target would be token 5. This means that for a single input-output pair in the context of LLMs, there can be multiple prediction tasks, equal to the context size. [0:05]

We can iterate through the token IDs, creating context chunks for input and shifted chunks for targets. For instance, if the context size is four, the first input chunk could be the first four token IDs, and the corresponding target would be the token ID at the fifth position. This process is repeated, sliding the window across the tokenized text. [0:05]

Decoding these token IDs back into text helps visualize the process. If "and" is the input, "established" is the target. If "and established" is the input, "himself" is the target, and so on. This demonstrates how the input-target pairs are constructed for next-word prediction. [0:05]

## Utilizing PyTorch DataLoader for Efficient Data Handling

While we can manually create input-output pairs, a more structured and efficient approach is needed for larger datasets and parallel processing. This is where PyTorch's `Dataset` and `DataLoader` classes come into play. [0:05]

The goal is to implement a `DataLoader` that returns input and target tensors. A `Dataset` class is defined, which takes the text, tokenizer, maximum length (context size), and stride as arguments. The `__init__` method processes the text to create input and target ID arrays. [0:05]

The process involves chunking the data. For a given stride, an input chunk is created, and a corresponding target chunk is generated by shifting the input chunk by one position. These chunks are appended to input and target ID tensors. The loop continues until the end of the dataset is reached, ensuring that the last chunk does not exceed the dataset length. [0:05]

The `__getitem__` method within the `Dataset` class is crucial. It allows the `DataLoader` to fetch specific input and target pairs based on an index. This enables the `DataLoader` to efficiently retrieve data in a map-style format. [0:05]

The `stride` parameter determines how much the window slides between creating consecutive input-output pairs. A stride of one means the next input starts just one token after the previous one, leading to significant overlap. A larger stride, such as the context length, ensures no overlap between consecutive input batches, which can help prevent overfitting. [0:05]

The `DataLoader` then takes this `Dataset` instance and handles batching and parallel processing. The `batch_size` determines how many input-output pairs are processed together before the model's parameters are updated. The `num_workers` parameter allows for parallel computation across multiple CPU threads, speeding up data loading. [0:05]

## Understanding Batch Size, Stride, and Workers

When testing the `DataLoader` with a batch size of one and a context size of four, the output consists of an input tensor and a target tensor. The input tensor contains sequences of four token IDs, and the target tensor contains the corresponding next token IDs. This structure mirrors the manually created input-output pairs but is now managed efficiently by the `DataLoader`. [46:12]

The `stride` significantly impacts how data is sampled. A stride of one results in overlapping input batches, potentially leading to overfitting. A stride equal to the context length (e.g., four) creates non-overlapping input batches, which can be beneficial for training stability. [46:12]

The `batch_size` is a hyperparameter that balances the speed of parameter updates with their noise level. Smaller batch sizes lead to quicker but potentially noisier updates, while larger batch sizes result in smoother but slower updates. [46:12]

Increasing the batch size to eight, for example, means the model processes eight input-output pairs before updating its parameters. When combined with a stride of four, this ensures that each of the eight input sequences within the batch is distinct and avoids overlap, helping to prevent overfitting. [46:12]

In summary, we have learned to break down text into input-target pairs, understanding the role of context size and autoregression. We then implemented this process using PyTorch's `Dataset` and `DataLoader`, gaining control over parameters like `batch_size`, `stride`, and `num_workers` for efficient data handling and model training. [52:01]

\*End of Chapter 9. Source: <https://www.youtube.com/watch?v=iQZFH8dr2yI>\*

## Chapter 10: Token Embeddings

---

*Source video: Lecture 10: What are token embeddings?*

*Duration: 60 minutes | Playlist position 10*

### Introduction to Token Embeddings

The process of building large language models involves several key steps, starting with input text. This text is first broken down into tokens, which can be words, subwords, or even characters. These tokens are then converted into numerical token IDs. The crucial third step is transforming these token IDs into token embeddings, which serve as the input for training the language model. This lecture focuses on understanding token embeddings, why they are necessary, and how they are created. [0:05]

### The Need for Token Embeddings

Computers cannot directly understand words; they require numerical representations. While token IDs can serve as numerical representations, they fail to capture the semantic relationships between words. For instance, assigning arbitrary numbers to "cat" and "kitten" does not reflect their inherent semantic similarity. This is analogous to how convolutional neural networks excel because they encode spatial relationships between pixels in an image, rather than just treating pixel values as a flat vector. Similarly, text has inherent meaning and relationships between words that need to be exploited for optimal language model training. [0:05]

One-hot encoding, another method of representing words numerically, also falls short. In one-hot encoding, each word is represented by a vector with all zeros except for a single one at a unique position. This approach, like random number assignment, does not encode the semantic relationship between words, making it difficult for models to understand that "dog" and "puppy" are more closely related than "dog" and "banana." [0:05]

### Vector Embeddings: Capturing Semantic Meaning

The idea of representing words as vectors emerged as a solution to capture semantic meaning. By assigning a vector to each word, where the vector's dimensions represent features, semantic relationships can be encoded. For example, features like "has a tail," "is eatable," "has four legs," "makes sound," and "is a pet" can be used to create vectors for words like "dog," "cat," "apple," and "banana." Words with similar meanings, such as "dog" and "cat," will have similar vector representations, with high values for shared features and low values for differing features. Conversely, words with dissimilar meanings, like "dog" and "banana," will have vectors that are farther apart. This vector representation allows for grouping similar words and understanding the semantic distance between them. [0:05]



These vector representations are also known as token embeddings because each token is converted into a vector embedding. The key takeaway is that vectors can effectively capture semantic meaning, which is crucial for language models. [0:05]

## Creating Token Embeddings

The process of creating effective token embeddings involves training a neural network. This training uses text data to learn which words are closer in meaning and should therefore have similar vector representations. For large vocabularies, such as the 50,000 tokens in GPT-2, this training process is computationally intensive. [0:05]

## Hands-On Demo with Pre-trained Embeddings

Many organizations provide pre-trained token embeddings, such as the Word2Vec Google News 300 model. This model was trained on a massive dataset of 100 billion words, mapping each word to a 300-dimensional vector. These pre-trained embeddings can capture semantic meaning, as demonstrated by vector arithmetic. For instance, the operation "king" + "woman" - "man" results in a vector that is most similar to "queen," indicating that the embeddings encode gender and royalty relationships. [27:51]

Similarly, calculating the similarity between vectors for related words like "woman" and "man," or "king" and "queen," yields high similarity scores. Conversely, unrelated words like "paper" and "water" have low similarity scores. The magnitude of the difference between two word vectors also serves as an indicator of their semantic closeness. [27:51]

## Constructing Embedding Matrices for Large Language Models

To create token embeddings for large language models, two key parameters are needed: the vocabulary size and the vector dimension. For GPT-2, the vocabulary size was 50,257, and the embedding dimension was 768. This results in an embedding layer weight matrix with 50,257 rows and 768 columns. [27:51]

Initially, the weights of this embedding matrix are initialized randomly. These weights are then optimized during the language model's training process using backpropagation. The training data provides the information about word relationships, guiding the optimization of each weight parameter to ensure that similar words have similar vector embeddings. [27:51]

## The Embedding Layer as a Lookup Table

The embedding layer can be conceptualized as a lookup table. Given a vocabulary size and an embedding dimension, it creates a weight matrix where each row corresponds to the vector embedding of a specific token ID. To obtain the embedding for a particular token ID, the layer simply retrieves the corresponding row from this matrix. This lookup operation can be performed for single token IDs or for a batch of IDs simultaneously. [34:48]

While a neural network's linear layer can perform a similar function, the embedding layer is more computationally efficient. This is because the linear layer involves numerous multiplications with zeros when using one-hot encoded inputs, which are unnecessary computations, especially with large vocabularies. [34:48]

## **Summary of Token Embeddings**

Token embeddings are essential for representing words numerically in a way that captures their semantic meaning. Unlike random number assignments or one-hot encoding, vector embeddings allow models to understand relationships between words. The creation of these embeddings involves initializing a weight matrix randomly and then optimizing it during the language model's training process. The embedding layer functions as a lookup table, efficiently retrieving the vector representation for each token ID. While this lecture focused on the conceptual understanding and creation of token embeddings, future lectures will delve into training these embeddings and incorporating positional information. [27:51]

\*End of Chapter 10. Source: [https://www.youtube.com/watch?v=ghCSGRgVB\\_o](https://www.youtube.com/watch?v=ghCSGRgVB_o)\*

---

# Chapter 11: Positional Encoding

---

*Source video: Lecture 11: The importance of Positional Embeddings*

*Duration: 48 minutes | Playlist position 11*

## Introduction and Recap

This lecture focuses on positional encoding, a crucial topic in building large language models. We will first review previously covered concepts, then delve into the necessity of positional encoding, and finally perform a hands-on coding exercise to integrate a positional encoding layer with token embeddings. It is highly recommended to review the previous lecture on token embeddings, as positional encoding builds upon that foundation. [\[0:05\]](#)

Previously, we discussed token embedding, which is the third step in the LLM training process. This step involves converting token IDs, obtained from text tokenization, into vectors. This conversion is vital for retaining the semantic meaning between words. Without token embeddings, the relationship between words like "dog" and "puppy" or "cat" and "kitten" would be lost. For instance, GPT-2 was trained on a vocabulary of approximately 50,000 tokens, each represented by a vector in a high-dimensional space. [\[3:47\]](#)

## The Need for Positional Encoding

While token embeddings capture semantic meaning, they do not inherently account for the order of words in a sentence. Consider the sentences "The cat sat on the mat" and "On the mat the cat sat." The token "cat" appears in both, and its token embedding would be identical. However, its position differs, potentially altering the sentence's meaning. Without positional information, the model treats these sentences as if they contained the same sequence of word meanings, ignoring the positional nuances. [\[5:40\]](#)

The token embedding layer maps each token ID to a unique vector. If the same token ID appears multiple times in a sequence, it will always be mapped to the same vector, regardless of its position. This means that the model does not leverage the information conveyed by the order of tokens. To address this, positional information must be injected into the model alongside the token embeddings. [\[7:10\]](#)

## Types of Positional Embeddings

There are two primary types of positional embeddings: absolute and relative. Absolute positional embedding assigns a unique embedding to each position in the input sequence. This positional embedding is then added to the token embedding of the word at that position. For example, if "cat" appears at position two in one sentence and position five in another, it will receive different positional embeddings, resulting in distinct final vector representations for

"cat" in each sentence. [9:00]

In absolute positional embedding, the positional vectors have the same dimension as the token embeddings. This is necessary to allow for element-wise addition. If the positional embedding had a different dimension, it could not be directly added to the token embedding. [12:00]

Relative positional embedding, on the other hand, focuses on the distance or relative position between tokens rather than their absolute positions. This approach allows the model to generalize better to sequences of varying lengths, as it learns relationships based on how far apart tokens are, not their exact location. This is particularly beneficial for very long sequences where understanding the relationships between distant words is crucial. [13:00]

Both absolute and relative positional embeddings are valuable because they enable large language models to understand the order and relationships between tokens, leading to more accurate and context-aware predictions. [15:00]

## Practical Considerations and Implementation

Absolute positional encoding is generally preferred for tasks where the fixed order of tokens is critical, such as sequence generation. Both GPT models and the original Transformer paper utilized absolute positional encoding. GPT models, in particular, use absolute positional embeddings that are optimized during the training process. [17:00]

The values within the positional embedding vectors, much like token embeddings, are not predetermined. They are optimized during the LLM's training process. This optimization is an integral part of training, alongside the optimization of token embeddings. [18:30]

The original Transformer paper proposed a formula using sinusoidal and cosine functions to calculate positional embeddings. However, GPT models, including GPT-2 and GPT-3, optimize these positional embedding values directly during training without relying on a predefined formula. [19:30]

## Hands-On: Implementing Positional Embeddings

For a practical demonstration, we will consider a vocabulary size of 50,257, consistent with GPT-2's byte-pair encoder tokenizer. The input tokens will be embedded into a 256-dimensional vector space. This dimension is smaller than that used in larger models like GPT-3 (11,288 dimensions) but is of a similar order of magnitude to some smaller GPT-2 models. [22:22]

We begin by creating a token embedding layer using `torch.nn.Embedding`. This layer requires the vocabulary size (50,257) and the output dimension (256). This embedding layer acts as a lookup table, converting token IDs into their corresponding 256-dimensional vector

representations. [24:00]

Next, we utilize a data loader to manage input data. For this example, we set a batch size of 8 and a context size of 4. This means each input to the model will consist of 4 token IDs, and the model will process 8 such sequences in parallel. The data loader processes raw text, chunking it into batches of token IDs, where each sequence has a maximum length of four tokens. [26:00]

When these input token IDs (an 8x4 tensor) are passed through the token embedding layer, the output is an 8x4x256 tensor. This signifies that each of the 32 token IDs (8 sequences \* 4 tokens/sequence) is represented by a 256-dimensional vector. [29:00]

Now, we create a positional embedding layer. Since our context size is 4, we only need to encode four positions. The positional embedding layer will have 4 rows (for the four positions) and 256 columns (matching the token embedding dimension). This layer also functions as a lookup table. [31:00]

To generate the positional embedding vectors, we use `torch.arange(max_length)` to create a tensor of positions [0, 1, 2, 3]. These position indices are then used to look up the corresponding 256-dimensional positional vectors from the positional embedding layer. This results in a 4x256 tensor containing the positional embeddings for each of the four positions. [33:00]

The final step is to add the positional embeddings to the token embeddings. The token embeddings tensor has a shape of 8x4x256, while the positional embeddings tensor has a shape of 4x256. Through broadcasting, PyTorch expands the positional embeddings to match the token embeddings' dimensions. Specifically, the 4x256 positional embedding tensor is effectively duplicated 8 times to align with the batch size. This results in an 8x4x256 tensor of input embeddings, which are the token embeddings augmented with positional information. [35:00]

These input embeddings, which combine semantic meaning from token embeddings and positional information, are then fed as input to the large language model for training. The process involves understanding the dimensions: 8 for the batch size, 4 for the context length, and 256 for the embedding dimension. [39:00]

## Conclusion

In summary, we have added positional embeddings to token embeddings, creating input embeddings that capture both semantic meaning and positional information. While we randomly initialized the positional embedding matrix in this demonstration, the actual values are optimized during the LLM training process, similar to token embeddings. Understanding these dimensions and the interplay between token and positional embeddings is crucial for

grasping how LLMs process textual data. [\[41:00\]](#)

---

\*End of Chapter 11. Source: <https://www.youtube.com/watch?v=ufrPLpKnapU>\*

---

## Chapter 12: The Data Preprocessing Pipeline for Large Language Models

---

*Source video: Lecture 12: The entire Data Preprocessing Pipeline of Large Language Models (LLMs)*

*Duration: 94 minutes | Playlist position 12*

### Introduction to Data Preprocessing

Large language models (LLMs) process text, but raw text data, such as a PDF file, cannot be directly fed as input. Data preprocessing is the crucial procedure of transforming this raw data into a format suitable for LLMs. This lecture covers the fundamental steps involved in this process, which are essential for building high-performing LLMs. [0:05]

The data preprocessing pipeline for LLMs can be broken down into four fundamental steps. First is tokenization, followed by token embeddings, then positional embeddings, and finally, the creation of input embeddings. Input embeddings are the summation of token embeddings and positional embeddings, serving as the final input to the LLM. [0:05]

Tokenization itself can be further divided into three types: word-based, subword-based, and character-based tokenizers. Subword-based tokenizers, such as the Byte Pair Encoder (BPE) used in GPT models, are particularly important. Token embeddings involve converting token IDs into vectors, while positional embeddings encode information about the position of tokens within a sequence. [0:05]

### Tokenization: Word-Based Approach

The first step in preparing data for an LLM is tokenization. For educational purposes, we will start by constructing a word-based tokenizer from scratch, even though GPT models use subword-based tokenizers. This approach helps build intuition about how tokenization works. [0:05]

Word-based tokenization involves splitting text into individual words. To demonstrate this, we will use a book titled "The Verdict" as our dataset. The process begins by reading the raw text of the book into a Python variable. [0:05]

After reading the data, we can inspect it by printing the total number of characters and a sample of the text. For "The Verdict," the total character count is 20,479. [0:05]

The next crucial step is splitting the text into tokens. Initially, one might consider splitting based on whitespace. However, this approach incorrectly groups punctuation with words and treats whitespace itself as tokens. To address this, we need to split on punctuation and special characters, and also remove whitespace tokens. [0:05]

Using Python's regular expression library, we can define a pattern to split the text not only on whitespace but also on various punctuation marks like commas, periods, question marks, and quotation marks. Subsequently, we filter out any tokens that consist solely of whitespace.

[0:05]

This refined process results in individual words and punctuation marks becoming separate tokens. For example, the sentence "Hello, world." would be tokenized into "Hello", ",", "world", and ".". Whitespace tokens are excluded. [0:05]

Applying this to the entire dataset, we obtain a list of tokens representing the book. Printing the first 30 tokens shows how the text has been broken down into words and individual special characters. The total number of tokens generated from "The Verdict" is 4,690. [0:05]

## Token IDs and Vocabulary Construction

Computers require numerical representations, so the next step is to convert these tokens into token IDs. This involves creating a vocabulary, which is a mapping from unique tokens to unique integers. First, all duplicate tokens are removed, and the unique tokens are sorted alphabetically. [0:05]

Each unique token is then assigned an integer ID, starting from zero for the first token in the sorted list. This vocabulary acts as a dictionary for the LLM, allowing it to process numerical representations instead of raw text. The size of the vocabulary derived from "The Verdict" is 1,130. [0:05]

Python's `enumerate` function is useful for creating this mapping. It iterates through the sorted unique tokens, assigning an increasing integer to each. The resulting vocabulary dictionary maps each token to its corresponding ID, and vice versa. [0:05]

## Implementing a Tokenizer Class

To manage the tokenization process, a `Tokenizer` class can be implemented in Python. This class typically includes an `encode` method to convert text into token IDs and a `decode` method to convert token IDs back into text. [0:05]

The `__init__` method of the `Tokenizer` class takes the vocabulary as input and creates two dictionaries: one mapping strings to integers (`str_to_int`) and a reverse mapping from integers to strings (`int_to_str`). [0:05]

The `encode` method first tokenizes the input text using the same logic as before (splitting on special characters and removing whitespace) and then uses the `str_to_int` dictionary to convert each token into its corresponding ID. The `decode` method performs the reverse operation, using the `int_to_str` dictionary to convert token IDs back into tokens and then joining them to form text. [0:05]



Testing the encode and decode methods with a sample sentence demonstrates their functionality. The encode method converts the sentence into a list of token IDs, and the decode method successfully reconstructs the original sentence from these IDs. [0:05]

## Handling Out-of-Vocabulary (OOV) Words

A significant limitation of word-based tokenization is its inability to handle words not present in the vocabulary, known as out-of-vocabulary (OOV) words. If an OOV word is encountered during encoding, the tokenizer will raise an error. [27:34]

To address this, special context tokens can be added to the vocabulary. Two important tokens are the "unknown" token (<unk>) and the "end of text" token (<eos>). The unknown token is used to represent any word not found in the vocabulary. [27:34]

The "end of text" token is crucial for segmenting different text sources or documents. When processing multiple documents, this token signals the end of one document and the beginning of another, aiding the LLM in understanding context boundaries. [27:34]

By augmenting the vocabulary with these special tokens, the tokenizer can now handle OOV words by mapping them to the unknown token's ID. The vocabulary size increases by two to accommodate these new tokens. [27:34]

A modified `Tokenizer` class (version 2) incorporates logic to replace unknown words with the unknown token ID during encoding. Testing this version with text containing OOV words demonstrates that it can now encode and decode such sequences without errors, mapping unknown words to <unk>. [27:34]

While the unknown token is useful, GPT models typically do not use an explicit unknown token but do utilize the end-of-text token. [27:34]

## Types of Tokenizers: Beyond Word-Based

Word-based tokenization, despite its simplicity, has several drawbacks. It struggles with OOV words, requires large vocabulary sizes to cover all words in a language (estimated at 600,000 to 1 million words), and fails to capture semantic similarities between words with common roots (e.g., "boy" and "boys"). [27:34]

Character-based tokenization offers a solution to the large vocabulary problem, as the number of characters in a language is finite (around 256). This approach effectively handles OOV words. However, it loses the semantic meaning of words by breaking them down into individual characters and results in much longer tokenized sequences, which can be computationally inefficient. [27:34]

Subword-based tokenization aims to strike a balance between word-based and character-based approaches. It splits rare words into smaller, meaningful subwords while retaining frequently used words as single tokens. This preserves the root meaning of words and leads to more manageable vocabulary sizes. [27:34]

## Byte Pair Encoding (BPE)

Byte Pair Encoding (BPE) is a popular subword-based tokenization algorithm. Originally developed for data compression, it works by iteratively merging the most frequent pair of adjacent bytes (or characters/tokens) in a dataset. [27:34]

In the context of LLMs, BPE is applied to a corpus to build a vocabulary of subword units. The algorithm identifies common character sequences and merges them into new tokens. For example, if "es" is a frequent pairing, it might become a single token. This process continues, creating tokens that represent common words, subwords, and characters. [27:34]

The advantage of BPE is that it creates a vocabulary that is significantly smaller than a word-based vocabulary (e.g., around 50,000 tokens for GPT-2) while still capturing semantic relationships between words with shared roots. It also inherently handles OOV words by breaking them down into known subword units or characters. [27:34]

The `tiktoken` library provides an implementation of the BPE tokenizer used in OpenAI models. Using `tiktoken`, we can directly encode and decode text, and it handles OOV words gracefully without requiring explicit unknown tokens. [45:35]

The BPE tokenizer's ability to break down unknown words into known subwords or characters is key to its effectiveness. This process ensures that any input text can be represented numerically, even if it contains words not explicitly seen during vocabulary creation. [45:35]

## Input Target Pairs and Data Loaders

Before feeding data into an LLM, it's essential to structure it into input-target pairs. LLMs typically predict the next word in a sequence. Given a context size (the maximum number of tokens the LLM sees at once), we can create pairs where the input is a sequence of tokens, and the target is the subsequent token. [48:23]

For example, with a context size of four, if the input is "in the heart of", the target would be "the". If the input is "the heart of the", the target is "city". This process is repeated across the entire dataset using a sliding window approach. [48:23]

PyTorch's `DataLoader` is a powerful tool for managing this data preparation. It allows us to efficiently create batches of input-target pairs. The concept of "stride" determines how much the sliding window moves forward for each new input sequence, influencing the overlap

between consecutive inputs. [48:23]

The `DataLoader` takes the encoded token IDs and chunks them into overlapping sequences based on the context length and stride. It then generates batches of input IDs and corresponding target IDs, which are ready to be fed into the LLM. [48:23]

The batch size parameter in the `DataLoader` specifies how many input-output pairs are processed together before the model's parameters are updated. This is a standard practice in machine learning for efficient training. [48:23]

## Token Embeddings

Token IDs, while numerical, do not inherently capture the semantic meaning or relationships between words. Simply assigning arbitrary IDs to words would lose valuable information. One-hot encoding, another numerical representation, also fails to capture semantic similarity. [48:23]

Token embeddings address this by representing each token ID as a dense vector in a high-dimensional space. Words with similar meanings are mapped to vectors that are close to each other in this space. For instance, the vectors for "dog" and "cat" would be closer than the vectors for "dog" and "apple." [48:23]

These embedding vectors are learned during the training process. An embedding layer, typically implemented as a weight matrix, serves as a lookup table. Each row of this matrix corresponds to a token ID, and the row itself is the embedding vector for that token. [48:23]

In PyTorch, `torch.nn.Embedding` is used to create this layer. It requires the vocabulary size and the desired embedding dimension. For example, with a vocabulary size of 50,257 (used in GPT-2) and an embedding dimension of 256, the embedding layer will have 50,257 rows and 256 columns. [48:23]

When input token IDs are passed to the embedding layer, it retrieves the corresponding row vectors, effectively converting the sequence of token IDs into a sequence of embedding vectors. For a batch of data with a context length of four and an embedding dimension of 256, the resulting token embeddings tensor will have dimensions of 8x4x256. [1:21:25]

## Positional Embeddings

While token embeddings capture semantic meaning, they do not account for the position of tokens within a sequence. The sentence "the cat sat on the mat" has the same token embeddings for "cat" as "the mat sat on the cat," despite the different meanings conveyed by word order. [1:21:25]

Positional embeddings (or positional encoding) are added to token embeddings to inject information about the position of each token. This allows the LLM to understand the order of words, which is crucial for language comprehension. [1:21:25]

There are two main types: absolute and relative positional encoding. Absolute positional encoding assigns a unique embedding vector to each position in the sequence. These positional embedding vectors have the same dimension as the token embeddings (e.g., 256) and are added element-wise to the corresponding token embeddings. [1:21:25]

For a fixed context length (e.g., four tokens), a positional embedding matrix with dimensions `context_length x embedding_dimension` is created. This matrix contains the positional embedding vectors for each position. When processing a batch, these positional vectors are broadcast and added to the token embeddings. [1:21:25]

Relative positional encoding focuses on the distance between tokens rather than their absolute positions. However, absolute positional encoding is more commonly used, especially in sequence generation tasks like those performed by GPT models. [1:21:25]

## Input Embeddings

The final step in preparing the input for the LLM is creating input embeddings. This is achieved by adding the token embeddings and the positional embeddings. [1:21:25]

The resulting input embeddings tensor combines both the semantic meaning of the tokens and their positional information. For a batch of eight sequences, each with a context length of four and an embedding dimension of 256, the token embeddings tensor has dimensions  $8 \times 4 \times 256$ . The positional embeddings, after broadcasting, also effectively align with these dimensions. [1:21:25]

The addition operation, leveraging broadcasting, ensures that the positional information is correctly applied to each token within each sequence in the batch. This combined representation is then fed into the LLM architecture for further processing and prediction. [1:21:25]

Similar to token embeddings, the positional embedding vectors are also initialized randomly and are optimized during the LLM's training process. This ensures that the model learns the most effective positional representations for its task. [1:21:25]

## Conclusion

This lecture has covered the entire data preprocessing pipeline for LLMs, starting from tokenization (word-based, subword-based with BPE) to token IDs, token embeddings, positional embeddings, and finally, input embeddings. This comprehensive process prepares the data to be fed into the LLM architecture for pre-training or fine-tuning. [1:21:25]

---

\*End of Chapter 12. Source: <https://www.youtube.com/watch?v=mk-6cFebjis>\*

---

## Chapter 13: Introduction to the Attention Mechanism in Large Language Models (LLMs)

---

*Source video: Lecture 13: Introduction to the Attention Mechanism in Large Language Models (LLMs)*

*Duration: 51 minutes | Playlist position 13*

### Motivation for the Attention Mechanism

The process of building a large language model (LLM) pipeline involves three stages: data preparation, attention mechanism, and LLM architecture; training and model evaluation; and fine-tuning. Previously, significant time was dedicated to data preparation and sampling, covering aspects like word embedding, tokenization, byte-pair encoding, and positional encoding. Now, the focus shifts to the second building block of stage one: the attention mechanism. This mechanism is considered the engine that drives the power of LLMs like ChatGPT. [0:07]

To understand the intuition behind the name "attention mechanism" and the problems it solves, consider a complex sentence such as "The cat that was sitting on the mat which was next to the dog jumped." For a human, it's clear that the cat is the subject and the cat jumped. However, for a large language model, such sentences with long-term dependencies can be confusing. The model needs to understand that the primary action (jumped) is performed by the main subject (cat), even though other elements like "mat" and "dog" are present in the sentence. Without an attention mechanism, an LLM might incorrectly associate the action with the dog or focus solely on the cat being on the mat, missing the crucial information that the cat jumped. [0:07]

The attention mechanism addresses this by allowing the LLM to determine which words in a sentence are most relevant to a particular word. This is crucial for processing sequences and understanding relationships between words, especially in long-term dependencies where traditional neural networks struggle. [0:07]

### Types of Attention Mechanisms

The attention mechanism has evolved over time, with several types being developed. The most prominent one used in modern LLMs like GPT is multi-head attention. However, directly understanding multi-head attention can be challenging. Therefore, the planned series of lectures will build up the concept sequentially, starting with simplified self-attention, which is the most basic form. This will be followed by self-attention, which introduces trainable weights. Subsequently, causal attention will be covered, which masks out future inputs, allowing the model to focus only on previous and current inputs in a sequence.

Finally, multi-head attention will be explored, which is essentially a collection of causal attention heads stacked together, enabling the model to attend to information from different representation subspaces simultaneously. [0:07]

## Limitations of Pre-Attention Architectures

Before the advent of attention mechanisms, sequence-to-sequence tasks like language translation faced significant challenges. A word-by-word translation approach, for instance, fails because it does not account for contextual understanding and grammatical alignment. For example, translating "Can you help me?" from English to Hindi might result in a different word order and association than a direct word-for-word translation would suggest. [0:07]

To address the limitations of simple neural networks that lack memory, architectures like Recurrent Neural Networks (RNNs) were developed. RNNs incorporate an encoder-decoder structure. The encoder processes the input sequence and generates a context vector, which is essentially a final hidden state that encapsulates the meaning and memory of the entire input. This context vector is then passed to the decoder, which generates the output sequence word by word. [0:07]

The core innovation in RNNs was the hidden state, which was updated at each step, accumulating memory of previous inputs. However, a major drawback of RNNs, particularly in the context of long sequences, is that the decoder only has access to the final hidden state. This places immense pressure on the single final hidden state to retain all necessary information, often leading to a loss of context, especially when dependencies span long distances. [22:37]

This loss of context means that the RNN struggles to capture long-range dependencies. For instance, in the sentence "The cat that was sitting on the mat which was next to the dog jumped," the action "jumped" is directly related to the subject "cat," but understanding this requires considering the entire context, including the cat sitting on the mat and next to the dog. The RNN's reliance on a single final hidden state makes it difficult to retain and access this crucial information, hindering accurate translation or prediction. [22:37]

## The Emergence of the Attention Mechanism

The limitations of RNNs, particularly their inability to effectively handle long-range dependencies due to relying on a single final hidden state, led to the development of the attention mechanism. In 2014, researchers introduced the Bahdanau attention mechanism, which modified the encoder-decoder RNN architecture. This innovation allowed the decoder to selectively access different parts of the input sequence at each decoding step, rather than solely relying on the final hidden state. [28:26]

The core idea of the Bahdanau attention mechanism is that at each decoding step, the decoder has access to all the hidden states from the encoder. It can then decide how much "attention" to pay to each input token. For example, when translating the word "jumped," the decoder could pay significant attention to the input word "jumped" itself, but also to the word "cat," recognizing it as the subject performing the action. This selective focus on relevant parts of the input sequence, determined by attention weights, helps overcome the loss of context experienced in traditional RNNs. [28:26]

This concept of dynamic focus, where the model can selectively choose which inputs to focus on and how much attention to give at each decoding step, proved revolutionary. It enabled models to learn long-range dependencies much more effectively. While the Bahdanau attention mechanism was introduced in 2014, the Transformer architecture, which heavily utilized and popularized this concept, was proposed in the seminal 2017 paper "Attention Is All You Need." [28:26]

The Transformer architecture removed the need for recurrent connections altogether, relying solely on attention mechanisms. This shift marked a significant advancement in natural language processing, leading to the powerful LLMs we see today. The historical progression shows a clear path from RNNs with their limitations to the attention-based Transformer models. [28:26]

## **Self-Attention: A Key Component of Modern LLMs**

The Transformer architecture introduced the concept of "self-attention." Unlike traditional attention mechanisms used in tasks like machine translation, where attention is directed between two different sequences (e.g., English to French), self-attention operates within a single sequence. It allows each position in the input sequence to attend to all other positions within the same sequence. [40:00]

In the context of LLMs, which are primarily trained for next-word prediction, self-attention is crucial. When predicting the next word, the model needs to understand how different words within the current sentence relate to each other. Self-attention enables the model to compute attention weights by relating different positions in a single input sequence, thereby learning the relationships between various parts of the input itself. This is a fundamental difference from traditional attention mechanisms that focus on relationships between elements of two distinct sequences. [40:00]

The lectures will proceed by first implementing a simplified self-attention mechanism in Python, followed by self-attention, causal attention, and finally, multi-head attention. This structured approach aims to build a strong theoretical foundation and practical coding skills for understanding the core components of LLMs. [40:00]



\*End of Chapter 13. Source: <https://www.youtube.com/watch?v=XN7sevVxyUM>\*

---

## Chapter 14: Simplified Self-Attention Mechanism Without Trainable Weights

---

*Source video: Lecture 14: Simplified Attention Mechanism - Coded from scratch in Python | No trainable weights*

*Duration: 79 minutes | Playlist position 14*

### Introduction to Attention and the Lecture's Goal

The lecture series aims to build large language models from scratch, with this installment focusing on the mathematical foundations of attention and a simplified implementation. Previous lectures covered the intuition behind attention and the limitations of recurrent neural networks that led to its invention. This lecture will implement a simplified self-attention mechanism without trainable weights to introduce the core concepts. [\[0:05\]](#)

The plan for covering attention involves multiple lectures. This lecture will focus on a simplified self-attention mechanism without trainable weights. Subsequent lectures will cover self-attention with trainable weights (as used in modern LLMs like GPT), causal attention, and multi-head attention. Each lecture will include mathematical explanations and a from-scratch Python implementation. [\[0:05\]](#)

The primary goal of this lecture is to implement a simple variant of the self-attention mechanism that does not involve any trainable weights. This simplified version will help in understanding the fundamental idea of how attention works. [\[0:05\]](#)

### Input Representation and the Need for Context

To motivate the lecture, a sample sentence, "your journey starts with one step," is used. In large language models, this sentence would first be pre-processed into tokens, then token IDs, and finally converted into vector embeddings. These embeddings represent words in a high-dimensional space, capturing semantic meaning. For demonstration, a three-dimensional space is used, where semantically related words are closer in vector space. [\[0:05\]](#)

While embedding vectors capture a word's meaning, they lack information about how that word relates to others in the sentence. For instance, the embedding for "journey" doesn't inherently convey its relationship with "your," "starts," "with," "one," or "step." This contextual information is crucial for tasks like predicting the next word, as it helps determine which words are most relevant to the current word. [\[0:05\]](#)

The attention mechanism addresses this by transforming an embedding vector into a "context vector." This context vector is an enriched representation that includes not only the original semantic meaning but also information about the word's relationships with other words in the

sentence. This contextual information significantly aids in predicting subsequent words in large language models. [0:05]

A context vector is generated for each input embedding vector. The goal of all attention mechanisms, from simplified versions to complex multi-head attention, is to convert embedding vectors into these context vectors, which then serve as input to the language model. [0:05]

## Calculating Attention Scores

The input sentence "your journey starts with one step" is represented by input vectors  $X_1, X_2, \dots, X_6$ , where each  $X_i$  is a three-dimensional vector embedding for the  $i$ -th token. The goal is to compute a context vector  $Z_i$  for each input vector  $X_i$ . To compute  $Z_2$  (the context vector for "journey"), we need to determine the importance of each input vector ( $X_1$  through  $X_6$ ) relative to  $X_2$ . This importance is quantified by "attention weights." [0:05]

The token being focused on, for which we are computing the context vector, is called the "query." In this case, "journey" ( $X_2$ ) is the query. The context vector  $Z_2$  will be an enriched embedding containing information about  $X_2$  and its relationship with all other input elements. [18:19]

The first step in computing the context vector is to calculate intermediate values known as "attention scores." These scores quantify how much importance should be given to each input word relative to the query word. An attention score exists between the query and every input vector. [18:19]

Intuitively, to find the importance or alignment between two vectors, the dot product is a suitable mathematical operation. The dot product of two vectors is related to the cosine of the angle between them. If vectors are aligned (angle close to  $0^\circ$ ), the cosine is close to 1, resulting in a higher dot product. If vectors are perpendicular (angle close to  $90^\circ$ ), the cosine is 0, resulting in a dot product of 0. Therefore, a higher dot product indicates greater alignment and similarity. [18:19]

In self-attention, the dot product between the query vector and each input vector is used to calculate the attention scores. This quantifies how aligned the vectors are, indicating how much attention one element should pay to another. Higher dot products signify greater similarity and thus higher attention scores. [18:19]

For the query "journey" ( $X_2$ , indexed as  $\text{inputs}[1]$  due to zero-based indexing), attention scores are computed by taking the dot product of  $X_2$  with each input vector  $X_1, X_2, \dots, X_6$ . The resulting attention scores reveal that "journey" itself has the highest score, followed by "starts" and "step," which are semantically aligned with "journey." The word

"one" has the lowest score, as its vector is nearly perpendicular to "journey's" vector. [18:19]

## Normalizing Attention Scores to Attention Weights

The next step is normalization, primarily for interpretability. The goal is to convert attention scores into "attention weights" that sum up to one, allowing them to be expressed as percentages. This normalization also contributes to stability during training, particularly when using backpropagation and gradient descent. [31:11]

A simple normalization method involves summing all attention scores and then dividing each score by this sum. This ensures the resulting attention weights add up to one. However, the softmax function is generally preferred in machine learning for normalization, especially when dealing with extreme values. [31:11]

Softmax normalizes values by exponentiating each score and then dividing by the sum of all exponentiated scores. This process is particularly effective because it amplifies larger values and diminishes smaller ones significantly. For instance, if one score is extremely large compared to others, its corresponding softmax weight will be very close to 1, while the weights for smaller scores will be close to 0. This behavior is desirable for highlighting the most relevant elements. [31:11]

PyTorch's `torch.softmax` implementation includes a numerical stability improvement: it subtracts the maximum value from each score before exponentiation. Mathematically, this does not change the final result but helps prevent overflow or underflow errors with very large or small numbers during computation. [31:11]

The naive softmax implementation involves calculating  $e^{\text{score}_i} / \sum_j e^{\text{score}_j}$ . PyTorch's implementation calculates  $e^{\text{score}_i - \max(\text{scores})} / \sum_j e^{\text{score}_j - \max(\text{scores})}$ . Both methods yield the same normalized weights, but PyTorch's approach is more robust against numerical instability. [31:11]

The calculated attention weights for the sentence "your journey starts with one step" indicate the proportion of attention each word should receive when considering "journey." For example, "journey" and "starts" receive higher attention weights (around 23%), while "one" receives a lower weight (around 10%). These interpretable percentages are possible because the attention weights sum to one. [46:10]

## Computing the Context Vector

The final step in calculating the context vector is to use the computed attention weights. Each input embedding vector is multiplied by its corresponding attention weight. These scaled vectors are then summed together to produce the final context vector for the query word. [46:10]

Visually, this process involves scaling down each input vector according to its attention weight and then summing these scaled vectors. For the query "journey," its own embedding contributes significantly (e.g., 25%), while other words contribute proportionally to their attention weights (e.g., "starts" contributes 20%, "one" contributes 10%). This summation results in an enriched context vector that incorporates information from all input words, weighted by their relevance to the query. [46:10]

Mathematically, the context vector  $Z_i$  for an input  $X_i$  is computed by multiplying each input embedding vector  $X_j$  by its corresponding attention weight  $w_{ij}$  and summing the results:  $Z_i = \sum_j w_{ij} X_j$ . This operation effectively creates a weighted average of all input embeddings, where the weights are determined by the attention mechanism. [46:10]

To compute context vectors for all tokens simultaneously, an "attention weight matrix" is used. This matrix, where each row represents the attention weights for a specific query token relative to all input tokens, is multiplied by the input embedding matrix. The resulting matrix contains the context vectors for all input tokens. [52:52]

The calculation of attention scores for all tokens involves computing the dot product between each pair of input vectors. This can be efficiently performed using matrix multiplication: multiplying the input matrix by its transpose. The resulting matrix contains all pairwise attention scores. [52:52]

Normalization is then applied row-wise to the attention score matrix using `torch.softmax(dim=-1)`. This ensures that the attention weights in each row sum to one, representing the distribution of attention for each query token across all input tokens. [52:52]

Finally, the context vectors for all tokens are computed by multiplying the attention weight matrix with the original input embedding matrix. This single matrix multiplication operation efficiently calculates all context vectors, where each row of the resulting matrix corresponds to the context vector for a specific input token. [52:52]

## The Role of Trainable Weights

The simplified attention mechanism implemented so far relies solely on the dot product of embeddings to determine attention scores. This means attention is primarily based on semantic similarity. However, in real-world scenarios, a word might be important in a given context even if it's not semantically similar to the query word. [1:16:10]

For example, in the sentence "The cat sat on the mat because it is warm," if the query is "warm," a dot-product-based attention mechanism might focus on words semantically related to "warm." However, the word "mat" might be contextually very important (the mat is warm), even if "mat" and "warm" are not semantically similar in a general embedding space.

[1:16:10]

Trainable weights allow the model to learn these contextual relationships. By introducing learnable parameters, the model can adjust the attention mechanism to prioritize words that are important in a specific context, regardless of their direct semantic similarity to the query. This ability to capture context is crucial for understanding long-range dependencies and is a key reason why trainable weights are essential in modern LLMs. [1:16:10]

The next lecture will build upon this foundation by introducing trainable weights into the self-attention mechanism, along with the concepts of key, query, and value, which are fundamental to more advanced attention mechanisms like causal and multi-head attention.

[1:16:10]

\*End of Chapter 14. Source: <https://www.youtube.com/watch?v=eSRhpYLERw4>\*

---

## Chapter 15: Coding the Self-Attention Mechanism with Key, Query, and Value Matrices

---

*Source video: Lecture 15: Coding the self attention mechanism with key, query and value matrices*

*Duration: 79 minutes | Playlist position 15*

### Recap of Previous Lecture: Self-Attention Without Trainable Weights

In the previous lecture, a simplified self-attention mechanism was implemented without trainable weights. The process involved converting input sentence embeddings into context vectors for each token. This was achieved by first calculating attention scores through a dot product between a query vector and all input embedding vectors. These scores were then normalized to obtain attention weights, which sum up to one. Finally, these weights were used to compute a context vector as a weighted sum of the input vectors. [0:05]

The context vector can be understood as an enriched embedding vector. While an embedding vector only captures the semantic meaning of a word, the context vector also encodes how that word relates to other words in the sequence. This enrichment is achieved by considering the weighted contributions of other words based on their relevance to the query word. [4:48]

### Introduction to Self-Attention with Trainable Weights

This lecture introduces the self-attention mechanism with trainable weights, a key component in modern large language models like GPT. The core idea is to introduce weight matrices that are optimized during training to produce effective context vectors. These trainable weight matrices are central to learning how to generate good context representations for each token. [5:40]

At the heart of this mechanism are three crucial terminologies: query, key, and value. These terms refer to three distinct trainable weight matrices: the query weight matrix (WQ), the key weight matrix (WK), and the value weight matrix (WV). These matrices are used to transform the input embeddings into query, key, and value vectors. [7:15]

### Step 1: Converting Input Embeddings to Query, Key, and Value Vectors

The first step in constructing the self-attention mechanism is to convert the input embedding vectors into query, key, and value vectors. This is achieved by multiplying the input embedding matrix with the respective trainable weight matrices (WQ, WK, and WV). For instance, if the input embedding matrix has dimensions (num\_tokens, d\_model), and the weight matrices have dimensions (d\_model, d\_k), the resulting query, key, and value matrices will have dimensions (num\_tokens, d\_k). [8:30]

In the provided example, the input embeddings are 3-dimensional vectors for six tokens. The weight matrices ( $W_Q$ ,  $W_K$ ,  $W_V$ ) are initialized with dimensions  $3 \times 2$ . Multiplying the input matrix ( $6 \times 3$ ) with each weight matrix ( $3 \times 2$ ) results in query, key, and value matrices, each with dimensions  $6 \times 2$ . This transformation projects the input embeddings into a different dimensional space, which can be learned and optimized during training. [10:00]

The code implementation involves defining the input tensor and then initializing the query, key, and value weight matrices using `torch.nn.Parameter`. These parameters are initialized with random values and have a shape of  $(d_{in}, d_{out})$ , where  $d_{in}$  is the input embedding dimension and  $d_{out}$  is the desired dimension for queries, keys, and values. [14:00]

To illustrate the process for a single token, the code demonstrates how to compute the query, key, and value vectors for the token "Journey" by multiplying its corresponding input embedding vector with the respective weight matrices. [17:00]

Subsequently, the code generalizes this to compute the entire query, key, and value matrices for all input tokens by performing matrix multiplication between the input matrix and each of the weight matrices. The resulting matrices (queries, keys, values) all have a shape of  $(num\_tokens, d_{out})$ . [23:03]

## Step 2: Computing Attention Scores

The next step is to compute the attention scores. This involves determining how much each query vector should attend to each key vector. Intuitively, a higher dot product between a query and a key indicates a stronger alignment and thus a higher attention score. The attention score quantifies the relevance of each input token (represented by its key vector) to the current token being processed (represented by its query vector). [25:00]

To calculate the attention scores for a single query, its dot product is taken with all the key vectors. In matrix form, this is achieved by multiplying the query matrix with the transpose of the key matrix. If the queries matrix is  $(num\_tokens, d_k)$  and the keys matrix is  $(num\_tokens, d_k)$ , then the keys transpose is  $(d_k, num\_tokens)$ . The resulting attention scores matrix will have dimensions  $(num\_tokens, num\_tokens)$ , where each row represents the attention scores of a specific query with respect to all keys. [31:18]

The code implements this by performing a matrix multiplication between the queries matrix and the transpose of the keys matrix, resulting in the `attention_scores` matrix. [33:00]

## Step 3: Computing Attention Weights (Scaling and Softmax)

The raw attention scores are not directly interpretable as weights because they do not sum to one. To address this, two operations are performed: scaling and normalization using the softmax function. Before applying softmax, the attention scores are scaled by dividing them



by the square root of the key dimension ( $\sqrt{d_k}$ ). This scaling is crucial for stabilizing learning and preventing the softmax function from producing overly peaked distributions. [35:00]

The scaling by  $\sqrt{d_k}$  helps to keep the variance of the dot products close to one, regardless of the dimension of the key and query vectors. This prevents the gradients from becoming too large or too small during backpropagation, leading to more stable training. The softmax function is then applied to the scaled scores, ensuring that the resulting attention weights for each query sum up to one and are positive, making them interpretable as probabilities. [38:00]

The code calculates the attention weights by first dividing the `attention_scores` by the square root of the key dimension and then applying the softmax function along the appropriate dimension (`dim=-1`). This results in the `attention_weights` matrix, where each row sums to one. [41:00]

#### Step 4: Computing Context Vectors

The final step is to compute the context vectors. This is achieved by taking a weighted sum of the value vectors, where the weights are the attention weights calculated in the previous step. For each query, its context vector is a weighted sum of all value vectors, with the weights determined by how much the query attends to each key. [45:00]

Intuitively, this means that the context vector for a given token is formed by aggregating information from all tokens in the sequence, with each token's contribution being proportional to its attention weight. This process allows the model to capture the contextual relationships between words. The calculation is performed by multiplying the `attention_weights` matrix with the values matrix. The resulting context vector matrix has dimensions  $(\text{num\_tokens}, d_v)$ , where  $d_v$  is the dimension of the value vectors. [48:00]

The code implements this by performing a matrix multiplication between the `attention_weights` and the values matrices to obtain the `context_vectors`. [52:00]

#### Implementing a Self-Attention Class in PyTorch

To encapsulate the self-attention mechanism, a Python class `SelfAttentionV1` is implemented using `torch.nn.Module`. This class takes the input dimension (`d_in`) and output dimension (`d_out`) as arguments. The `__init__` method initializes the trainable weight matrices (query, key, value) using `torch.nn.Parameter`. The `forward` method orchestrates the entire process: computing keys, queries, and values; calculating attention scores; normalizing them to get attention weights; and finally computing the context vectors. [55:00]

An instance of this class is created, and when an input tensor is passed to it, the forward method is executed, returning the context vector matrix. The dimensions of the output context vector matrix are (num\_tokens, d\_out). [1:00:00]

## Improving the Implementation with nn.Linear

The SelfAttentionV1 implementation can be further improved by utilizing torch.nn.Linear layers instead of torch.nn.Parameter for initializing the weight matrices. nn.Linear layers offer optimized weight initialization schemes that can lead to more stable and effective model learning. By setting the bias parameter to False, nn.Linear effectively performs the same matrix multiplication as nn.Parameter but with a more sophisticated initialization. [1:03:00]

A SelfAttentionV2 class is presented, demonstrating this improved approach. Creating an instance of SelfAttentionV2 and passing input data yields context vectors, which may differ from SelfAttentionV1 due to the different initialization schemes. [1:05:00]

## Understanding Query, Key, and Value

The terms query, key, and value have intuitive meanings within the attention mechanism. The **query** represents the current token the model is focusing on, analogous to a search query in a database. The **keys** are associated with each item in the input sequence and are used to match against the query to determine relevance. The **values** represent the actual content or representation of the input items. Once the model identifies relevant keys based on the query, it retrieves the corresponding values to construct the context vector. [1:08:00]

The lecture concludes by summarizing the four key steps of the self-attention mechanism: converting inputs to queries, keys, and values; computing attention scores; computing attention weights through scaling and softmax; and finally, computing the context vectors. The importance of understanding matrices, dimensions, and vector calculus for mastering transformers and large language models is emphasized. [1:12:00]

\*End of Chapter 15. Source: <https://www.youtube.com/watch?v=UjdRN80c6p8>\*

## Chapter 16: Causal Self-Attention Mechanism

---

*Source video: Lecture 16: Causal Self Attention Mechanism | Coded from scratch in Python*

*Duration: 55 minutes | Playlist position 16*

### Recap of Self-Attention

The lecture begins by recapping the self-attention mechanism. The process starts with converting words into tokens, then token IDs, and finally into vector embeddings. For demonstration, a three-dimensional vector embedding is used, though in models like GPT, these dimensions can be much larger. The core goal of attention is to transform these input embeddings into context embeddings, which not only encode the semantic meaning of a token but also its relationship with other tokens in the sequence. [\[0:05\]](#)

To achieve this, input embeddings are multiplied by trainable weight matrices for queries (WQ), keys (WK), and values (WV) to obtain the query, key, and value matrices. Subsequently, attention scores are computed by multiplying the query matrix with the transpose of the key matrix. These scores are then scaled by the square root of the key dimension and passed through a softmax function to obtain attention weights. Finally, these attention weights are multiplied by the value matrix to produce the context vectors. [\[0:05\]](#)

### Introduction to Causal Attention

Causal attention, also known as masked attention, is a specialized form of self-attention that restricts the model to only consider previous and current tokens in a sequence when processing a given token. This contrasts with standard self-attention, which allows access to the entire input sequence. In causal attention, when computing attention scores for a token, only tokens that appear at or before the current token are considered; future tokens are masked out. [\[0:05\]](#)

Visually, this means that in the attention weight matrix, all attention weights corresponding to future tokens (those appearing after the current token) are masked out, effectively set to zero. For instance, when processing the token "journey," attention is only paid to "your" and "journey" itself, while "starts," "with," "one," and "step" are masked. Similarly, for the token "width," attention is paid to "your," "journey," "starts," and "width," with subsequent tokens masked. For the first token "your," all subsequent tokens are masked. [\[0:05\]](#)

### Implementing Causal Masking

The implementation of causal attention involves masking out the attention weights above the diagonal of the attention weight matrix. This is achieved by first generating a mask where all elements above the diagonal are zero. In PyTorch, this can be done using `torch.tril`, which

creates a lower triangular matrix. This mask is then applied to the attention weights, setting the elements above the diagonal to zero. [12:44]

After masking, the attention weights no longer sum to one in each row. To rectify this, a renormalization step is performed. The sum of each row is calculated, and each element in the row is divided by this sum. This ensures that the attention weights in each row sum up to one, maintaining their probabilistic interpretation. [12:44]

However, a potential issue with this approach is data leakage. Applying softmax before masking and then renormalizing can still allow future tokens to influence the attention weights of previous tokens due to the nature of the softmax function. A more robust method involves replacing the attention scores corresponding to future tokens with negative infinity before applying the softmax function. [12:44]

When negative infinity is exponentiated in the softmax function, it results in zero. This effectively masks out future tokens without introducing data leakage. The softmax function then automatically handles the renormalization, ensuring that the resulting attention weights sum to one in each row. This is achieved by creating an upper triangular mask where elements above the diagonal are marked for replacement with negative infinity using a function like `torch.nan_to_num` or `mask_fill`. [12:44]

## Incorporating Dropout

Dropout is a regularization technique used to prevent overfitting by randomly setting a fraction of neuron activations to zero during training. In the context of attention, dropout can be applied after the calculation of attention weights. This involves creating a dropout mask where a certain probability of elements are randomly set to zero. The remaining active weights are then scaled up to compensate for the dropped-out elements, maintaining the overall balance of attention. [12:44]

The scaling factor is typically  $1 / (1 - p)$ , where  $p$  is the dropout probability. For example, with a dropout rate of 0.5, the remaining weights are scaled by 2. This process randomly zeros out some attention weights and rescales the others, contributing to better generalization performance. [12:44]

## Implementing Causal Attention Class with Batches

The causal attention class is implemented to handle batches of input sequences. The input shape now includes a batch size dimension, making it a 3D tensor (`batch_size`, `num_tokens`, `embedding_dimension`). The class computes queries, keys, and values, and then calculates attention scores. [12:44]

The causal mask is applied by replacing elements above the diagonal of the attention scores with negative infinity. This is followed by scaling, dropout, and softmax to obtain the final attention weights. These weights are then multiplied by the values to produce the context vectors. The `register_buffer` method is used for fixed tensors like the causal mask, ensuring they are moved to the correct device (CPU or GPU) along with the model parameters. [12:44]

The implementation also accounts for cases where the number of tokens in a batch might be smaller than the maximum context size. The resulting context vectors for each sequence in the batch are stacked together, forming a 3D tensor representing the output for the entire batch. [12:44]

---

\*End of Chapter 16. Source: <https://www.youtube.com/watch?v=h94TQOK7NRA>\*

---

## Chapter 17: Multi-Head Attention Part 1 - Basics and Python Code

---

*Source video: Lecture 17: Multi Head Attention Part 1 - Basics and Python code*

*Duration: 32 minutes | Playlist position 17*

### Introduction to Multi-Head Attention

The lecture series has progressed through simplified self-attention, self-attention with trainable weights, and causal attention, building a foundation for understanding multi-head attention. Multi-head attention is a crucial mechanism in modern large language models (LLMs), but its complexity requires a solid understanding of prior concepts. [\[0:05\]](#)

This lecture will cover two types of multi-head attention mechanisms. The first type involves concatenating context vector matrices obtained from different query, key, and value matrices. The second, more common approach in modern LLMs, will be discussed in a subsequent lecture. This lecture focuses on the first type. [\[1:18\]](#)

### Recap of Causal Attention

Causal attention is a precursor to multi-head attention. The process begins with input embedding vectors for each token in a sequence. The goal of any attention mechanism is to transform these input embeddings into context vectors, which are enriched representations that encode not only the semantic meaning of a word but also its relationship with other words in the sequence. [\[2:10\]](#)

The first step involves multiplying the input embeddings with three trainable weight matrices: one for queries ( $W_q$ ), one for keys ( $W_k$ ), and one for values ( $W_v$ ). These matrices are initialized randomly and optimized through backpropagation. The multiplication results in the query, key, and value matrices, whose dimensions are determined by the number of tokens and an output dimension defined by the weight matrices. [\[2:55\]](#)

Next, attention scores are computed by multiplying the query matrix with the transpose of the key matrix ( $Q * K^T$ ). Each element in the attention score matrix indicates the relevance of a particular key to a given query. In causal attention, to ensure that a token only attends to preceding tokens, all attention scores above the main diagonal are set to zero. This masking is crucial for tasks like language modeling where future information should not influence the current prediction. [\[3:35\]](#)

After masking, the attention scores are converted into attention weights, typically by applying a softmax function. These attention weights are then multiplied by the value matrix to produce the context vector matrix. The context vector matrix is an enriched representation of

the input embeddings, incorporating information about the relationships between tokens in the sequence. [4:30]

The causal attention mechanism can also handle batches of input sequences. If the input has a shape of  $(\text{batch\_size}, \text{num\_tokens}, \text{input\_dimension})$ , the output context vector matrix will have a shape of  $(\text{batch\_size}, \text{num\_tokens}, \text{output\_dimension})$ . [5:15]

The causal attention class, as implemented previously, takes input  $x$  with shape  $(\text{batch\_size}, \text{num\_tokens}, \text{input\_dimension})$ . It then computes queries, keys, and values using trainable weight matrices. Attention scores are calculated, masked for causality, and then converted to attention weights using softmax. Finally, these weights are multiplied by the values to obtain the context vectors. [6:20]

When creating an instance of the causal attention class, parameters like  $d_{\text{in}}$  (input dimension),  $d_{\text{out}}$  (output dimension),  $\text{context\_length}$  (number of tokens), and  $\text{dropout\_rate}$  are specified. For an input batch of shape  $(2, 6, 3)$  (2 batches, 6 tokens, 3 dimensions), with  $d_{\text{out}}=2$ , the resulting context vector shape is  $(2, 6, 2)$ . [7:30]

## Implementing Multi-Head Attention

Multi-head attention extends the single attention mechanism by dividing it into multiple "heads." Each head operates independently, allowing the model to jointly attend to information from different representation subspaces at different positions. This is achieved by creating multiple instances of the single attention mechanism, each with its own set of weights. [8:10]

Instead of one set of trainable weight matrices for query, key, and value, multi-head attention uses multiple sets, one for each head. For example, with two attention heads, there would be two sets of query, key, and value weight matrices. These are multiplied by the input  $x$  to produce multiple query, key, and value matrices. [9:15]

Each set of query, key, and value matrices is used to compute attention scores and subsequently context vectors independently. If there are  $h$  attention heads, this results in  $h$  context vector matrices. These individual context vector matrices are then concatenated along the last dimension (columns) to form the final, larger context vector matrix. [10:00]

For instance, if the output dimension  $d_{\text{out}}$  for a single head is 2, and there are two attention heads, the final context vector matrix will have an output dimension of  $d_{\text{out}} * \text{num\_heads}$ , which is  $2 * 2 = 4$ . The shape of the final context vector matrix will be  $(\text{batch\_size}, \text{num\_tokens}, d_{\text{out}} * \text{num\_heads})$ . [10:45]

The implementation involves creating a wrapper class that instantiates multiple causal attention modules. The `forward` method of this wrapper class concatenates the outputs from

each attention head along the last dimension (dim=-1). [11:30]

```
import torch
import torch.nn as nn

# Assuming CausalAttention class is defined as in previous lectures
class CausalAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout_rate=0.0, bias=False):
        super().__init__()
        self.d_in = d_in
        self.d_out = d_out
        self.context_length = context_length

        self.W_k = nn.Linear(d_in, d_out, bias=bias)
        self.W_q = nn.Linear(d_in, d_out, bias=bias)
        self.W_v = nn.Linear(d_in, d_out, bias=bias)

        self.softmax = nn.Softmax(dim=-1)
        self.dropout = nn.Dropout(dropout_rate)

    def forward(self, x):
        batch_size, seq_len, _ = x.shape
        assert seq_len == self.context_length, "Input sequence length must match context length"

        keys = self.W_k(x)
        queries = self.W_q(x)
        values = self.W_v(x)

        # Attention scores: (batch_size, seq_len, d_out) x (batch_size, d_out, seq_len) -> (batch_size, seq_len, seq_len)
        attention_scores = torch.bmm(queries, keys.transpose(1, 2))

        # Causal mask
        mask = torch.tril(torch.ones(seq_len, seq_len)).bool()
        attention_scores = attention_scores.masked_fill(~mask, float('-inf'))

        attention_weights = self.softmax(attention_scores)
        attention_weights = self.dropout(attention_weights)

        context_vectors = torch.bmm(attention_weights, values)
        return context_vectors

class MultiHeadAttentionWrapper(nn.Module):
    def __init__(self, num_heads, causal_attention_module):
        super().__init__()
        self.num_heads = num_heads
        self.heads = nn.ModuleList([causal_attention_module for _ in range(num_heads)])
        # Assuming d_out of the causal_attention_module is consistent
        self.d_out_head = causal_attention_module.d_out

    def forward(self, x):
        # x shape: (batch_size, seq_len, d_in)
        head_outputs = [head(x) for head in self.heads]
        # Each head_output shape: (batch_size, seq_len, d_out_head)
```



```
# Concatenate along the last dimension
return torch.cat(head_outputs, dim=-1)
```

[12:00]

To demonstrate, let's create an instance of the `MultiHeadAttentionWrapper`. We'll use an input batch with a shape of (2, 6, 3), representing 2 batches, 6 tokens per batch, and an input embedding dimension of 3. We'll set `d_out` for each causal attention head to 2 and the number of heads to 2. [13:00]

```
# Example Usage
batch_size = 2
num_tokens = 6
d_in = 3
d_out_head = 2
num_heads = 2
dropout_rate = 0.0

# Create input batch
inputs = torch.randn(batch_size, num_tokens, d_in)
# Batch shape: (2, 6, 3)

# Instantiate a single causal attention module
causal_attn_module = CausalAttention(d_in=d_in, d_out=d_out_head, context_length=num_tokens, dropout=dropout_rate)

# Instantiate the multi-head attention wrapper
mha_wrapper = MultiHeadAttentionWrapper(num_heads=num_heads, causal_attention_module=causal_attn_module)

# Get the context vector
context_vector = mha_wrapper(inputs)

# Print the shape of the resulting context vector
print(f"Context Vector Shape: {context_vector.shape}")
```

[13:30]

The output shape of the context vector is (2, 6, 4). The first dimension (2) corresponds to the batch size. The second dimension (6) corresponds to the number of tokens. The third dimension (4) is the result of concatenating the outputs from the two attention heads, each having an output dimension of 2 ( $d\_out\_head * num\_heads = 2 * 2 = 4$ ). [14:00]

This context vector matrix is the final output of the multi-head attention mechanism and is further passed to the rest of the large language model. While the current implementation processes each head sequentially, leading to inefficiencies, the next lecture will explore a more efficient parallel implementation using weight splits, which is commonly used in models like GPT. [14:30]

## Efficiency Considerations and Future Directions

The current implementation of multi-head attention involves sequentially processing each attention head. This can be computationally inefficient, especially when dealing with a large number of heads, such as the 96 heads used in GPT-3. [15:00]

A more efficient approach is to process the attention heads in parallel. This can be achieved by restructuring the weight matrices and performing computations via matrix multiplication. This optimized method, known as multi-head attention with weight splits, is the standard implementation in modern LLMs like GPT and will be the focus of the next lecture. [15:30]

The number of attention heads is a significant hyperparameter. For instance, GPT-3 utilized 96 attention heads in its largest version. While the exact number for GPT-4 has not been publicly disclosed, it is presumed to be considerably higher. The concatenation of outputs from such a large number of heads underscores the substantial computational resources required for training LLMs. [16:00]

This lecture has provided a foundational understanding of multi-head attention by extending the causal attention mechanism. The core idea is to run the attention mechanism multiple times with different learned projections and then combine the results. The next lecture will delve into the efficient implementation of multi-head attention with weight splits, including a hands-on example with matrix computations. [16:30]

---

\*End of Chapter 17. Source: <https://www.youtube.com/watch?v=cPaBCoNdCtE>\*

---

## Chapter 18: Multi-Head Attention with Weight Splits

---

*Source video: Lecture 18: Multi Head Attention Part 2 - Entire mathematics explained*

*Duration: 61 minutes | Playlist position 18*

### Introduction to Efficient Multi-Head Attention

In the previous lecture, multi-head attention was implemented by creating separate weight matrices for each attention head. For instance, with two attention heads, two weight matrices for queries, two for keys, and two for values were generated. This involved multiple matrix multiplications at the outset. For models like GPT-3, which utilize 96 attention heads, this approach necessitates 96 multiplications for queries, 96 for keys, and 96 for values, which is computationally inefficient. This lecture introduces a more efficient method: multi-head attention with weight splits, aiming to reduce the number of initial matrix multiplications.

[\[0:05\]](#)

The core idea of multi-head attention with weight splits is to use larger initial weight matrices that encompass all attention heads. After performing a single matrix multiplication with these larger matrices, the resulting query, key, and value tensors are then split into components corresponding to each head. This contrasts with the previous approach where separate weight matrices were used for each head. The subsequent steps, such as calculating attention scores, weights, and context vectors, remain similar, but the initial computation is significantly streamlined. [\[4:15\]](#)

Previously, multi-head attention was implemented using a wrapper class and separate causal attention modules for each head. The results from these individual heads were then concatenated. The new approach integrates both the multi-head functionality and the causal attention logic into a single `MultiHeadAttention` class. This class achieves multi-head functionality by reshaping the query, key, and value tensors to incorporate dimensions for the number of heads and the head dimension. [\[8:08\]](#)

### Understanding the Input and Dimensions

The input to the multi-head attention mechanism is typically a three-dimensional tensor with dimensions representing batch size, number of tokens, and input embedding dimension (`d_in`). For example, a tensor of shape `(1, 3, 6)` signifies a batch size of 1, 3 tokens, and each token represented by a 6-dimensional embedding vector. [\[11:05\]](#)

Key decisions are made regarding the output dimension (`d_out`) and the number of attention heads. Often, `d_out` is set equal to `d_in`, as seen in GPT-based models. The number of heads is also specified. From these two values, the dimension of each head (`head_dim`) is calculated as  $d_{out} / num\_heads$ . For instance, if `d_out` is 6 and there are 2 heads, `head_dim` will be 3.

[12:24]

## Initializing Weight Matrices and Computing Q, K, V

Trainable weight matrices for the query ( $w_Q$ ), key ( $w_K$ ), and value ( $w_V$ ) are initialized. The dimensions of these matrices are  $d_{in}$  by  $d_{out}$ . These matrices are typically initialized using a neural network's linear layer with bias set to zero, as this is optimized for weight initialization and subsequent backpropagation. [13:28]

The input tensor is then multiplied by these initialized weight matrices ( $w_Q$ ,  $w_K$ ,  $w_V$ ) to obtain the query, key, and value matrices. If the input has dimensions ( $batch\_size$ ,  $num\_tokens$ ,  $d_{in}$ ) and the weight matrices are ( $d_{in}$ ,  $d_{out}$ ), the resulting Q, K, and V matrices will have dimensions ( $batch\_size$ ,  $num\_tokens$ ,  $d_{out}$ ). Each row in these matrices corresponds to a token, and the  $d_{out}$  dimensions represent the token's representation after projection. [16:45]

## Reshaping for Multi-Head Computation

To incorporate the concept of multiple heads, the  $d_{out}$  dimension is unrolled into two dimensions: the number of heads and the head dimension ( $head\_dim$ ). This transforms the three-dimensional Q, K, and V tensors into four-dimensional tensors. For example, a tensor of shape ( $batch\_size$ ,  $num\_tokens$ ,  $d_{out}$ ) becomes ( $batch\_size$ ,  $num\_tokens$ ,  $num\_heads$ ,  $head\_dim$ ). This reshaping allows computations to be performed in parallel across different heads. [19:40]

To facilitate parallel computation of attention scores per head, the dimensions are further rearranged. The tensor is transposed to group by the number of heads rather than by the number of tokens. This changes the shape from ( $batch\_size$ ,  $num\_tokens$ ,  $num\_heads$ ,  $head\_dim$ ) to ( $batch\_size$ ,  $num\_heads$ ,  $num\_tokens$ ,  $head\_dim$ ). This arrangement allows for direct multiplication of queries and keys belonging to the same head. [26:40]

## Calculating Attention Scores and Weights

Attention scores are calculated by multiplying the query tensor with the transpose of the key tensor. Due to the previous reshaping and transposing, this multiplication is performed independently for each head. The resulting attention scores matrix has dimensions ( $batch\_size$ ,  $num\_heads$ ,  $num\_tokens$ ,  $num\_tokens$ ). Each element in this matrix represents the attention score between a query token and a key token for a specific head. [29:23]

To implement causal attention, a mask is applied to the attention scores. Elements above the main diagonal are replaced with negative infinity. This ensures that a token can only attend to itself and preceding tokens. Before applying the softmax function, the attention scores are

scaled by dividing by the square root of the head dimension ( $\sqrt{\text{head\_dim}}$ ). This scaling helps stabilize gradients during training. [36:42]

The softmax function is then applied along the last dimension to obtain the attention weights. This ensures that the attention weights for each token sum up to one, representing a probability distribution over the tokens it attends to. The attention weights matrix retains the same dimensions as the attention scores:  $(\text{batch\_size}, \text{num\_heads}, \text{num\_tokens}, \text{num\_tokens})$ . [41:00]

## Computing Context Vectors and Final Output

The context vector matrix is computed by multiplying the attention weights with the value tensor. The dimensions of the attention weights are  $(\text{batch\_size}, \text{num\_heads}, \text{num\_tokens}, \text{num\_tokens})$ , and the dimensions of the values tensor are  $(\text{batch\_size}, \text{num\_heads}, \text{num\_tokens}, \text{head\_dim})$ . The resulting context vector matrix has dimensions  $(\text{batch\_size}, \text{num\_heads}, \text{num\_tokens}, \text{head\_dim})$ . [44:43]

To consolidate the information from multiple heads, the context vector matrix is reshaped. The dimensions corresponding to the number of heads and head dimension are brought together. This is achieved by transposing the tensor to group the number of heads closer to the head dimension, resulting in a shape of  $(\text{batch\_size}, \text{num\_tokens}, \text{num\_heads}, \text{head\_dim})$ . [47:48]

Finally, the tensor is reshaped to merge the `num_heads` and `head_dim` dimensions back into the `d_out` dimension. This results in the final context vector matrix with dimensions  $(\text{batch\_size}, \text{num\_tokens}, \text{d\_out})$ . An optional linear projection layer can be applied at the end to further refine the output. [51:00]

## Practical Implementation and Scaling

The implemented `MultiHeadAttention` class can handle multiple batches simultaneously. The example demonstrates creating a batch of two inputs, each with three tokens and a 6-dimensional embedding. The `d_out` is set to 6, and the number of heads is 2, resulting in a `head_dim` of 3. The output context vector matrix for each batch has dimensions  $(\text{num\_tokens}, \text{d\_out})$ , which is  $(3, 6)$  in this example. [54:15]

While the example uses small embedding sizes and a limited number of heads for clarity, real-world models like GPT-2 and GPT-3 employ significantly larger values. For instance, the smallest GPT-2 model has 12 attention heads and a context embedding size of 768, while GPT-3 can have up to 96 attention heads. In these models, `d_in` is typically equal to `d_out`. [57:45]

Understanding the intricate details of dimensions and linear algebra is crucial for becoming a proficient machine learning engineer. The ability to work with higher-dimensional tensors, such as the four-dimensional tensors encountered in multi-head attention, is a key skill. Writing down these concepts and visualizing the transformations can demystify complex operations and aid in deeper comprehension. [59:00]

\*End of Chapter 18. Source: <https://www.youtube.com/watch?v=K5u9eEaoxFg>\*

---

## Chapter 19: Birds Eye View of the LLM Architecture

---

*Source video: Lecture 19: Birds Eye View of the LLM Architecture*

*Duration: 48 minutes | Playlist position 19*

### Introduction to the LLM Architecture Module

This lecture series aims to build a large language model (LLM) from scratch, divided into three stages: foundational setup, pre-training, and fine-tuning. Currently, the focus is on Stage 1, which has covered data preparation and sampling, including tokenization, vector embeddings, and positional embeddings. Most recently, the attention mechanism was explored in detail through several lectures on simplified self-attention, causal attention, and multi-head attention. Understanding the attention mechanism is crucial as it forms the fundamental building block for comprehending subsequent topics. [\[0:05\]](#)

The upcoming lectures will delve into the LLM architecture itself. This module will be broken down into multiple videos, each covering specific aspects in detail and from scratch. The current lecture serves as the first in this module, providing an overview of how previously learned concepts like attention, input embeddings, and position embeddings integrate into a complete LLM, such as GPT. It will clarify where training and backpropagation occur and the role of neural networks within the architecture. [\[0:05\]](#)

### The Transformer Block and its Components

The LLM architecture is fundamentally a deep neural network, with the Transformer being a key component. The attention mechanism is central to Transformers, and this lecture will begin to unravel what a Transformer truly is, how training is performed, and how the next word is generated. The architecture will be explored layer by layer, starting with a bird's-eye view to understand how all the pieces fit together. [\[0:05\]](#)

The process begins with tokenization, followed by vector embedding and positional embedding. The resulting embedding vectors are then transformed into context vectors through masked multi-head attention. These context vectors are richer representations than embedding vectors, capturing both semantic meaning and relationships between tokens within a sentence. [\[0:05\]](#)

Masked multi-head attention is a critical part of the Transformer block, which is the most important component of the LLM architecture. Zooming into the Transformer block reveals its constituent parts. For an input sentence like "every effort moves you," each token is converted into an embedding vector, to which positional embeddings are added. [\[0:05\]](#)

The Transformer block includes layer normalization, followed by masked multi-head attention that converts input embeddings into context vectors. These are then passed through a dropout layer. Shortcut connections, indicated by plus signs, run from the input of a sub-layer to its output. Another layer normalization follows, then a feed-forward neural network, another dropout layer, and a final shortcut connection. The feed-forward network incorporates a "JLo" activation function. [0:05]

The entire architecture contains a large number of trainable parameters. During pre-training, these parameters are optimized. The output of the Transformer block has the same dimensions as the input and is further processed by output layers to generate the final text, predicting the next word. For the example "every effort moves you," the predicted next word is "forward." [0:05]

## Upcoming Lectures and GPT-2 Configuration

Subsequent lectures will focus on dissecting the Transformer block. The plan includes dedicated lectures on stacking layers, layer normalization, shortcut connections, and the feed-forward neural network with JLo activation. Finally, a lecture will cover how the Transformer output is decoded to produce the next word. The importance of the masked multi-head attention block is emphasized, as its removal would revert LLMs to older architectures like Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. [0:05]

The current lecture series will utilize the GPT-2 architecture, specifically the small version with 124 million parameters and 12 layers. This model has a `d_model` (embedding dimension) of 768. GPT-2 is chosen because it is smaller, allowing for local execution, and its weights are publicly available, unlike GPT-3 and GPT-4. [0:05]

The configuration for this series includes:

- Vocabulary Size: 50,257 (for GPT-2's byte pair encoder, a subword tokenizer). This size accounts for characters, subwords, and full words. ``
- Context Length: 1024 tokens, meaning the model considers up to 1024 tokens to predict the next one. ``
- Embedding Dimension (`d_model`): 768. Each token is projected into a 768-dimensional vector space, trained to capture semantic meaning. ``
- Number of Heads: 12 (for multi-head attention). This implies 12 sets of query, key, and value matrices. ``
- Number of Layers: 12 (Transformer blocks). ``
- Dropout Rate: A parameter for the dropout layers. ``
- Query Key Value Bias (`q_k_v_bias`): Set to False by default. ``



The GPT-2 small model has 12 Transformer blocks, while larger versions like medium, large, and extra-large have 24, 36, and 48 blocks, respectively, with increasing embedding dimensions. [0:05]

## Implementing a Dummy GPT Model

The next step is to implement a placeholder GPT architecture. This "dummy GPT model" will serve as a skeleton, allowing visualization of how different components fit together before they are coded in detail in subsequent lectures. [0:05]

The `forward` method of this dummy model takes an input, such as the sentence "every effort moves you," and aims to produce the next word, "forward." The core components to be implemented later are the Transformer block and the layer normalization block. Layer normalization appears both within the Transformer block and at other points in the architecture. [0:05]

The input, a sequence of tokens like "every effort moves you," is first converted into token IDs using a tokenizer (GPT-2 uses a byte pair encoder). These token IDs are then fed into the GPT model. [24:27]

Inside the model, each token ID is converted into a token embedding vector of dimension 768 using an `nn.Embedding` layer. This layer utilizes a token embedding matrix where each row corresponds to a token ID and contains its 768-dimensional vector representation. These embeddings are initially random and will be trained via backpropagation. [25:22]

Next, positional embeddings are added to the token embeddings. A positional embedding matrix, with dimensions `context_length` (1024) by `embedding_dimension` (768), is used. For each token's position in the sequence (e.g., 0, 1, 2, 3), a corresponding positional embedding vector is retrieved and added to the token embedding vector. This results in a combined embedding vector for each token. [25:22]

Following the addition of positional embeddings, a dropout layer (`drop_emb`) is applied to randomly zero out some embedding values, aiding generalization and preventing overfitting. [35:55]

The combined embeddings are then passed through the Transformer block, which comprises multiple layers including multi-head attention, dropout, shortcut connections, layer normalization, and a feed-forward neural network with `JLo` activation. This process is repeated for 12 Transformer blocks in GPT-2. A final layer normalization is applied before the output head. [35:55]

## Output Logits and Prediction

The output of the Transformer blocks is a sequence of vectors, each with a dimension of 768. To predict the next word, this output is passed through an output head, typically a linear layer, which transforms the 768-dimensional vectors into logits. The logits vector has a dimension equal to the vocabulary size (50,257). Each element in this vector represents the unnormalized probability of a specific token being the next word in the sequence. [35:55]

For an input sequence of length  $N$ , there are  $N$  prediction tasks happening in parallel. For example, with the input "every effort moves you" (length 4), the model predicts:

1. "effort" after "every"
2. "moves" after "every effort"
3. "you" after "every effort moves"
4. "forward" after "every effort moves you"

Therefore, the output logits tensor will have dimensions (batch\_size, sequence\_length, vocabulary\_size). In the example, for a batch size of 1 and sequence length of 4, the output shape would be (1, 4, 50257). [35:55]

The code demonstrates passing a batch of two text samples through the dummy GPT model. The output shape confirms the expected (2, 4, 50257) dimensions, where each token's 50,257-dimensional vector represents the probability distribution for the next token. These probabilities are initially random and will be refined through backpropagation and training. Later, a softmax function will be applied to convert these logits into probabilities between 0 and 1. [35:55]

## Summary and Next Steps

This lecture provided a top-down view of the GPT architecture, outlining inputs, outputs, and the overall workflow. The subsequent lectures will sequentially detail each component: layer normalization, feed-forward networks, shortcut connections, the Transformer block, and finally, the complete GPT model implementation. The last lecture in this series will cover post-processing to convert the output logits into decodable token IDs and then into human-readable text. [0:05]

The GPT backbone is composed of layer normalization, JLo activation, feed-forward networks, and shortcut connections, which collectively form the Transformer block. The attention mechanism is a crucial component within the Transformer, but other elements are equally important for its functionality. [0:05]

---

\*End of Chapter 19. Source: <https://www.youtube.com/watch?v=4i23dYoXp-A>\*

---

## Chapter 20: Layer Normalization in the LLM Architecture

---

*Source video: Lecture 20: Layer Normalization in the LLM Architecture*

*Duration: 38 minutes | Playlist position 20*

### Introduction to Layer Normalization

In this lecture, we delve into a specific component of the large language model (LLM) architecture: layer normalization. Previously, we established a high-level view of the LLM architecture, including the GPT backbone, which comprises several components arranged together. [0:05]

The GPT architecture consists of token embedding, positional embedding, Transformer blocks, and output layers. Within each Transformer block, there are multiple layers, including layer normalization, multi-head attention, dropout, and shortcut connections. The feed-forward network, also part of the Transformer block, includes linear layers and activation functions. [0:05]

Layer normalization appears multiple times within the Transformer block, both before and after the multi-head attention layer. It also appears outside the Transformer block, after the output of the Transformer blocks and before the final output layers. This widespread use necessitates defining a separate class for layer normalization. [0:05]

### The Need for Layer Normalization

The primary motivation for using layer normalization is to improve the stability and efficiency of neural network training, particularly in deep networks with many stacked layers. Deep neural networks can suffer from vanishing or exploding gradient problems, leading to unstable training dynamics. [0:05]

During the backward pass, gradients are propagated through layers. If the outputs of a layer are excessively large or small, the gradient magnitudes can become too large (exploding gradients) or too small (vanishing gradients). Layer normalization helps control the magnitude of layer outputs, thereby stabilizing gradient magnitudes and ensuring a more stable learning procedure. [0:05]

Another significant advantage of layer normalization is its ability to prevent internal covariate shift. As training progresses, the distribution of inputs to each layer can change, making it difficult for the network to learn. Layer normalization fixes the mean and standard deviation of the outputs from each layer, reducing this shift and accelerating convergence. [0:05]

### The Mechanism of Layer Normalization

The core idea of layer normalization is to adjust the outputs of a specific layer so that they have a mean of zero and a variance of one. For a given layer's outputs (e.g.,  $X_1, X_2, X_3, X_4$ ), we first calculate their mean ( $\bar{x}$ ) and variance ( $s^2$ ). Then, each output is normalized by subtracting the mean and dividing by the standard deviation (the square root of the variance).

[\[0:05\]](#)

This normalization procedure ensures that the resulting normalized values have a mean of zero and a variance of one. This simple process contributes to the stability of neural network training and mitigates the problem of internal covariate shift, leading to faster convergence.

[\[0:05\]](#)

In the GPT architecture, layer normalization is applied before and after the multi-head attention module within the Transformer block, and also before the final output layer. This results in its application at multiple points, reinforcing the need for a dedicated class. [\[0:05\]](#)

## Implementing Layer Normalization in Code

To illustrate layer normalization, we can consider a simple neural network layer with multiple batches of inputs. For each batch, the layer produces several outputs. Layer normalization is then applied independently to each batch. [\[21:03\]](#)

When calculating the mean and variance, we use `output.mean(dim=-1, keepdim=True)` and `output.var(dim=-1, keepdim=True)`. The `dim=-1` argument specifies that the operation should be performed along the columns (features), and `keepdim=True` ensures that the dimensions of the mean and variance tensors are preserved for subsequent calculations.

[\[21:03\]](#)

The normalization step involves subtracting the calculated mean from the layer outputs and dividing by the square root of the variance. A small constant, epsilon ( $\epsilon$ ), is added to the variance to prevent division by zero. [\[21:03\]](#)

After applying layer normalization, the mean of the normalized outputs for each batch should be close to zero, and the variance should be close to one. This can be verified by printing the mean and variance of the normalized outputs. [\[21:03\]](#)

## The Layer Normalization Class

We define a `LayerNorm` class to encapsulate the normalization logic. This class takes the embedding dimension as an argument during initialization, as it will be used for trainable scale and shift parameters. [\[27:19\]](#)

The `forward` method of this class performs the normalization. It calculates the mean and variance along the column dimension (embedding dimension), subtracts the mean, and divides by the square root of the variance (plus epsilon). [\[27:19\]](#)

Additionally, the `LayerNorm` class includes learnable scale ( $\gamma$ ) and shift ( $\beta$ ) parameters. These parameters, with dimensions matching the embedding dimension, allow the model to fine-tune the normalization process during training. The normalized output is then scaled by  $\gamma$  and shifted by  $\beta$ . [27:19]

A detail regarding variance calculation is the `unbiased` parameter. Setting `unbiased=False` means we divide by  $N$  (the number of elements) instead of  $N-1$  (Bessel's correction). For large embedding dimensions, the difference is negligible. This implementation detail is chosen for compatibility with models like GPT-2 and TensorFlow's default behavior. [27:19]

To test the `LayerNorm` class, we create an instance with an embedding dimension and pass a sample batch through it. The resulting normalized output, when its mean and variance are checked, confirms that the values have been normalized to have a mean of zero and a variance of one. [27:19]

## Layer Normalization vs. Batch Normalization

It is crucial to distinguish between layer normalization and batch normalization. Layer normalization normalizes across the feature dimension (columns) and is independent of the batch size. This independence provides flexibility, especially in distributed training environments or when hardware limitations necessitate smaller batch sizes. [0:05]

Batch normalization, on the other hand, normalizes across the entire batch. Its performance is dependent on the batch size, making it less flexible when batch sizes vary or are small. Layer normalization's independence from batch size makes it a more robust choice for LLM architectures. [0:05]

---

\*End of Chapter 20. Source: <https://www.youtube.com/watch?v=G3W-LT79LSI>\*

---

## Chapter 21: GELU Activation and Feed-Forward Networks in LLMs

---

*Source video: GELU Activation Function in the LLM Architecture*

*Duration: 27 minutes | Playlist position 21*

### Introduction to Transformer Block Components

The Transformer block, a core component of the GPT architecture, is comprised of several stacked layers. These include layer normalization, multi-head attention, dropout, shortcut connections, and a feed-forward neural network. This lecture focuses on the GELU (Gaussian Error Linear Unit) activation function and the feed-forward neural network, which are integral parts of this block. [0:05]

The feed-forward neural network within the Transformer block consists of linear layers and the GELU activation function. While linear layers are standard in neural networks, the GELU activation is less common than ReLU, sigmoid, or Tanh, and is a key element to understand for large language models. [0:05]

### Understanding Activation Functions: ReLU vs. GELU

The ReLU (Rectified Linear Unit) activation function returns the input value for positive inputs and zero for negative inputs. A key issue with ReLU is its non-differentiability at  $x=0$ , which can lead to the "dead neuron" problem where neurons become inactive and stop contributing to the learning process if their output becomes negative. [0:05]

The GELU activation function, mathematically represented as  $\text{GELU}(x) = x * \Phi(x)$ , where  $\Phi(x)$  is the cumulative distribution function of the standard Gaussian distribution, offers an alternative. Unlike ReLU, GELU does not output zero for negative inputs, thus mitigating the dead neuron problem. [0:05]

For practical implementation, an approximation of the GELU function is often used, particularly the one employed in the training of GPT-2. This approximation is given by  $0.5 * x * (1 + \tanh(\sqrt{2/\pi} * (x + 0.044715 * x^3)))$ . [0:05]

Comparing GELU and ReLU visually, GELU is smooth and differentiable across all values of  $x$ , whereas ReLU has a sharp corner at  $x=0$ . For negative inputs, GELU produces small non-zero outputs, preventing neurons from becoming dead. For positive inputs, GELU closely resembles ReLU, being nearly linear. [0:05]

The advantages of GELU over ReLU in large language models include its smoothness, which aids optimization, and its prevention of the dead neuron problem. Experimental results generally show that GELU performs better than ReLU in LLMs. [0:05]

## Implementing the GELU Activation Function

A Python class for the GELU activation function can be implemented using the approximated formula. The `forward` method takes an input `x` and returns the result of the GELU computation. This implementation mirrors the activation function used in GPT-2. [0:05]

```
import torch
import torch.nn as nn
import math

class GELU(nn.Module):
    def forward(self, x):
        return 0.5 * x * (1 + torch.tanh(math.sqrt(2 / math.pi) * (x + 0.044715 * torch.pow(x, 3))
```

## The Feed-Forward Neural Network Architecture

The feed-forward neural network within the Transformer block typically consists of two linear layers with a GELU activation function in between. This structure is often described as an "expansion-contraction" network. [0:05]

For a token with an embedding dimension of 768 (as in the smallest GPT-2 model), the first linear layer expands this dimension to four times its size, resulting in 3072 dimensions. This expansion allows the network to capture richer properties between inputs. [15:47]

Following the expansion, the GELU activation function is applied. Subsequently, the second linear layer contracts the dimension back to the original embedding size of 768. This expansion-contraction design is powerful because it preserves the input dimensionality while enabling a richer exploration of the representation space. [15:47]

Crucially, this feed-forward network operates on each token independently, unlike the attention mechanism which considers relationships between tokens. It modifies each dimension of a single token's representation. [15:47]

## Implementing the Feed-Forward Network in PyTorch

The feed-forward network can be implemented in PyTorch using `nn.Sequential` to chain the layers together. The `__init__` method defines a sequence containing a linear layer for expansion, the GELU activation, and a linear layer for contraction. [18:27]

The input dimension to the first linear layer is the embedding dimension (`cfg.n_embd`), and the output dimension is `4 * cfg.n_embd`. The second linear layer takes `4 * cfg.n_embd` as input and outputs `cfg.n_embd`, ensuring the output dimensionality matches the input. [18:27]

```
import torch
import torch.nn as nn
import math
```

```
# Assuming GELU class is defined as above

class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        # Expansion layer
        self.layers = nn.Sequential(
            nn.Linear(cfg.n_embd, 4 * cfg.n_embd),
            GELU(),
            # Contraction layer
            nn.Linear(4 * cfg.n_embd, cfg.n_embd),
        )

    def forward(self, x):
        return self.layers(x)
```

The feed-forward module's expansion-contraction design enhances the model's ability to learn and generalize by exploring a richer representation space internally, even though the input and output dimensions remain the same. This uniformity in dimensions across layers is a key aspect of the GPT architecture, enabling scalability and the stacking of multiple Transformer blocks. [18:27]

In the GPT-2 architecture, each of the 12 Transformer blocks contains such a feed-forward network, contributing significantly to the model's overall representational power. [18:27]

---

\*End of Chapter 21. Source: [https://www.youtube.com/watch?v=d\\_PiwZe8UF4](https://www.youtube.com/watch?v=d_PiwZe8UF4)\*

---



## Chapter 22: Shortcut Connections in the LLM Architecture

---

*Source video: Shortcut connections in the LLM Architecture*

*Duration: 32 minutes | Playlist position 22*

### Introduction to Shortcut Connections

This lecture introduces shortcut connections, also known as skip connections or residual connections, as a crucial component in the architecture of large language models (LLMs). These connections address the significant challenge of vanishing gradients during the training of deep neural networks. The GPT architecture is composed of several building blocks, including layer normalization, the GELU activation with a feed-forward neural network, and now, shortcut connections. Understanding these components is essential for grasping the overall Transformer block, which forms the core of the GPT architecture. [0:05]

The Transformer block, a key element in LLMs, incorporates several components: layer normalization, masked multi-head attention, dropout layers, and a feed-forward neural network. The shortcut connections are visually represented by plus symbols within the Transformer block diagram, indicating an alternative path for information flow alongside the primary linear progression. [1:30]

### The Vanishing Gradient Problem

The primary problem that shortcut connections solve is the vanishing gradient problem. This issue arises when gradients become progressively smaller as they are propagated backward through a deep neural network. When gradients become extremely small, the weight updates are minimal, leading to stagnant learning and delayed convergence, hindering the effective training of LLMs. [2:30]

Consider a deep neural network with multiple hidden layers. During the backward pass, gradients are calculated with respect to the weights of each layer, flowing from the output layer towards the input layer. If the gradients in later layers are small, multiplying them with subsequent backpropagated values results in even smaller gradients in earlier layers. This multiplicative effect causes gradients to shrink exponentially, eventually approaching zero by the time they reach the initial layers. [3:00]

The weight update rule in neural networks is typically  $w_{\text{new}} = w_{\text{old}} - \eta * \frac{\partial L}{\partial w}$ , where  $\frac{\partial L}{\partial w}$  represents the gradient of the loss with respect to the weight  $w$ , and  $\eta$  is the learning rate. If the gradient  $\frac{\partial L}{\partial w}$  approaches zero, the weight  $w$  will not be updated, effectively halting the learning process for that parameter. This stagnation prevents the network from learning effectively. [4:15]

## How Shortcut Connections Work

Shortcut connections create an alternative path for gradients to flow by skipping one or more layers. This is achieved by adding the output of one layer to the output of a subsequent layer. Visually, this is represented by arrows that bypass layers, allowing gradients to flow more directly. [5:10]

Without shortcut connections, the gradient magnitudes can diminish significantly as they propagate backward. For instance, a gradient magnitude of 0.5 in the outermost layer might decrease to 0.0013 in an inner layer and further to 0.007 in an even earlier layer, illustrating the vanishing gradient problem. [5:50]

With shortcut connections, the output of an earlier layer is added to the output of a later layer. This addition ensures that even if the gradient through the intermediate layers becomes small, the gradient from the shortcut path, which includes a direct connection, remains significant. This additive effect prevents the overall gradient from vanishing. [6:30]

Mathematically, consider a layer  $l+1$  whose output  $y_{l+1}$  is computed from the output of the previous layer  $y_l$ . Without shortcut connections,  $y_{l+1} = f(y_l)$ , where  $f$  is the function of the layer. With shortcut connections, the output becomes  $y_{l+1} = f(y_l) + y_l$ . When calculating the gradient of the loss  $L$  with respect to  $y_l$  (i.e.,  $\partial L / \partial y_l$ ), the chain rule gives:  $\partial L / \partial y_l = \partial L / \partial y_{l+1} * \partial y_{l+1} / \partial y_l$ . Since  $\partial y_{l+1} / \partial y_l = \partial f(y_l) / \partial y_l + \partial y_l / \partial y_l = \partial f(y_l) / \partial y_l + 1$ , the gradient becomes  $\partial L / \partial y_l = \partial L / \partial y_{l+1} * (\partial f(y_l) / \partial y_l + 1)$ . The  $+1$  term ensures that  $\partial L / \partial y_l$  does not vanish, even if  $\partial f(y_l) / \partial y_l$  becomes very small. [8:30]

This  $+1$  term is the core contribution of shortcut connections. It provides a direct path for the gradient, ensuring that  $\partial L / \partial y_l$  remains substantial, which in turn helps maintain significant gradients for the weights in layer  $l$ , preventing learning stagnation. [10:00]

## Visualizing the Impact of Shortcut Connections

The impact of shortcut connections can be visualized by examining the loss landscape of a neural network. Without skip connections, the loss landscape often appears rugged, with numerous local minima, peaks, and valleys, making optimization difficult. However, when skip connections are included, the loss landscape becomes smoother, typically exhibiting a single, well-defined local minimum, which facilitates easier and more stable gradient descent. [11:30]

## Code Implementation of Shortcut Connections

To demonstrate the effect of shortcut connections, a Python class `ExampleDeepNeuralNetwork` is implemented using PyTorch. This class allows for the

creation of a deep neural network with a specified number of layers and neurons per layer. It also includes a `use_shortcut` flag to enable or disable shortcut connections. [13:00]

The `__init__` method constructs the neural network using `nn.Sequential`, chaining linear layers and GELU activation functions. The layer sizes are defined by a list, where each element specifies the number of neurons in a layer. The input and output dimensions of each layer are determined by consecutive elements in this list. [14:00]

The `forward` method implements the network's forward pass. If shortcut connections are enabled (`self.use_shortcut` is `True`), the input `x` is added to the output of each layer before passing it to the next. This accumulation of shortcut connections ensures that the output at each stage incorporates information from previous layers directly. [15:30]

```
import torch
import torch.nn as nn

class ExampleDeepNeuralNetwork(nn.Module):
    def __init__(self, layer_sizes, use_shortcut=False):
        super().__init__()
        self.use_shortcut = use_shortcut
        layers = []
        for i in range(len(layer_sizes) - 1):
            layers.append(nn.Linear(layer_sizes[i], layer_sizes[i+1]))
            if i < len(layer_sizes) - 2: # Add GELU for all but the last layer
                layers.append(nn.GELU())
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        if self.use_shortcut:
            shortcut_input = x
            for i, layer in enumerate(self.network):
                if isinstance(layer, nn.Linear):
                    # Apply the linear layer
                    x = layer(x)
                    # Add shortcut connection if it's not the last linear layer
                    if i < len(self.network) - 1 and isinstance(self.network[i+1], nn.GELU):
                        x = x + shortcut_input
                        shortcut_input = x # Update shortcut_input for the next linear layer
                    elif i == len(self.network) - 1: # Handle the case for the last layer if no GELU
                        x = x + shortcut_input
                else: # GELU activation
                    x = layer(x)
            return x
        else:
            return self.network(x)
```

## Demonstrating Gradient Flow Without Shortcut Connections

To observe the vanishing gradient problem, a neural network is initialized without shortcut connections (`use_shortcut=False`). A sample input is provided, and a squared loss is

calculated against a target of zero. The `loss.backward()` method computes the gradients for all parameters. The mean absolute gradient for each layer's weight matrix is then calculated and printed. [17:00]

The results show a significant decrease in gradient magnitudes as they propagate backward. For instance, layer 4 might have a mean gradient of 0.05, layer 3 of 0.005, layer 2 of 0.0013, and layer 1 of 0.0007. The first layer's gradient approaches zero, illustrating the vanishing gradient problem. [19:00]

## Demonstrating Gradient Flow With Shortcut Connections

Next, a neural network is instantiated with shortcut connections enabled (`use_shortcut=True`). The same procedure of calculating and printing mean absolute gradients is followed. The presence of shortcut connections creates alternative paths for gradient flow. [20:00]

With shortcut connections, the gradient magnitudes remain much larger and more stable across layers. For example, layer 4 might have a mean gradient of 1.32, layer 3 of 0.26, layer 2 of 0.32, and layer 1 of 0.22. These values are significantly higher than those observed without shortcut connections, demonstrating that the vanishing gradient problem is effectively mitigated. [21:30]

This practical demonstration provides empirical evidence that shortcut connections are crucial for maintaining effective gradient flow during the training of deep neural networks, preventing learning stagnation. [22:30]

## Conclusion and Application in Transformer Blocks

In summary, shortcut connections are vital for overcoming the vanishing gradient problem in deep neural networks and are a core component of LLMs like GPT. They ensure consistent gradient flow across layers, facilitating more effective training. The conceptual understanding and practical implementation shown highlight how these connections stabilize training, which is essential for constructing robust LLM architectures. [23:30]

The arrows visually representing shortcut connections within the Transformer block diagram are now more understandable. These connections provide alternative gradient paths, ensuring that gradients do not vanish during the backward propagation through the entire block, thereby enabling stable learning. [24:30]

Having covered layer normalization, GELU activation, feed-forward networks, and shortcut connections, all the necessary building blocks are in place to understand the Transformer block in detail in the upcoming lecture. [25:00]

\*End of Chapter 22. Source: <https://www.youtube.com/watch?v=2r0QahNdwMw>\*

---

## Chapter 23: Coding the Transformer Block

---

*Source video: Coding the entire LLM Transformer Block*

*Duration: 45 minutes | Playlist position 23*

### Introduction to the Transformer Block

The Transformer block is the fundamental building block of large language models (LLMs) like GPT. It integrates several previously developed components: layer normalization, the JALU activation function, feed-forward neural networks, and shortcut connections. This lecture focuses on coding the entire Transformer block, which is repeated multiple times in architectures like GPT-2. [\[0:05\]](#)

The Transformer block fits within the larger LLM stack after input tokenization, embedding, and positional encoding. Within the block, a specific sequence of operations occurs: layer normalization, masked multi-head attention, dropout, a shortcut connection, another layer normalization, a feed-forward network with JALU activation, another dropout, and a final shortcut connection. [\[0:05\]](#)

In the GPT-2 architecture, the Transformer block is repeated 12 times. The code developed in this lecture will be replicated for each of these blocks. The core components that make up the Transformer block are masked multi-head attention, layer normalization, dropout, the feed-forward neural network, and the JALU activation function. [\[0:05\]](#)

### Recap of Transformer Block Components

Multi-head attention takes input vectors and transforms them into context vectors. This process involves trainable query, key, and value matrices, with multiple "heads" allowing the model to attend to different parts of the input simultaneously. The dot product of queries and keys yields attention scores, which are normalized into attention weights. These weights are then multiplied by the value vectors to produce context vectors, which are richer representations than simple embedding vectors as they capture relationships between tokens. [\[0:05\]](#)

Layer normalization normalizes the outputs of a layer to have a mean of zero and a variance of one. This stabilizes training by preventing exploding or vanishing gradients and mitigating the internal covariate shift problem, where the distribution of layer inputs changes during training. Layer normalization is applied twice within the Transformer block, before the multi-head attention and before the feed-forward network. [\[0:05\]](#)

Dropout is a regularization technique where a fraction of neurons are randomly turned off during training. This prevents overfitting and improves generalization by forcing "lazy"

neurons to learn and update their weights, as they cannot rely on other neurons being present in every iteration. [0:05]

The feed-forward neural network within the Transformer block consists of an expansion layer, a JALU activation, and a contraction layer. The expansion layer increases the dimensionality (typically by a factor of four) to explore a richer parameter space, while the contraction layer brings it back to the original embedding dimension. This preserves the input dimensionality, allowing for stacking of blocks. [0:05]

The JALU activation function is a variation of ReLU that is non-zero for negative inputs and differentiable at zero, helping to solve the "dead neuron" problem where neurons can become inactive. Experiments show JALU often performs better than ReLU in LLMs. [0:05]

Shortcut connections add the output of a layer to its input. This provides an alternative path for gradients to flow during backpropagation, effectively solving the vanishing gradient problem and ensuring more stable learning. [0:05]

## Constructing the Transformer Block

The Transformer block follows a specific sequence of operations: layer normalization, multi-head attention, dropout, and a shortcut connection. This is followed by a second layer normalization, a feed-forward network, another dropout, and a final shortcut connection. Crucially, all operations within the Transformer block are designed to preserve the dimensionality of the input vectors. This preservation of dimensionality is key to stacking multiple Transformer blocks together. [0:05]

The multi-head attention mechanism analyzes relationships between input elements, assigning attention scores, while the feed-forward network processes each element independently. The output of the Transformer block, while having the same dimensions as the input, is a context vector that encapsulates information about how each token relates to others in the sequence. [0:05]

The configuration used for the GPT-2 model includes a vocabulary size of 50257, a context length of 1024, an embedding dimension of 768, 12 attention heads, and 12 layers (Transformer blocks). A dropout rate of 0.1 is applied, and query, key, and value biases are set to false. [0:05]

The `TransformerBlock` class in PyTorch incorporates instances of `nn.LayerNorm`, `nn.MultiheadAttention`, `nn.Dropout`, and the custom feed-forward network. The forward method implements the sequential application of these components, including the shortcut connections where the output of a dropout layer is added to the input of the preceding normalization layer. [30:52]

The shortcut connections are implemented by adding the input  $x$  to the output of the dropout layers. This ensures that the original information is preserved and passed along with the transformed information, aiding gradient flow. [30:52]

The Transformer block returns the modified input, which has the same dimensions as the original input vector. This dimensionality preservation is a critical design feature that allows for the easy stacking of multiple Transformer blocks. [30:52]

### **Demonstrating Dimensionality Preservation**

To demonstrate the dimensionality preservation, an input tensor  $x$  with a shape of  $(2, 4, 768)$  is created, representing two batches, four tokens per batch, and an embedding dimension of 768. This input is passed through an instance of the `TransformerBlock` class. [30:52]

The output shape of the Transformer block is confirmed to be  $(2, 4, 768)$ , identical to the input shape. This confirms that the Transformer block processes the data without altering its shape, a crucial aspect enabling its scalability and application in sequence-to-sequence tasks. [30:52]

While the physical dimensions remain unchanged, the content of each output vector is re-encoded to integrate contextual information from the entire input sequence through the attention mechanism. This rich output vector contains information about how each token relates to others in the input sequence. [39:10]

The Transformer block is a core component of the GPT architecture, built upon the attention mechanism. Understanding the Transformer block is essential for comprehending the entire GPT architecture and how it predicts the next word in a sequence. [39:10]

---

\*End of Chapter 23. Source: <https://www.youtube.com/watch?v=dvH6lFGhFrS>\*

---



## Chapter 24: Assembling the GPT Model Architecture

---

*Source video: Coding the 124 million parameter GPT-2 model*

*Duration: 61 minutes | Playlist position 24*

### Introduction to the GPT Architecture Assembly

This lecture focuses on constructing the entire GPT architecture, coding the complete GPT model by assembling previously developed building blocks. The process involves integrating layer normalization, feed-forward networks, shortcut connections, and the Transformer block, culminating in the final GPT model. [\[0:05\]](#)

The overall GPT architecture begins with tokenized text, followed by dropout and input embeddings. These embeddings are then processed by a series of Transformer blocks. Finally, another layer normalization and a linear output layer are applied to produce the model's output. [\[0:05\]](#)

The primary goal of the GPT model is next-word prediction. Given an input text sequence, the model aims to predict the subsequent word. This lecture will cover the steps from input text to the final output tensor, with the subsequent lecture focusing on decoding this tensor to predict the next word. [\[0:05\]](#)

Previously, a dummy GPT model class was introduced with several components left blank. This lecture will fill in these components, including the Transformer block and layer normalization, to create a fully functional GPT model. [\[0:05\]](#)

The objective is to assemble a working version of the original 124 million parameter GPT-2 model. This involves passing input text through the entire architecture to obtain an output, all executable on a local machine. [\[0:05\]](#)

### Understanding the Flow and Dimensions

To facilitate understanding, a visual flowchart and dimension analysis are presented. This approach helps clarify the flow of data and the transformations occurring at each step, particularly for students who find tensor dimensions confusing. [\[0:05\]](#)

Consider the input sentence "every effort moves you." The GPT model processes this sequence to predict the next word, "forward." The process involves several stages, starting with tokenization and embedding. [\[0:05\]](#)

Inputs are typically processed in batches. For simplicity, the analysis focuses on a single batch of four tokens: "every," "effort," "moves," and "you." The GPT-2 vocabulary size is 50,257. [\[0:05\]](#)

The first step is token embedding, where each token is mapped to a vector of dimension 768. These vectors are initially initialized randomly and learned during training to capture semantic meaning. [0:05]

The second step involves positional embedding. Each position in the sequence (1 through 4 in this example) is also represented by a 768-dimensional vector, initialized randomly and learned during training. [0:05]

In the third step, token embeddings and positional embeddings are added together to create input embeddings. This combined embedding for each token has a dimension of 768. [0:05]

The fourth step introduces dropout, where a certain percentage of elements in each input embedding vector are randomly set to zero. This technique helps prevent overfitting and improve generalization. [0:05]

## The Transformer Block

After dropout, the input embeddings enter the Transformer block. The first component within the Transformer block is layer normalization, which normalizes the embedding values for each token to have a mean of zero and a variance of one. [0:05]

The core of the Transformer block is the masked multi-head attention mechanism. This mechanism allows the model to weigh the importance of different tokens in the sequence when processing a given token, generating context vectors. [0:05]

Following attention, another dropout layer is applied to the context vectors. Subsequently, a shortcut connection adds the output of the dropout layer back to the input of the attention layer, aiding gradient flow and preventing vanishing gradients. [0:05]

Another layer normalization is applied, followed by a feed-forward neural network. This network typically expands the dimensionality of the embeddings, processes them, and then contracts them back to the original dimension, allowing for richer parameter exploration. [0:05]

A final dropout layer and another shortcut connection complete the Transformer block. The output of the Transformer block retains the original embedding dimension, ensuring dimensionality is preserved throughout these operations. [0:05]

The Transformer block itself comprises eight distinct steps: layer normalization, masked multi-head attention, dropout, shortcut connection, followed by another layer normalization, feed-forward network, dropout, and a final shortcut connection. [31:24]

## Final Output Layers and Logits

After the Transformer blocks, an additional layer normalization is applied to the output. This step further stabilizes the representations before they are passed to the final output head. [\[31:24\]](#)

The output head is a linear layer that maps the 768-dimensional embeddings to the vocabulary size (50,257). This layer produces logits, which are unnormalized log probabilities for each token in the vocabulary. [\[31:24\]](#)

The logits matrix has dimensions corresponding to the batch size, the sequence length, and the vocabulary size. For an input of two batches with four tokens each, the output shape is (2, 4, 50257). [\[31:24\]](#)

This final output tensor contains the predictions for each token in the sequence, enabling next-word prediction. The process involves multiple prediction tasks, one for each token in the input sequence. [\[31:24\]](#)

## GPT Configuration and Implementation

The GPT configuration for this model includes a vocabulary size of 50,257, a context length of 1024, and an embedding dimension of 768. It also specifies 12 attention heads, 12 Transformer blocks, and a dropout rate of 0.1. [\[37:57\]](#)

The implementation chains 12 Transformer blocks together using `nn.Sequential`. Each Transformer block contains the eight previously detailed steps. [\[37:57\]](#)

The `forward` method orchestrates the process: it computes token and positional embeddings, adds them, applies dropout, passes the result through the Transformer blocks, applies final layer normalization, and computes the logits via the output head. [\[37:57\]](#)

A test run with an input batch demonstrates the model's output shape, confirming it matches the expected dimensions of (batch\_size, sequence\_length, vocabulary\_size). [\[37:57\]](#)

## Parameter Count and Weight Tying

The implemented model has approximately 163 million parameters. This count is higher than the 124 million parameters of the original GPT-2 model due to the use of separate token embedding and output layers. [\[37:57\]](#)

The original GPT-2 model employed weight tying, reusing the parameters from the token embedding layer for the output layer. This technique reduces the total parameter count and memory footprint. [\[37:57\]](#)

While weight tying is efficient, using separate layers for token embedding and the output head can lead to better training and model performance, which is why modern LLMs often adopt this approach. [\[37:57\]](#)

The total memory requirement for the 163 million parameters, assuming each parameter is a 32-bit float (4 bytes), is approximately 600 megabytes. This highlights the significant storage needed for even relatively small LLMs. [37:57]

## Conclusion and Next Steps

In this lecture, the GPT model architecture was implemented, producing numerical tensor outputs. The next lecture will focus on converting these logits into tokens and generating text. [1:00:07]

Despite its complexity, the core GPT model implementation can be achieved with a concise amount of code. Understanding the visual flow map and the underlying theory is crucial for comprehending these lines of code. [1:00:07]

Creating visual flow maps, similar to the one demonstrated, can significantly enhance understanding and confidence in working with LLM architectures. [1:00:07]

\*End of Chapter 24. Source: <https://www.youtube.com/watch?v=G3-JgHckzjw>\*

---

## Chapter 25: Generating Text from GPT-2 Output

---

*Source video: Coding GPT-2 to predict the next token*

*Duration: 40 minutes | Playlist position 25*

### Understanding Text Generation

A large language model generates tokens sequentially, predicting the next token based on a given sequence of input tokens. The model considers a context size, which is the maximum number of tokens it looks at before predicting the next one. In each iteration, the newly generated token is appended to the input sequence for the subsequent prediction. This iterative process continues until a predefined maximum number of new tokens has been generated. For instance, if the input is "hello I am" and the model predicts "a", the next input becomes "hello I am a". This continues until the desired output length is achieved. The context size in the GPT-2 architecture was 1024 tokens. [\[0:05\]](#)

The process begins with input tokens, which are converted into token IDs. These IDs are then transformed into token embeddings, to which positional embeddings are added, resulting in input embeddings. A dropout layer is applied, and the output is passed through a Transformer block. Within the Transformer block, a normalization layer is followed by multi-head attention, which enriches the embeddings with contextual information. After multi-head attention, there's a dropout layer and a shortcut connection, followed by another normalization layer, a feed-forward neural network, another dropout layer, and a final shortcut connection. The output from the Transformer block then goes through a final layer normalization and an output head, which produces the output tensor. [\[3:47\]](#)

The output tensor from the GPT model has dimensions of batch size, number of tokens, and vocabulary size. For an input sequence of four tokens, the model performs four input-output prediction tasks. The first prediction task is to predict the token following the first input token, the second task predicts the token after the first two input tokens, and so on. The final prediction task uses the entire input sequence to predict the next token. The output tensor's dimensions are batch size by number of tokens by vocabulary size. [\[7:16\]](#)

To generate text, we first extract the last vector from the output tensor, which corresponds to the prediction for the final token in the input sequence. This vector, known as the logits vector, is then converted into a set of probabilities using the softmax function. The token ID with the highest probability is identified, and its corresponding word is decoded from the vocabulary. This process is repeated iteratively: the newly generated token ID is appended to the input sequence, and the model predicts the next token. This continues until the maximum number of new tokens is generated. [\[10:05\]](#)

The iterative text generation process involves several steps. First, the output tensor from the model is examined. Second, the last vector (logits) from this tensor is extracted for each batch. Third, the softmax function is applied to convert these logits into probabilities. Fourth, the index (token ID) with the highest probability is identified. Fifth, this token ID is appended to the previous input tokens for the next iteration. This cycle repeats until the specified maximum number of new tokens is generated. [14:00]

The number of iterations in text generation is determined by the `max_new_tokens` parameter. For example, if `max_new_tokens` is set to six, the process will run for six iterations. The final output is the concatenation of the initial input tokens and the generated tokens. This mechanism allows the GPT model to generate novel text sequences that were not explicitly present in its training data. [17:30]

## Implementing Text Generation in Code

The `generate_text_simple` function is implemented to generate text. It takes the model, input token IDs (`idx`), the maximum number of new tokens to generate (`max_new_tokens`), and the context size as arguments. Initially, the input `idx` is processed to ensure its length does not exceed the `context_size` by taking the last `context_size` tokens. [19:30]

The processed input `idx` is then passed to the GPT model, which returns a logits tensor with dimensions (`batch_size`, `num_tokens`, `vocab_size`). To predict the next token, the last row (corresponding to the last token in the sequence) is extracted from this logits tensor for each batch using `logits[:, -1, :]`. This results in a tensor of shape (`batch_size`, `vocab_size`). [22:00]

The extracted logits are then converted into probabilities using the softmax function along dimension -1. This ensures that the probabilities for each batch sum up to one. Subsequently, the token ID with the highest probability is identified using `torch.argmax` on the probability distribution. This token ID represents the model's prediction for the next token. [24:30]

Finally, the predicted token ID is appended to the input `idx` using `torch.cat`. This updated `idx` becomes the input for the next iteration. This process is repeated within a loop for `max_new_tokens` times, effectively generating a sequence of new tokens. [26:00]

While the softmax function is used to convert logits to probabilities, it is noted that for simply finding the index with the highest value, softmax is technically redundant because the `argmax` of logits and the `argmax` of softmax-transformed logits will be the same. However, softmax is retained for its utility in future steps, such as implementing sampling techniques with temperature variability to introduce creativity and diversity in the generated text. [28:30]

## Testing the Text Generation Function

To test the `generate_text_simple` function, a sample input sentence "hello I am" is encoded into token IDs using the `tiktoken` encoder. The model is then set to evaluation mode using `model.eval()`, which bypasses certain layers like dropout for efficiency during inference. The `generate_text_simple` function is called with the model, the encoded input, `max_new_tokens=6`, and the `context_length` from the GPT configuration. [31:00]

The output consists of the original input tokens plus the six newly generated tokens. The total length of the output sequence is the sum of the input tokens and `max_new_tokens`. The generated token IDs are then decoded back into text using the vocabulary and the encoder. Currently, the generated text appears random because the model's parameters are randomly initialized and have not been trained. [34:00]

The random output highlights the necessity of training the model. Although the GPT-2 architecture with 124 million parameters has been successfully implemented and can predict outputs, these predictions are not meaningful without training. The subsequent lectures will focus on training these parameters to improve the quality and coherence of the generated text, making it comparable to outputs from models like ChatGPT. [37:01]

---

\*End of Chapter 25. Source: <https://www.youtube.com/watch?v=F1Sm7z2R96w>\*

---

## Chapter 26: Measuring LLM Performance with Loss Functions

---

*Source video: Measuring the LLM loss function*

*Duration: 56 minutes | Playlist position 26*

### Introduction to Loss Functions in LLMs

In traditional machine learning, loss functions for regression and classification tasks are well-defined, using metrics like mean squared error for regression and cross-entropy or hinge loss for classification. However, defining a loss function for large language models (LLMs) presents a unique challenge: how do we quantitatively measure if an LLM is performing well? This lecture focuses on addressing this question by introducing and implementing loss functions for LLMs. [\[0:10\]](#)

The series has progressed through stage one of building an LLM, covering data preparation, the attention mechanism, and the LLM architecture. The GPT model developed so far can take input text and predict the next tokens. However, the predictions are currently random, necessitating a training phase to improve performance. [\[0:10\]](#)

Stage two of building an LLM centers on training. To train the model effectively, we need a quantitative metric to capture how "bad" the model's output is. This metric is the loss function, which quantifies the LLM's performance. By minimizing this loss, we can guide the LLM to produce better predictions. [\[0:10\]](#)

The training process involves integrating gradient descent and backpropagation algorithms, concepts previously covered in neural networks. This lecture will cover text generation recap and text evaluation, specifically defining a loss function to quantify the difference between the LLM's generated text and the desired output. Subsequent lectures will cover dataset processing, training and validation loss calculation, backpropagation, and loading pre-trained weights. [\[0:10\]](#)

### Text Generation Recap and Model Configuration

To generate text, we first need to configure the GPT model. This configuration includes parameters such as vocabulary size (50257 for the tiktoken tokenizer used by GPT-2), context length (256 for computational efficiency, though GPT-2 uses 1024), embedding dimension (768), number of attention heads (12), number of Transformer blocks (12), dropout rate (0.1), and a setting to disable query, key, and value bias. [\[0:10\]](#)

The GPT model architecture takes input tokens, converts them into embeddings, adds positional embeddings, passes them through Transformer blocks, applies a final



normalization layer, and outputs logits. These logits have a dimension equal to the vocabulary size for each token. [0:10]

## Inputs and Target Values

The loss function requires comparing the model's predictions (outputs) with the true values (targets). The input to the LLM is a tensor where each row represents a batch. For example, an input might be ["every effort moves"] for one batch and ["I really like"] for another. [0:10]

Crucially, the target tensor is not just the single next word but includes targets for each prediction task within the input sequence. For an input of three tokens, there are three prediction tasks. For "every effort moves," the targets are the token IDs for "effort," "moves," and "you." Similarly, for "I really like," the targets are for "like," "chocolate," and potentially another word depending on the context length. [0:10]

The model's output is a logit tensor. After applying a softmax function, this tensor represents probabilities for each token in the vocabulary being the next token. The goal is to make the probabilities of the target tokens as high as possible. [14:07]

## Understanding Model Outputs and Token IDs

When the input tokens are passed through the GPT model, it generates a logit tensor. This tensor is then converted into probabilities using the softmax function. For an input sequence, the model produces a probability distribution for the next token at each position. The index with the highest probability is selected as the predicted token ID. [14:07]

For example, if the input is "every effort moves," the model outputs probabilities for the next token. The highest probability might correspond to "effort" for the first prediction, "moves" for the second, and "you" for the third. These predicted token IDs are then compared against the target token IDs. [14:07]

The output token IDs generated by the model are not yet accurate because the model has not been trained. The objective is to make these output token IDs as close as possible to the target token IDs. [14:07]

## Implementing the Loss Function

The core of training an LLM involves minimizing a loss function that quantifies the difference between the predicted output and the target values. The process begins with the input tokens, which are fed into the GPT model to generate logits. These logits are then converted into probabilities using the softmax function. [25:01]

The target values are the correct token IDs that the model should predict. The loss function measures how well the predicted probabilities align with these target token IDs. [25:01]

The probability tensor has dimensions of batch size, number of tokens, and vocabulary size. To calculate the loss, we need to extract the probabilities corresponding to the target token IDs for each prediction task. [32:59]

The goal is to make these extracted probabilities as close to 1 as possible. This is achieved by calculating the negative log-likelihood, which is also known as the cross-entropy loss. This loss function is minimized during training. [32:59]

The entire process of calculating the cross-entropy loss can be encapsulated in a single line of PyTorch code: `torch.nn.functional.cross_entropy(logits_flat, targets_flat)`. This function first applies softmax to the logits, then finds the probabilities corresponding to the target indices, takes the negative logarithm of these probabilities, and finally computes the mean. [49:30]

## Understanding Perplexity

Perplexity is another metric used to evaluate LLMs, offering a more interpretable measure of model uncertainty than cross-entropy loss. It is calculated as the exponent of the cross-entropy loss. A lower perplexity score indicates better predictions. [49:30]

For instance, a perplexity score of 48725 suggests that the model is as uncertain as if it were randomly choosing the next token from approximately 48725 options in the vocabulary. This is considered poor performance, indicating significant uncertainty in predicting the next token. In contrast, a perplexity of 2 would be excellent, implying the model is highly confident in its predictions. [49:30]

The lecture concluded by summarizing the process: starting with inputs and targets, generating model outputs, and calculating the loss using cross-entropy. The concept of perplexity was also introduced as a more intuitive measure of model performance. [49:30]

\*End of Chapter 26. Source: <https://www.youtube.com/watch?v=7TKCrt--bWI>\*

---

## Chapter 27: Evaluating LLM Performance on a Real Dataset

---

*Source video: Evaluating LLM performance on real dataset | Hands on project | Book data*

*Duration: 58 minutes | Playlist position 27*

### Stage Two Recap and Hands-On Approach

We are currently in stage two of building a large language model (LLM), which focuses on training or pre-training the foundational model. In previous lectures, we covered data preparation, the attention mechanism, and LLM architecture. We also explored how to calculate the cross-entropy loss for an LLM. This lecture will be a hands-on session where we will collect data from a storybook, make predictions using our previously built LLM, and measure the loss against the actual data. [\[0:07\]](#)

In the prior lecture, we used simple examples with only two inputs: "every effort moves" and "I really like." Today, we will use an actual dataset to calculate both training and validation losses. We will also split the dataset into training and validation sets. This hands-on approach will equip you to use any storybook or dataset to perform a forward pass with your LLM and calculate the loss against true values. While we won't be training the parameters in this lecture, the subsequent lecture will cover the LLM training function and backpropagation. [\[0:07\]](#)

### Dataset Selection and Preparation

The dataset we will use is "The Verdict," a book by Edith Wharton published in 1906. This book is publicly available and relatively short, making it suitable for a demonstration that runs quickly. The dataset contains approximately 20,000 characters. [\[1:30\]](#)

We will first convert this text data into tokens using the Byte Pair Encoding (BPE) tokenization scheme. BPE is a subword-based tokenization method where a word is not necessarily a single token; even characters or pairs of letters can be tokens. Using BPE, the 20,000 characters in "The Verdict" translate to about 5,000 tokens. We will use the `tiktoken` library, which is the same encoder used by OpenAI. [\[1:58\]](#)

### Data Splitting and Input-Output Pair Creation

For any machine learning problem, it's crucial to split data into training and validation sets. The training loss alone is not sufficient; we need to evaluate how the LLM performs on unseen data. We will use a 90/10 split, where the first 90% of the data will be for training and the remaining 10% for validation. [\[2:27\]](#)

Unlike image classification where input-output pairs are straightforward (e.g., image of a cat -> label "cat"), creating input-output pairs for LLMs requires a different approach due to their autoregressive nature. We construct these pairs directly from the text. The process involves defining a context size, which is the maximum number of tokens the LLM can see before predicting the next token. For demonstration, we'll use a context size of four. [2:57]

Let's consider the input "I had always thought" with a context size of four. The first input sequence is "I had always thought." The corresponding target output is the input sequence shifted by one token: "had always thought Jack." This single input-output pair actually represents four prediction tasks: "I" predicts "had," "had" predicts "always," "always" predicts "thought," and "thought" predicts "Jack." [3:25]

When constructing subsequent input sequences, we need to consider the stride. A stride of one would mean the next input ("had always thought Jack") has significant overlap with the previous one. However, models like GPT typically use a stride equal to the context size. With a stride of four and a context size of four, the next input sequence ("Jack gisburn rather a") starts immediately after the first sequence ends, ensuring no overlap and no skipped tokens. [4:00]

The input tensor is created by striding through the dataset, with each row representing an input sequence. The corresponding target tensor is created by shifting the input sequences by one token. For example, if the input is "Jack gisburn rather a," the target output would be "gisburn rather a cheap." [4:35]

## Understanding the Loss Calculation

The loss is calculated between the LLM's output predictions and the target sequences. The input sequences are passed through the LLM architecture, producing a logits tensor. This logits tensor, with dimensions equal to the vocabulary size for each token in the sequence, is then converted into a probability tensor using the softmax function. [5:15]

The context vectors generated within the Transformer block are richer than input embeddings because they incorporate information about how tokens relate to each other through the multi-head attention mechanism. This attention mechanism is the core of the LLM's ability to capture meaning and context. [5:50]

After passing through the GPT architecture, we obtain a logits tensor. For each token in the input sequence, there is a corresponding logits vector whose dimension equals the vocabulary size. These logits are not normalized. To convert them into probabilities, we apply the softmax function, ensuring that the probabilities for each token sum to one. [6:15]

The prediction for each token is the one with the highest probability in the softmax output. However, to calculate the loss, we compare these probabilities with the target tokens. We find

the index of each target token in the vocabulary and then retrieve the probability associated with that index from the LLM's output probability tensor. [6:45]

The cross-entropy loss is then calculated using these probabilities. Specifically, we take the logarithm of these probabilities, sum them up, take the mean, and then negate the result. This is also known as the negative log-likelihood. The goal of training is to minimize this loss, bringing it as close to zero as possible. [7:15]

## Batch Processing and Loss Aggregation

When processing data in batches, the input and target sequences are stacked together. For a batch size of two, we have two input sequences (X1, X2) and their corresponding target sequences (Y1, Y2). The LLM processes these inputs to produce logits for each sequence. [8:00]

The logits tensors from each input in the batch are then flattened and concatenated. After applying the softmax function and indexing with the target token IDs, we obtain the probabilities for the target tokens. The cross-entropy loss is calculated for each input-output pair within the batch, and these losses are aggregated to get the total loss for the batch. [8:30]

The `DataLoader` in PyTorch is used to efficiently process data in batches. We specify parameters like `batch_size`, `max_length` (context size), and `stride`. `shuffle=True` randomizes the data order for each epoch, while `drop_last=True` discards the last batch if it's smaller than the specified batch size. [9:30]

For our "The Verdict" dataset, we will use a context length of 256 tokens. The training data loader will have nine batches, and the validation data loader will have one batch, each with a batch size of two. We perform sanity checks to ensure the number of training and validation tokens is sufficient for the chosen context length. [10:30]

## Implementing the Loss Calculation in Code

The `GPT` model class, built from scratch in previous lectures, takes input tokens and returns a logits tensor. This tensor is then processed to calculate the loss. We use `torch.nn.functional.cross_entropy`, a powerful PyTorch function that efficiently computes the softmax, indexes the probabilities with the target token IDs, and calculates the negative log-likelihood in a single step. [12:00]

The `calculate_loss_batch` function takes an input batch and a target batch, passes the input through the model to get logits, flattens the logits and targets, and then uses `F.cross_entropy` to compute the loss for that batch. The `calculate_loss_loader` function iterates through all batches in a data loader (either training or validation), calculates the loss

for each batch using `calculate_loss_batch`, aggregates these losses, and returns the average loss per batch. [13:00]

We then call `calculate_loss_loader` for both the training and validation data loaders, passing the model and the device (CPU or GPU) as arguments. This process yields the training loss and validation loss for our LLM on the "The Verdict" dataset. [14:30]

## Running the Code and Interpreting Results

The code is executed on a CPU, but it can be adapted to run on Apple Silicon chips or GPUs for faster processing. The entire process, from loading the dataset to calculating the training and validation losses, takes less than 30 seconds on a CPU for this small dataset. [15:00]

You can modify the context size (e.g., to 1024) or use different datasets, such as downloading and processing the Harry Potter books. The code is generalizable and can be applied to any text dataset. The obtained training and validation losses are crucial for the next step: backpropagation and model training. [15:30]

This lecture demonstrated how to take a dataset, prepare it into input-output pairs, run it through a custom-built LLM, and calculate the loss. The LLM architecture used was developed from scratch in previous lectures, without relying on external libraries like LangChain. The next lecture will focus on LLM pre-training, implementing backpropagation to minimize the calculated losses and improve the model's output coherence. [16:00]

---

\*End of Chapter 27. Source: [https://www.youtube.com/watch?v=zuj\\_NJNouAA](https://www.youtube.com/watch?v=zuj_NJNouAA)\*

---

## Chapter 28: Training the Large Language Model

---

*Source video: Coding the entire LLM Pre-training Loop*

*Duration: 43 minutes | Playlist position 28*

### Recapping Loss Calculation and Introducing Backpropagation

In the previous lecture, we extensively covered how to calculate training and validation losses for a large language model (LLM). This process involves feeding an input to the model, obtaining its output, comparing this output with the target value, and subsequently calculating the loss function. With the loss function now established, we are prepared to implement backpropagation to minimize this loss. [0:05]

The dataset used for this training process is "The Verdict," a book published in 1960. This dataset is open-source and contains approximately 5,000 to 5,500 tokens, translating to around 20,000 characters. The dataset is first divided into training and validation sets, with 90% allocated for training and the remaining 10% for validation. [1:19]

Within the training data, input and target pairs are created. Since LLMs are autoregressive models, they do not have explicit labels. Instead, the training and validation data are processed to generate these input-target pairs using a data loader. For instance, a batch might contain two samples, each with four tokens (though in practice, the context size is much larger, around 256 tokens). The target is essentially the input sequence shifted one position to the right. [1:59]

This input-target pair construction is illustrated with a context size of four tokens. The first input consists of the first four tokens, the second input consists of the next four tokens, and so on. The stride, typically equal to the context size, determines the overlap between consecutive input sequences. The target sequence is generated by shifting the input sequence to the right. [2:40]

### LLM Output and Logit Tensor

The LLM architecture processes these input batches sequentially. The process involves tokenization, embedding, positional embedding, a dropout layer, and then the core Transformer block. This block contains multiple layers, including normalization, multi-head attention, dropout, shortcut connections, and a feed-forward network. Following the Transformer block, another normalization layer is applied, leading to an output head that produces the logit tensor. [3:20]

The shape of the logit tensor is significant. For a batch with two samples and four tokens per sample, and a vocabulary size of 50,257, the logit tensor would have dimensions of

2x4x50,257. This tensor is then flattened, merging the samples into a unified tensor of shape 8x50,257. [4:00]

The entries in the logit tensor do not represent probabilities directly. To convert these logits into probabilities, a softmax function is applied. Each value in the resulting probability vector corresponds to the likelihood of a specific token being the next token in the sequence. For example, if the input is "I", the token with the highest probability should ideally be "had". [4:30]

## Loss Function and Optimization Goal

The goal of training is to ensure that the predicted probabilities for the target tokens are as close to one as possible. Initially, when the LLM is untrained, these probabilities will be far from one. The cross-entropy loss, specifically the negative log-likelihood, is employed to quantify the difference between the predicted probabilities and the target token IDs. The primary objective during LLM training is to minimize this loss function, driving it as close to zero as possible. [5:20]

The pre-training process aims to optimize all parameters within the GPT architecture. This optimization is achieved through backpropagation, which calculates the gradients of the loss function with respect to these parameters. By minimizing the loss, the model's outputs are brought closer to the target values, indicating that the LLM is learning to predict the correct next tokens. [5:50]

## The Pre-training Loop Schematic

The pre-training loop involves iterating through the training data multiple times, known as epochs. An epoch consists of processing the entire training set once. Within each epoch, the data is divided into batches, and for each batch, the loss function is calculated. Subsequently, a backward pass is performed to compute the gradients of the loss. [6:30]

These gradients are crucial for updating the model's weights and parameters. The update rule for a parameter  $P$  is generally expressed as  $P_{\text{new}} = P_{\text{old}} - \text{step\_size} * \text{loss\_gradient}$ . After updating the parameters, training and validation losses are printed for monitoring, and a sample of the LLM's output is generated for visual inspection. The gradients are then reset to zero before processing the next batch. [7:00]

This entire process is repeated for multiple epochs to allow the model to learn and refine its parameters. The foundation of this training loop lies in the ability to calculate the loss function, which in turn depends on understanding the LLM's architecture, including tokenization, embeddings, attention mechanisms, and other components. [7:40]

## Backpropagation with `loss.backward()`



The core of the pre-training code involves calculating loss gradients using the `loss.backward()` method. This single line of code efficiently computes the backward pass, determining the gradient of the loss with respect to all model parameters. This is possible because the entire operation, from input to loss calculation, is differentiable. [8:10]

The differentiability is maintained throughout the GPT model's workflow, enabling backpropagation to calculate the partial derivative of the loss with respect to every parameter within each layer. The total number of parameters in the model can be substantial, reaching approximately 161 million. [8:40]

## Parameter Count Breakdown

The large number of parameters stems from various components of the GPT model. The embedding parameters, including token and positional embeddings, contribute significantly. For instance, token embeddings have a size equal to the vocabulary size multiplied by the embedding dimension, and positional embeddings are defined by the context size multiplied by the embedding dimension. [9:20]

In the multi-head attention mechanism, there are trainable weights for query, key, and value matrices. Each matrix has dimensions corresponding to the embedding dimension. Additionally, the feed-forward network within the Transformer block, with its expansion and contraction layers, adds a considerable number of parameters. The final output head, which projects the features to the vocabulary size, also contributes to the total parameter count. [10:00]

Summing these components—embeddings, Transformer blocks (each containing multi-head attention and feed-forward networks), and the final output layer—results in a total parameter count in the order of 160 million. For comparison, the smallest GPT-2 model has around 124 million parameters, with discrepancies often arising from techniques like weight tying in the output projection layer. [10:40]

## Parameter Updates and Gradient Descent

During backpropagation, the gradient for each parameter is computed. These gradients are then used to update the model's weights. While the example uses a simplified gradient descent update rule ( $P_{\text{new}} = P_{\text{old}} - \text{step\_size} * \text{gradient}$ ), the actual implementation often employs more sophisticated optimizers like AdamW, which incorporates weight decay. The core principle remains the same: adjusting parameters based on the calculated gradients to minimize the loss function. [11:20]

The goal is for the loss function value to decrease over iterations, eventually reaching a minimum. This process ensures that the model learns to generate outputs that closely match the target values. [11:40]

## Implementing the Pre-training Loop in Code

The pre-training loop begins by iterating through the training data loader. For each batch, the input and target pairs are extracted. The loss is then computed using the cross-entropy function. The critical step is `loss.backward()`, which calculates the gradients for all parameters. Following this, `optimizer.step()` updates the model's parameters based on these gradients. [\[12:20\]](#)

A counter tracks the total number of tokens seen by the model. This loop continues for a specified number of epochs, with each epoch processing the entire training dataset. An evaluation step is included to monitor the training and validation losses periodically. [\[12:50\]](#)

The `evaluate_model` function calculates the loss over both the training and validation sets. It ensures the model is in evaluation mode, disabling gradient tracking and dropout to obtain consistent loss measurements. This evaluation is performed at a set frequency (e.g., every five batches) to provide insights into the model's progress. [\[13:30\]](#)

Additionally, a `generate_and_print_sample` function is used to visualize the model's text generation capabilities after each epoch. This function takes an initial text snippet, converts it to token IDs, and feeds it to the LLM to generate a sequence of new tokens. This allows for tracking improvements in the model's language generation skills over time. [\[14:00\]](#)

## Model Configuration and Training Execution

The GPT model instance is created using a specific configuration. This configuration includes the vocabulary size (50,257), context length (256 tokens, adjustable to 1024), embedding dimension (768), number of attention heads (12), number of Transformer blocks (12), and dropout rate (0.1). Bias terms for query, key, and value matrices are set to `False`. [\[15:00\]](#)

The training utilizes the AdamW optimizer, a variant of Adam that includes weight decay. AdamW is a popular choice for modern machine learning tasks due to its effectiveness in avoiding local minima and promoting faster convergence. Key hyperparameters like the learning rate and weight decay can be tuned. [\[15:40\]](#)

The training is set to run for 10 epochs. The evaluation frequency is set to five batches, meaning training and validation losses are reported after every five batches. An initial text snippet, "every effort moves you," is provided to the `generate_and_print_sample` function. The training process is timed to measure its duration. [\[16:10\]](#)

## Training Results and Observations

The training process, involving the optimization of over 160 million parameters for 10 epochs, completed in approximately 6.6 minutes on a MacBook Air. This demonstrates the efficiency of modern deep learning frameworks. [\[17:00\]](#)

Analysis of the training and validation losses reveals a significant decrease in training loss, from 9.781 to 3.91. However, the validation loss stagnated around 6.4, indicating a classic sign of overfitting. The model is performing exceptionally well on the training data but is not generalizing as effectively to unseen validation data. [17:40]

The generated text samples illustrate the model's learning progression. Initially, the model outputs repetitive characters like commas. As training progresses, it begins to generate more coherent phrases and sentences, eventually producing text that incorporates words and structures from the training data. [18:20]

The observation that the model directly recycles text from the dataset, such as "quite insensible to the irony," is a clear indicator of overfitting. This memorization is expected given the small size of the training dataset (5,000 tokens). In real-world scenarios, much larger datasets are used, and models are often trained for fewer epochs to mitigate overfitting. [19:00]

A plot of the training and validation losses shows the training loss continuously decreasing, while the validation loss decreases initially and then plateaus or even slightly increases, confirming the divergence and overfitting. The number of tokens seen increases linearly with epochs, reflecting the processing of the entire dataset multiple times. [19:40]

The instructor encourages experimenting with hyperparameters such as learning rate, weight decay, and the number of epochs to observe their impact on overfitting and model performance. Despite the overfitting, the core objective of minimizing the loss function and enabling the LLM to learn has been achieved. [20:20]

## Next Steps: Decoding Strategies

In the next lecture, the focus will shift to decoding strategies, such as temperature scaling, to control randomness in the model's predictions. These strategies aim to prevent the model from merely memorizing the training data and encourage it to generate more novel and coherent text. [20:50]

This lecture marks a significant milestone, as a functional LLM has been trained from scratch, demonstrating the minimization of loss and the learning process. While overfitting was observed, it provides a valuable learning opportunity to address and improve model performance in subsequent steps. [21:20]

\*End of Chapter 28. Source: <https://www.youtube.com/watch?v=Zxf-34voZss>\*

---

## Chapter 29: Temperature Scaling in Large Language Models

---

*Source video: Temperature Scaling in Large Language Models (LLMs)*

*Duration: 26 minutes | Playlist position 29*

### Recap of Previous Training and Introduction to Temperature Scaling

In the previous lecture, a large language model was trained from scratch for 10 epochs. The process involved feeding input tokens or sentences to the model and recording the output tokens. However, the generated output tokens, such as "every effort moves was one of the XM he laid down across dash dash dash," did not make much sense, indicating a high degree of randomness. This lecture aims to address this issue by reducing the randomness in the output tokens to make them more coherent. One technique to achieve this is called temperature scaling. [\[0:05\]](#)

Until now, the process for predicting the next token involved identifying the token with the highest probability score from the vocabulary. This deterministic approach, often referred to as greedy decoding, involves feeding an input to the GPT architecture, obtaining a logic tensor, passing it through a softmax function to get probabilities, and then selecting the token ID with the maximum probability. For instance, if "every" is the input, the token ID with the highest probability is chosen as the next token. Similarly, with "every effort moves" as input, the token ID corresponding to the highest probability is selected, leading to the next token. [\[3:47\]](#)

### Probabilistic Sampling and the Multinomial Distribution

The deterministic selection of the token with the highest probability can lead to a lack of randomness and diversity in the generated text. To introduce more variety, the approach can be shifted from selecting the maximum probability to sampling the next token probabilistically from a probability distribution. This is where temperature scaling and top-k sampling come into play, with temperature scaling being the focus of this lecture. [\[5:10\]](#)

Instead of solely relying on the maximum probability, temperature scaling involves replacing the argmax operation with a method that samples from a probability distribution. Specifically, the multinomial probability distribution is used. This means the next token is sampled according to its probability score, rather than blindly choosing the token with the highest probability. Sampling from a distribution implies that while the most probable outcomes are more likely, other outcomes still have a chance of being selected. [\[6:00\]](#)

The multinomial distribution is suitable for scenarios with multiple mutually exclusive outcomes, each having a corresponding probability. In the context of language models, these

outcomes are the tokens in the vocabulary. If there are  $K$  possible tokens, each with a probability, and we conduct  $N$  independent trials to predict the next token, the resulting distribution is multinomial. [7:00]

## Implementing Probabilistic Sampling with a Small Vocabulary

To illustrate the concept, consider a small vocabulary consisting of nine tokens: "closer," "every," "effort," "forward," "inches," "moves," "Piza," "towards," and "you." Each token has a unique token ID. When an input like "every effort moves you" is passed through the model, it generates a logic tensor with dimensions corresponding to the vocabulary size. This logic tensor is then converted into probabilities using the softmax function. [8:30]

Previously, the token ID with the maximum probability was selected. For example, if the probability for "forward" was the highest (0.5721), then "forward" would be the predicted next token. To implement probabilistic sampling, the `argmax` function is replaced with PyTorch's `multinomial` function, which samples from the probability distribution. While in a single trial, the most likely token might still be chosen, repeating this sampling process multiple times reveals the probabilistic nature. [9:45]

When the multinomial function is applied over a thousand trials, it samples tokens proportionally to their probabilities. This means that while "forward" might still be sampled most frequently (e.g., 582 times out of 1000), other tokens like "towards" (343 times), "closer" (73 times), and even "inches" (2 times) can also be selected. This probabilistic sampling allows for more creative and varied outputs, such as "every effort moves you towards" or "every effort moves you inches," instead of always predicting "every effort moves you forward." [11:50]

## Understanding Temperature Scaling

The term "temperature" in temperature scaling refers to a parameter used to adjust the probability distribution derived from the logits before sampling. Specifically, the logits are divided by a temperature value (a number greater than zero). This scaled logits tensor is then passed through the softmax function to obtain scaled probabilities. The rest of the process, including sampling using the multinomial distribution, remains the same. [13:30]

The effect of the temperature value on the probability distribution can be observed by comparing different temperature settings.

- Temperature = 1: This represents the baseline where the logits are not scaled. The probabilities are as calculated directly from the softmax function. For example, "forward" might have a probability of around 0.5, "towards" around 0.35, and "closer" around 0.06. ``

- Temperature  $< 1$  (e.g., 0.1): When the logits are divided by a small temperature value, the resulting probability distribution becomes sharper. The probability of the most likely token increases significantly, while the probabilities of other tokens approach zero. If the temperature is very low (e.g., 0.1), the scaled probability for "forward" becomes very high (close to 1), and other probabilities become negligible. This sharp distribution, when sampled by the multinomial function, will almost always select the most likely token, mimicking the behavior of the argmax function. ``
- Temperature  $> 1$  (e.g., 5): Dividing the logits by a higher temperature value results in a flatter probability distribution. This means that the probabilities for all tokens become more uniform, and less likely tokens have a higher chance of being selected. For instance, with a temperature of 5, the probability for "forward" might still be the highest, but tokens like "towards" and "closer" will have probabilities much closer to "forward." This increased uniformity leads to greater variability and creativity in the generated text, but it also increases the risk of generating nonsensical output, such as predicting "Piza" as the next token with a non-negligible probability (e.g., 4%). ``

## Conceptualizing Temperature and Entropy

The name "temperature" is conceptually linked to entropy. As the temperature increases, the probability distribution flattens, resembling a state of higher entropy where all tokens have a more equal probability of being chosen. This increased randomness can be interpreted as higher creativity or instability. Conversely, a low temperature leads to a sharper distribution with low entropy, concentrating the probability on a single token, thus reducing creativity and leading to more deterministic outputs. [19:00]

Very high temperatures can lead to almost equal probabilities for all tokens, while very low temperatures result in sharp distributions where only one token is likely. An ideal temperature value strikes a balance, providing some creativity without generating excessive nonsense. [20:00]

## Workflow and Conclusion

The sequential workflow for applying temperature scaling involves:

1. Taking the logits.
2. Scaling the logits by dividing them by the temperature value.
3. Applying the softmax function to obtain the scaled probabilities.
4. Sampling the next token using the multinomial distribution based on these probabilities. [21:30]

In summary, temperature scaling controls the randomness in token prediction. A low temperature leads to low entropy and less creativity, making the model's output predictable

and potentially repetitive. A high temperature increases entropy, leading to a flatter distribution, more variety, and increased creativity, but also a higher chance of nonsensical outputs. Therefore, careful selection of the temperature value is crucial. The next lecture will cover top-k sampling, another technique used in conjunction with temperature scaling to control randomization during decoding. [22:30]

---

\*End of Chapter 29. Source: <https://www.youtube.com/watch?v=oG1FPVnY0pI>\*

---

## Chapter 30: Top-K Sampling in Large Language Models

---

*Source video: Top-k sampling in Large Language Models*

*Duration: 23 minutes | Playlist position 30*

### Introduction to Decoding Strategies

In this lecture, we continue our discussion on strategies for Large Language Model (LLM) decoding to reduce randomness. Following the previous lecture on temperature scaling, we will now explore another decoding strategy known as top-K sampling. Top-K sampling is often used in conjunction with temperature scaling. [\[0:05\]](#)

The necessity for LLM decoding strategies arises from the limitations of a naive approach. If the next token is always selected based on the largest probability score among all tokens in the vocabulary, the generated text can become very random and nonsensical, especially over longer sequences. This deterministic method overemphasizes the token with the maximum probability, hindering creativity and coherence. [\[0:05\]](#)

To address this, we can sample the next token from a probability distribution rather than choosing it deterministically. After obtaining the logit tensor from the LLM architecture, we apply the softmax function to convert it into a tensor of probabilities. Instead of selecting the index corresponding to the maximum probability, we sample from this distribution using a multinomial distribution. This probabilistic sampling allows other tokens, proportional to their probability scores, to be selected, introducing more creativity into the LLM's output and offering a way to control randomness. [\[0:05\]](#)

Temperature scaling further refines this process. By dividing the logits by a temperature value before applying softmax, we can control the shape of the probability distribution. A higher temperature leads to a more uniform distribution, increasing the probability of sampling less likely tokens and resulting in more diverse outputs. Conversely, a lower temperature sharpens the distribution, favoring tokens with higher probabilities. [\[0:05\]](#)

However, even with temperature scaling, there's a potential issue: all tokens, including those with very low probabilities, remain in consideration. This can lead to nonsensical outputs, such as generating "pizza" after "every effort moves you," which has no semantic connection. We want to restrict the opportunity of being the next token to only a few sensible options. [\[0:05\]](#)

### Introducing Top-K Sampling

Top-K sampling addresses the issue of random tokens being considered by restricting the sampling pool to only the top K most likely tokens. All other tokens are excluded from



consideration. To implement this, we identify the K logits with the highest values. Then, all other logits are replaced with negative infinity. When the softmax function is applied, these negative infinity values effectively become zero, ensuring that only the top K tokens have a chance to be selected as the next token. [0:05]

This process ensures that tokens with very low probabilities, which are unlikely to make sense in the context, are not considered. For example, if "pizza" is not among the top K most likely tokens, its probability will be reduced to zero after applying the top-K masking and softmax. [0:05]

In the previous section, we implemented a probabilistic sampling approach with temperature scaling. While this allows for exploring less likely but potentially interesting paths, it can sometimes result in grammatically incorrect or nonsensical outputs. Top-K sampling is introduced to mitigate this by focusing the sampling on a restricted set of the most probable tokens. [0:05]

## Implementing Top-K Sampling in Code

To demonstrate top-K sampling, let's consider a next token logit tensor. The first step is to select the value for K, determining how many top tokens to consider. For instance, if K is set to 3, we will only focus on the three logits with the highest values. PyTorch provides the `torch.topk` function, which returns both the top K values and their corresponding indices from a given tensor. [0:05]

After identifying the top K logits and their positions, we replace all other logits with negative infinity. This is achieved using a conditional operation, for example, replacing any logit less than the K-th highest logit with negative infinity. When softmax is applied to this modified logit tensor, only the top K tokens will have non-zero probability scores, and these probabilities will sum up to one. [0:05]

The workflow then integrates temperature scaling and multinomial sampling. The logits are first masked by the top-K selection, then divided by the temperature value, followed by the softmax function. Finally, the next token is sampled from the resulting multinomial probability distribution. This ensures that while creativity is maintained through probabilistic sampling, the selection is restricted to a sensible set of tokens. [0:05]

The complete workflow for this decoding strategy involves obtaining the logit tensor from the LLM, applying top-K sampling to retain only the K most likely logits (replacing others with negative infinity), scaling the remaining logits by the temperature, applying softmax, and finally sampling from the multinomial distribution to predict the next token ID. This process ensures that randomness is controlled, and nonsensical tokens are avoided. [12:57]

A function can be written to combine temperature scaling and top-K sampling for next token prediction. This function takes the GPT model, maximum new tokens to generate, context size, temperature value, top-K value, and an optional end-of-sequence (EOS) token ID. If the temperature is greater than zero, the logits are scaled by the temperature, softmax is applied, and then the next token ID is sampled using the multinomial distribution. If temperature is not specified, the function defaults to using argmax to select the token with the maximum probability, similar to the naive decoding strategy. The generated token ID is appended to the sequence until the maximum number of new tokens is reached or the EOS token is encountered. [12:57]

## Comparing Decoding Strategies

To compare the effectiveness of these strategies, we can generate a sequence of tokens using both the naive greedy decoding and the new strategy incorporating top-K sampling and temperature scaling. When using the naive greedy approach, the generated tokens might be repetitive or nonsensical, potentially overfitting to the training data. For example, generating 25 tokens might result in a sequence that lacks coherence. [12:57]

Using the new generate function with parameters like `top_k=25` and `temperature=1.4`, we can observe a different output. A temperature of 1.4 is chosen as a good trade-off, avoiding the overly sharp distributions of low temperatures and the nonsensical outputs of very high temperatures. The generated sequence, while still potentially not perfectly coherent, shows a significant improvement over the naive approach. Crucially, the overfitting problem is addressed. [12:57]

The probabilistic nature of sampling strategies like top-K sampling and temperature scaling helps prevent the LLM from simply memorizing passages from the training data. This is essential for generative AI models, as demonstrated by interactions with models like ChatGPT, which produce novel outputs rather than recalling exact text. The generated words are often new and not present in the original context, indicating true generative capabilities. [12:57]

## Conclusion and Workflow Summary

In summary, we have explored two key decoding strategies: temperature scaling and top-K sampling. Temperature scaling allows for intuitive control over the sharpness of probability distributions, influencing output diversity and helping to prevent overfitting through multinomial sampling. Top-K sampling addresses a limitation of temperature scaling by restricting the sampling pool to the top K most likely tokens, thereby preventing nonsensical or random outputs and improving the meaning of generated tokens. [20:28]

The complete workflow for generating text involves obtaining the logit tensor from the GPT model, applying top-K sampling (which replaces non-top-K logits with negative infinity), performing temperature scaling by dividing the logits by the temperature value, applying the softmax function, and finally sampling the next token from the multinomial distribution. This integrated approach minimizes overfitting and enhances the quality of generated text.

[20:28]

\*End of Chapter 30. Source: <https://www.youtube.com/watch?v=EhU32O7DkA4>\*

---

## Chapter 31: Saving and Loading LLM Model Weights using PyTorch

---

*Source video: Saving and loading LLM model weights using PyTorch*

*Duration: 12 minutes | Playlist position 31*

### Introduction to Saving and Loading Model Weights

This lecture focuses on the crucial process of loading and saving PyTorch model weights, a technique particularly important for large models like the ones built in this series. This capability helps conserve memory and time, especially when dealing with extensive models.

[\[0:07\]](#)

Previously, the series covered evaluating the loss function for a large language model (LLM), defining the cross-entropy loss, and running the pre-training loop for an LLM over multiple epochs. During this process, an issue of overfitting was observed, where the model tended to memorize the training data. To address this, text generation strategies like temperature scaling and top-K sampling were explored. While these methods helped mitigate overfitting, the generated text still lacked significant meaning due to the limited size of the training dataset.

[\[0:07\]](#)

The upcoming lecture will involve loading pre-trained weights from OpenAI into the model, rather than training a new model from scratch. OpenAI has made its GPT-2 pre-trained weights open-source, which will be integrated into the GPT architecture developed in this series. Today's lecture serves as a foundational step, equipping the viewer with the necessary knowledge for handling pre-trained weights. [\[0:07\]](#)

### Core PyTorch Functions for Saving and Loading

Two primary PyTorch functions are essential for this process: `torch.save` and `model.load_state_dict`. When a GPT model is constructed, it contains a vast number of parameters, potentially exceeding 100 million. To avoid re-initializing and re-training these parameters each time, they can be saved using `torch.save`. This function takes two arguments: the model's state dictionary and the desired file name for storage. [\[0:07\]](#)

The `model.state_dict()` method returns a Python dictionary that maps each layer of the model to its learnable parameters. These parameters are typically stored in `torch.nn.Module.parameters()`. The `torch.save` function then serializes this dictionary into a specified file, conventionally given a `.pth` extension. [\[0:07\]](#)

Conversely, `model.load_state_dict()` is used to load these saved parameters. When a collaborator or the user themselves needs to resume training or use a pre-trained model, they

can create an instance of the model and then use `torch.load` along with `model.load_state_dict()` to populate the model with the saved parameters from the file. This process significantly reduces the time and computational resources required. [0:07]

### **Practical Example: Saving and Loading Model Weights**

An instance of the GPT model class, configured with approximately 124 million parameters, is created. The `torch.save` function is then used to save the model's state dictionary to a file named `model.pth`. The `.pth` extension is a convention for PyTorch files and can be any extension. [3:47]

A collaborator receiving this `model.pth` file can instantiate the GPT model class and load the parameters using `torch.load` and `model.load_state_dict()`. After loading, the model can be set to evaluation mode using `model.eval()`. This ability to load saved parameters is invaluable for resuming training sessions after interruptions or for sharing models with others, saving considerable time and memory. [3:47]

### **Saving and Loading Optimizer States**

Beyond model parameters, it is also crucial to save the state of the optimizer, especially for adaptive optimizers like Adam. These optimizers maintain historical data, such as the history of gradients and squared gradients, which are essential for their operation. If only the model parameters are saved, the optimizer loses this historical context, potentially hindering convergence. [6:00]

The `optimizer.state_dict()` method captures this historical data along with optimizer hyperparameters like the learning rate and weight decay. By saving both the model's state dictionary and the optimizer's state dictionary using `torch.save`, one can preserve the complete state of the training process. This allows for seamless resumption of training from the exact point where it was left off. [6:00]

The combined state of the model and optimizer can be saved into a single file, for instance, named `model_and_optimizer.pth`. This file contains all necessary information to restore both the model's weights and the optimizer's internal state, including gradient histories. [6:00]

### **Resuming Training with Saved States**

To resume training, the saved file (`model_and_optimizer.pth`) is loaded using `torch.load` into a checkpoint object. An instance of the GPT model is created, and its parameters are loaded from the checkpoint using `model.load_state_dict()`. Subsequently, the optimizer is instantiated, and its state is loaded from the checkpoint using `optimizer.load_state_dict()`. [8:30]

After loading both the model and optimizer states, the model is typically set to training mode using `model.train()`. This prepares the model for the next steps of the training process, which involve forward passes, backward passes, and gradient descent. Saving and loading both model and optimizer states is a vital practice for efficient training of large models, preventing the loss of progress and avoiding the frustration of starting from scratch. [8:30]

The lecture concludes by emphasizing the simplicity of this process once understood and provides links to relevant PyTorch documentation. This foundational knowledge is essential for the subsequent lecture on loading pre-trained weights from OpenAI for the GPT-2 model. [8:30]

---

\*End of Chapter 31. Source: <https://www.youtube.com/watch?v=Bc-9sf0VihQ>\*

---

## Chapter 32: Loading Pre-trained GPT-2 Weights

---

*Source video: Loading pre-trained weights from OpenAI GPT-2*

*Duration: 50 minutes | Playlist position 32*

### Introduction to Loading Pre-trained Weights

This lecture focuses on integrating pre-trained OpenAI GPT-2 weights into the custom GPT model architecture previously constructed. The goal is to enable coherent text generation by leveraging these publicly released weights. This process serves as a culmination of prior work, utilizing the defined GPT model class and architecture schematic. [0:05]

Previously, the series covered defining loss functions like cross-entropy, implementing the LLM pre-training loop, and generating text. The output from this initial training was noted as incoherent due to training on a small dataset for a limited number of epochs. Subsequent lectures introduced decoding strategies such as temperature scaling and top-k sampling to mitigate overfitting, which offered some improvement but did not fully resolve the coherence issue. [0:05]

The current lecture aims to address this by loading pre-trained weights from OpenAI. The expectation is that these weights will significantly improve the coherence of generated text, making the next tokens and sentences more meaningful. The process will involve understanding how to download these weights and integrate them with the existing codebase. [0:05]

OpenAI's release of GPT-2 weights is acknowledged as a crucial contribution, enabling this lecture. The training of these models likely incurred substantial costs, estimated in the millions of dollars, due to the vast datasets used. The availability of these weights on platforms like Kaggle makes this integration possible. [1:30]

### Setting Up the Environment

To begin, it's important to note that OpenAI originally saved GPT-2 weights using TensorFlow, while the current project utilizes PyTorch. Therefore, TensorFlow and an additional library called `tqdm` for tracking download progress need to be installed. The recommended versions are TensorFlow greater than or equal to 2.15 and `tqdm` greater than or equal to 4.66. [2:18]

The installation commands are provided, followed by verification of the installed versions. After installing these libraries, users are advised to pause the video and proceed with the subsequent steps. [2:18]

```
# Example installation commands (actual commands may vary based on environment)
```

```
# pip install tensorflow>=2.15 tqdm>=4.66
```

## Understanding the GPT-2 Weight Files

A critical step is downloading the GPT-2 weights and parameters. These are typically found on platforms like Kaggle and consist of seven files totaling approximately 500 megabytes. However, simply downloading these files is insufficient; several pre-processing steps are required before they can be integrated into the custom GPT architecture. [2:58]

The provided code includes a `gpt_download_3.py` file containing three functions: `download_and_load_gpt2`, `download_file`, and `load_gpt2_params_from_tf_checkpoint`. The main function, `download_and_load_gpt2`, handles both downloading the seven files to the local machine and then extracting parameters from these files using the helper functions. [3:25]

The `load_gpt2_params_from_tf_checkpoint` function is responsible for organizing these parameters into a dictionary named `params`, which has a specific, predefined format. Understanding this format is crucial for successful integration. [3:25]

After downloading, several files become available. The `checkpoint` file indicates the path to the model's weights, primarily stored in `model.ckpt.data`. The `encoder.json` file serves as the vocabulary, mapping tokens to their corresponding IDs, with a vocabulary size of 50257. [4:10]

The `vocab.bpe` file contains information related to Byte Pair Encoding (BPE), a subword tokenization scheme. It lists merged tokens based on frequency, with the most probable merges appearing at the top. The special convention "g." is used to signify the end of one token and the beginning of another, akin to a space. [4:10]

The `hparams.json` file stores the model's configuration settings, including vocabulary size (50257), context length (1024), embedding dimension (768), number of attention heads, and number of Transformer blocks. These parameters are for the smallest GPT-2 model (124 million parameters), which has 12 Transformer blocks. Larger GPT-2 models (355M, 774M, 1.5B parameters) also have their weights publicly available, and the same code can be adapted for them. [4:10]

## The `download_and_load_gpt2` Function

The `download_and_load_gpt2` function orchestrates the download and parameter loading process. It first calls `download_file` to retrieve the seven necessary files. [5:40]

Following the download, it identifies the latest TensorFlow checkpoint path using `tf.train.latest_checkpoint`. A settings dictionary is created, mirroring the information in `hparams.json`, including vocabulary size, context length, embedding dimension, number



of attention heads, and number of Transformer blocks. [5:40]

The core of the parameter loading happens in the call to `load_gpt2_params_from_tf_checkpoint`. This function takes the TensorFlow checkpoint path and the settings dictionary to extract and organize the model's parameters into a structured params dictionary. [5:40]

## The params Dictionary Structure

The params dictionary is central to integrating the pre-trained weights. It contains five key components: `wte` (weight token embeddings), `wpe` (weight positional embeddings), `blocks` (weights for Transformer blocks), `ln_f_scale` (final layer normalization scale), and `ln_f_shift` (final layer normalization shift). [6:25]

**Token Embeddings (`wte`):** This component holds the embedding matrix for each token in the vocabulary. For GPT-2, with a vocabulary size of 50257 and an embedding dimension of 768, this matrix has dimensions (50257, 768). [8:30]

**Positional Embeddings (`wpe`):** These embeddings capture the position of tokens within a sequence. Given a context size of 1024 and an embedding dimension of 768, this matrix has dimensions (1024, 768). [8:30]

**Transformer Blocks (`blocks`):** This is a nested dictionary containing all trainable weights and parameters within the Transformer blocks. It includes weights for the multi-head attention mechanism (query, key, value matrices and biases), the feed-forward neural network (fully connected and projection layers, including weights and biases), and the scale and shift parameters for the two layer normalization layers within each block. [9:15]

The structure within blocks is hierarchical, typically following `blocks -> transformers -> h{n} -> attention` for attention weights and biases, and `blocks -> transformers -> h{n} -> mlp` for feed-forward network weights and biases. Layer normalization parameters are accessed via `blocks -> transformers -> h{n} -> layer_norm_1` and `layer_norm_2`. [9:15]

**Final Layer Normalization (`ln_f_scale`, `ln_f_shift`):** These keys correspond to the scale and shift parameters of the final layer normalization layer applied after the last Transformer block. [12:30]

The `load_gpt2_params_from_tf_checkpoint` function populates this params dictionary by extracting the relevant weights and biases from the TensorFlow checkpoint files and organizing them according to this structure. The function returns both the settings dictionary and the populated params dictionary. [14:00]

## Executing the Download and Loading

The process begins by importing the `download_and_load_gpt2` function from the `gpt_download_3.py` script. This function is then called with the desired model size (e.g., '124M' for the 124 million parameter model) and a target directory (e.g., 'gpt2') for saving the downloaded files. [14:45]

The download process, which involves transferring approximately 500 megabytes of data, can take considerable time depending on internet connectivity. Users are advised to ensure a stable connection and avoid interrupting the process. The `tqdm` library provides visual feedback on the download progress. [15:15]

Once the download and parameter extraction are complete, the `settings` dictionary can be inspected. It contains keys such as `n_vocab`, `n_ctx`, `n_embed`, `n_head`, and `n_layer`, reflecting the model's configuration. [16:10]

Similarly, printing the keys of the `params` dictionary reveals the five main components: `blocks`, `b`, `g`, `wpe`, and `wte`. [16:10]

To verify the dimensions of the loaded weights, one can access specific components, such as the token embedding matrix (`params['wte']`). The shape of this tensor should match the expected dimensions (e.g., 50257x768 for the 124M model). [17:00]

## Adapting the GPT Configuration

Before integrating the weights, a slight adjustment to the GPT configuration is necessary. The original configuration used a context length of 256, whereas the GPT-2 model uses 1024. Additionally, the `bias` parameter for query, key, and value matrices in the attention mechanism was set to `False` during the initial training but is `True` in GPT-2. [17:40]

The configuration is updated to reflect these changes: the context length is set to 1024, and the query-key-value bias is enabled. While biases are not always used in modern LLMs, enabling them here ensures consistency with the pre-trained GPT-2 weights. [17:40]

A new instance of the GPT model class is then created using this updated configuration. [18:20]

```
# Example of updating configuration
# new_config = old_config
# new_config['n_ctx'] = 1024
# new_config['bias'] = True
# model = GPT(new_config)
```

## Integrating Weights into the Model Class

The core task is now to load the downloaded GPT-2 weights into the custom GPT model instance. This involves mapping the parameters from the `params` dictionary to the corresponding layers within the GPT model class. [18:50]

A function, `load_weights_into_gpt`, is defined to handle this integration. It takes the GPT model instance and the `params` dictionary as input. [18:50]

Within the Transformer blocks, the attention layer's query, key, and value weight matrices are updated. The fused weight matrix from `params['blocks']['transformers']['h{n}']['attn']['c_attn']['w']` is split along its columns to obtain the individual query, key, and value weight matrices, which are then assigned to the corresponding attributes of the attention object (e.g., `at.q_w`, `at.k_w`, `at.v_w`). The same process is applied to the biases. [19:30]

Similarly, the weights and biases for the feed-forward neural network's fully connected layer (`mlp.c_fc`) and projection layer (`mlp.c_proj`) are loaded from `params` and assigned to the model's feed-forward network object. [20:30]

The scale and shift parameters for the two layer normalization layers within each Transformer block (`ln_1` and `ln_2`) are also loaded from `params` and assigned to the corresponding attributes (`ln_1.g`, `ln_1.b`, `ln_2.g`, `ln_2.b`). [21:15]

Finally, the scale and shift parameters for the final layer normalization (`ln_f`) are loaded from `params['g']` and `params['b']` and assigned to `model.ln_f.g` and `model.ln_f.b`. [22:00]

A key aspect of GPT-2's architecture is weight tying, where the token embedding weights (`wte`) are reused for the final output projection layer. This means no separate weights need to be loaded for this layer, reducing the total parameter count. [22:30]

The `load_weights_into_gpt` function iterates through the `params` dictionary and assigns the loaded weights to the corresponding layers in the GPT model instance, effectively replacing the random initializations with the pre-trained GPT-2 weights. [23:00]

## Testing the Model with Pre-trained Weights

With the weights loaded, the model is now ready for testing. The `generate` function is used to produce text, passing the model instance with the loaded GPT-2 weights. The input prompt is "every effort moves you," and parameters like maximum new tokens (25), temperature (1.4), and top-k (50) are set. [23:45]

A comparison is made with the text generated *\*before\** loading the GPT-2 weights, which was incoherent. Upon running the generation with the pre-trained weights, the output is significantly more coherent: "every effort moves you toward finding an ideal new way to practice something what makes us want to be on top of that." This demonstrates the effectiveness of using pre-trained weights. [24:15]

The ability to experiment with hyperparameters like temperature is highlighted. Increasing the temperature to 10 results in more random output, as expected, illustrating the flexibility of

having a locally runnable model. [25:30]

The successful integration and coherent output confirm that the GPT-2 weights have been loaded correctly. This milestone enables further research and experimentation, such as modifying the model architecture, varying hyperparameters, or exploring different optimizers.

[27:00]

### **Next Steps: Fine-tuning**

The next phase of the lecture series will focus on fine-tuning the pre-trained model. This involves adapting the model for specific applications, such as text classification or building a quiz app, by further training it on domain-specific data. [28:30]

This application-oriented approach will build upon the foundation of having a functional GPT model running locally, providing a strong basis for machine learning engineering and research. [28:30]

---

\*End of Chapter 32. Source: <https://www.youtube.com/watch?v=yXrGeDNuymY>\*

---

## Chapter 33: Introduction to Large Language Model Fine-Tuning

---

*Source video: Introduction to LLM Finetuning | Python Coding with hands-on-example*

*Duration: 27 minutes | Playlist position 33*

### Understanding the Need for Fine-Tuning

After completing the stages of understanding the LLM architecture, attention mechanisms, data preparation, pre-training, and model evaluation, the series is ready to move to the final stage: fine-tuning. While a pre-trained large language model (LLM) can produce sensible outputs for general inputs, it may not suffice for specific tasks or domains. Fine-tuning adapts a pre-trained model to a particular task by training it further on a smaller, domain-specific dataset. [0:07]

The necessity for fine-tuning arises when a company wants to create a chatbot based on its private data, or an educational institution aims to build an app using its specific content. A model pre-trained on general internet data will not provide the desired, specialized answers. Fine-tuning involves retraining the model on this additional, custom data, allowing its weights and biases to adapt to the new information and specific requirements. [0:07]

Formally, LLM fine-tuning is the additional training of a pre-existing model using a smaller, domain-specific dataset. This process helps the model adapt its parameters, weights, and biases to better suit the target application. OpenAI also provides resources and instructions for fine-tuning their models. [0:07]

A practical example of fine-tuning involves creating a chatbot that mimics a specific person's writing style or tone. If a person has a personal website with their publications, talks, and blog posts, this data can be used to fine-tune a pre-trained LLM. The resulting chatbot would then communicate using the individual's unique tone and sentence construction, rather than the generic style of a standard pre-trained model. [0:07]

Another example is developing a chatbot to answer queries about a researcher's specific publications. By feeding the model data solely from these publications, the chatbot can provide accurate and contextually relevant answers. This approach addresses the need for specialized knowledge and communication styles that general pre-trained models lack. [0:07]

### Categories of Fine-Tuning

Fine-tuning can be broadly categorized into two main types: instruction fine-tuning and classification fine-tuning. Instruction fine-tuning is more common and involves training the

LLM on a set of tasks using specific instructions. For instance, instructing the model to determine if a given text is spam and respond with "yes" or "no," or to translate a sentence into German, are examples of instruction fine-tuning. [0:07]

Classification fine-tuning, while less common, is similar to traditional machine learning classification tasks but utilizes LLMs. In this approach, no explicit instructions are given to the LLM; instead, the model is trained to categorize input data into predefined classes. For example, classifying an email as "spam" or "not spam" without specific instructions on how to respond. [0:07]

The key difference lies in how the task is presented. Instruction fine-tuning provides explicit commands (e.g., "answer with yes or no"), while classification fine-tuning relies on the model learning to predict a category based on the input alone. Classification fine-tuning can also be used for sentiment analysis, categorizing text into emotions like "angry," "sad," or "happy." [0:07]

Instruction fine-tuning is generally more versatile, capable of handling a broader range of tasks such as translation and summarization, and typically requires larger datasets and more computational power. Classification fine-tuning, on the other hand, focuses on a narrower set of tasks, primarily categorization. [0:07]

## Parameter-Efficient Fine-Tuning Methods

Within instruction fine-tuning, methods like LoRA (Low-Rank Adaptation) and QLoRA have emerged to make the process more efficient. These techniques fall under parameter-efficient fine-tuning (PEFT), where only a subset of the model's parameters are updated, while the rest are frozen. This significantly reduces memory requirements. [0:07]

LoRA works by fine-tuning two smaller matrices that approximate the larger weight matrices of the model, rather than fine-tuning all the original weights. QLoRA is a further optimization that quantizes the LoRA adapter weights to lower precision, making it even more memory-efficient. These methods aim to reduce the substantial computational resources traditionally needed for full fine-tuning. [0:07]

## Hands-On Classification Fine-Tuning: Spam Detection

The current lecture will focus on a hands-on classification fine-tuning problem: building a model to classify emails as spam or not spam. This involves several stages, starting with data acquisition and preprocessing. [0:07]

The first step is downloading the dataset, which is available from the UC Irvine Machine Learning Repository. Specifically, the "SMS Spam Collection" dataset contains a large number of emails, categorized as either spam or ham (not spam). The dataset includes 747

spam messages and a larger number of non-spam messages, making it imbalanced. [0:07]

To address the imbalance, a simple approach is to subsample the non-spam (ham) messages to match the number of spam messages. In this case, 747 ham messages are randomly selected, resulting in a balanced dataset with 747 spam and 747 ham instances. [0:07]

Next, the labels "ham" and "spam" are encoded numerically, with "ham" assigned to 0 and "spam" to 1. This process is analogous to converting text tokens into numerical IDs during LLM pre-training. [0:07]

The balanced dataset is then split into training, validation, and testing sets. Typically, 70% of the data is used for training, 10% for validation, and 20% for testing. This split ensures that the model can be trained, its hyperparameters tuned, and its final performance evaluated on unseen data. [23:25]

After splitting, the training, validation, and testing dataframes are saved as CSV files for later use. The total number of instances in the balanced dataset is 1494 (747 spam + 747 ham). The split results in 1045 training instances, 149 validation instances, and 300 testing instances, which sum up correctly. [23:25]

For classification tasks using LLMs, the architecture is typically augmented at the end with another neural network layer, often a softmax layer, to produce the final classification output (spam or not spam, i.e., 0 or 1). [23:25]

The process of downloading and preprocessing the data, including balancing and splitting, marks the completion of the first major step. Subsequent lectures will cover creating data loaders, initializing the LLM, loading pre-trained weights, and implementing the fine-tuning process. [23:25]

---

\*End of Chapter 33. Source: [https://www.youtube.com/watch?v=yZpy\\_hsC1bE](https://www.youtube.com/watch?v=yZpy_hsC1bE)\*

---

## Chapter 34: Dataloaders in LLM Classification Finetuning

---

*Source video: Dataloaders in LLM Classification Finetuning | Python Coding | Hands on LLM project*

*Duration: 31 minutes | Playlist position 34*

### Recap of Fine-tuning and Data Preprocessing

This chapter continues the exploration of fine-tuning large language models (LLMs) for specific tasks. Previously, the focus was on classification fine-tuning, specifically an email classification problem where an LLM classifies emails as either spam or not spam (ham).

[\[0:05\]](#)

In the last lecture, the initial steps of data preparation were covered. This involved downloading an email dataset from the UC Irvine Machine Learning Repository, which contained approximately 747 spam emails and over 3,000 ham emails. The data was then preprocessed by balancing the dataset, randomly sampling 747 ham emails to match the number of spam emails. [\[0:05\]](#)

Further preprocessing included encoding the labels: "ham" was mapped to 0 and "spam" to 1. The entire dataset, now consisting of 1494 entries (747 spam, 747 ham), was split into training (70%), validation (10%), and testing (20%) sets. These processed dataframes were then saved as CSV files, ready for the next stage. [\[0:05\]](#)

The ultimate goal is to feed input text (emails) into an LLM architecture and obtain a classification output (spam or not spam). However, before reaching the model training stage, an essential intermediate step is required: creating data loaders. Data loaders are crucial for efficiently managing and batching data, especially in deep learning frameworks like PyTorch. [\[0:05\]](#)

PyTorch provides `torch.utils.data.Dataset` and `torch.utils.data.DataLoader` classes, which facilitate easy access to data samples, batch processing, and even parallel processing. The primary purpose of data loaders is to transform the raw dataset into structured input-target pairs, grouped into batches, suitable for model training. [\[0:05\]](#)

### Handling Variable Sequence Lengths

A key challenge in preparing text data for LLMs is that text messages (emails in this case) often have varying lengths. However, for efficient batch processing, all sequences within a batch must have the same length. For example, a desired batch might consist of eight emails, each converted into 120 tokens, with corresponding binary labels. [\[0:05\]](#)

Two main approaches exist to standardize sequence lengths:



**1. Truncation to the shortest length:** This involves finding the shortest email in the dataset and truncating all other emails to match that length. The disadvantage of this method is a significant loss of information from longer texts, making it generally not recommended.

[\[0:05\]](#)

**2. Padding to the longest length:** This approach identifies the longest email in the dataset and then pads all shorter emails with special tokens until they reach the maximum length. This method ensures no information is lost and is the recommended strategy. [\[0:05\]](#)

The padding token used is the "end of text" token. To tokenize the text, a byte pair encoder (BPE) tokenizer, specifically tiktoken (the tokenizer used by OpenAI's GPT models), is employed. This tokenizer converts sentences into a sequence of token IDs. Since BPE is a subword tokenizer, a single word might correspond to multiple tokens. [\[8:39\]](#)

For GPT-2 models, the "end of text" token has an ID of 50256. This ID is part of GPT-2's vocabulary, which has a size of 50257 (token IDs from 0 to 50256). The "end of text" token typically signifies the end of a document or sentence and helps the model distinguish between separate document sources during training. Using this specific token for padding makes symbolic sense, as it clearly indicates the absence of further meaningful text. [\[10:21\]](#)

The overall data preparation workflow involves:

1. Taking the input text from CSV files.
2. Using a tokenizer to convert the text into token IDs.
3. Identifying the longest sequence (email) in the dataset.
4. Padding all shorter sequences with the 50256 "end of text" token to match the maximum length. [\[12:29\]](#)

## Implementing the SpamDataset Class

To implement this workflow, a custom PyTorch Dataset class, named SpamDataset, is defined. This class handles the loading, tokenization, and padding/truncation of the email data. [\[12:29\]](#)

The SpamDataset class performs three main functions:

1. It identifies the longest sequence in the training dataset to determine the target length for all sequences.
2. It converts each text message into a sequence of token IDs using the specified tokenizer.
3. It ensures all sequences are padded with the pad\_token\_id (default 50256) to match the length of the longest sequence. [\[12:29\]](#)

When an instance of `SpamDataset` is created, it requires the path to the CSV file, the tokenizer object (e.g., from `tiktoken`), an optional `max_length` parameter, and the `pad_token_id`. If `max_length` is not provided, the dataset automatically computes the longest sequence length from the data itself. [12:29]

Inside the `__init__` method of the `SpamDataset` class:

1. The CSV file is read into a pandas `DataFrame` using `pd.read_csv`.
2. Each text message in the `DataFrame` is tokenized using `tokenizer.encode()`, converting sentences into lists of token IDs.
3. If `max_length` is `None`, a helper function `longest_encoded_length` is called to find the maximum length among all tokenized sequences.
4. If a `max_length` is provided by the user, and some sequences are longer than this value, those sequences are truncated to `max_length`.
5. Finally, all tokenized sequences are padded with the `pad_token_id` (50256) until they reach the `max_length`. [12:29]

The `__getitem__` method of the `SpamDataset` class is responsible for returning a single sample. When called, it returns two tensors: an encoded tensor containing the padded token IDs for a text message, and a `label` tensor containing its corresponding binary label (0 or 1). This ensures that every sentence has been converted into token IDs of equal length, ready for batching. [12:29]

An example of instantiating the `SpamDataset` for the training data:

```
# Assuming 'tokenizer' is an initialized tiktoken tokenizer
# and 'train.csv' is available
train_dataset = SpamDataset(csv_file='train.csv', tokenizer=tokenizer, max_length=None)
max_length_computed = train_dataset.max_length
print(f"Maximum sequence length in training data: {max_length_computed}")
```

When `max_length` is not explicitly set, the `SpamDataset` computes it from the training data. In this project, the longest sequence in the training dataset was found to be 120 tokens. This length is well within the GPT-2 model's context length of 1024 tokens. [12:29]

For the validation and test datasets, it is crucial to use the `max_length` determined from the training data (120 tokens in this case). This ensures consistency across all data splits. Any sequences in the validation or test sets that exceed this `max_length` will be truncated. [12:29]

```
# Create validation and test datasets, using the max_length from training
validation_dataset = SpamDataset(csv_file='validation.csv', tokenizer=tokenizer, max_length=max_length_computed)
test_dataset = SpamDataset(csv_file='test.csv', tokenizer=tokenizer, max_length=max_length_computed)
```

Printing the `max_length` for these datasets would confirm it is 120, as it was explicitly passed as a parameter. [\[22:53\]](#)

## Instantiating DataLoaders

Once the `Dataset` objects are defined, they serve as input to the `DataLoader`. The `DataLoader` is responsible for iterating over the dataset, creating batches, and providing them to the model. [\[22:53\]](#)

Key parameters for the `DataLoader` include:

- `dataset`: The `Dataset` object to load data from.
- `batch_size`: The number of samples per batch (e.g., 8).
- `num_workers`: The number of subprocesses to use for data loading. Setting this to 0 simplifies debugging by disabling parallel processing.
- `drop_last`: If `True`, the last batch will be dropped if its size is smaller than `batch_size`. ``

Instances of `DataLoader` are created for the training, validation, and test datasets:

```
import torch
from torch.utils.data import DataLoader

batch_size = 8
num_workers = 0 # For simplicity, no parallel processing

train_loader = DataLoader(train_dataset, batch_size=batch_size, num_workers=num_workers, drop_last=True)
validation_loader = DataLoader(validation_dataset, batch_size=batch_size, num_workers=num_workers, drop_last=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, num_workers=num_workers, drop_last=True)
```

These loaders can now be used to easily extract batches of input and target tensors. [\[22:53\]](#)

To verify the data loaders are working correctly, one can iterate through a loader and inspect the dimensions of the returned batches. For example, iterating through the `train_loader`:

```
for input_batch, label_batch in train_loader:
    print(f"Input batch dimension: {input_batch.shape}")
    print(f"Label batch dimension: {label_batch.shape}")
    break # Only print for the first batch
```

The expected output for the first batch, with a `batch_size` of 8 and `max_length` of 120, would be an input batch dimension of `torch.Size([8, 120])` and a label batch dimension of `torch.Size([8])`. This confirms that each input batch contains 8 samples, each with 120 token IDs, and each label batch contains 8 corresponding labels. [\[22:53\]](#)

The total number of samples in the balanced dataset is 1494 (747 spam + 747 ham). With 70% allocated for training, there are 1045 training samples. Given a `batch_size` of 8, the number of training batches is  $1045 / 8 \approx 130$ . [\[22:53\]](#)

The length of a `DataLoader` object directly provides the number of batches it will yield:

```
print(f"Number of training batches: {len(train_loader)}")
print(f"Number of validation batches: {len(validation_loader)}")
print(f"Number of test batches: {len(test_loader)}")
```

This would output 130 training batches, 19 validation batches, and 38 test batches, consistent with the data split percentages (10% validation, 20% test). The overall dimension of the entire training loader, if it were a single tensor, would be (130, 8, 120). [22:53]

## Conclusion of Data Preparation

With the creation of the `train_loader`, `validation_loader`, and `test_loader`, all data preparation steps are complete. These data loaders provide an efficient mechanism to extract input and target batches as needed during model training. Unlike pre-training LLMs where the target might be a shifted sequence of token IDs, for classification fine-tuning, the target labels are simply 0 or 1. [29:09]

The next lectures will focus on Stage 2 of the fine-tuning process: initializing the LLM, loading pre-trained weights, modifying the model for the classification task, and finally evaluating the fine-tuned model. The thorough data preprocessing and the use of `Dataset` and `DataLoader` classes lay a strong foundation for effective model training. [29:09]

---

\*End of Chapter 34. Source: <https://www.youtube.com/watch?v=f6zqClXOh7Y>\*

---

## Chapter 35: Coding the Model Architecture for LLM Classification Fine-tuning

---

*Source video: Coding the model architecture for LLM classification fine-tuning*

*Duration: 34 minutes | Playlist position 35*

### Introduction to Classification Fine-tuning

This chapter continues the classification fine-tuning example from previous lectures in the "Build Large Language Models from Scratch" series. The primary goals are to initialize the model with pre-trained weights and modify the LLM architecture to perform classification tasks. [\[0:05\]](#)

The project addresses a problem where a large language model is used to classify text as either "spam" or "not spam." This process falls under fine-tuning, which is crucial for adapting a pre-trained model to a specific task. While the series has covered building a pre-trained LLM, further fine-tuning is necessary for task-specific performance. [\[0:05\]](#)

Fine-tuning can be broadly categorized into instruction fine-tuning and classification fine-tuning. This hands-on email classification example focuses on the latter. [\[0:05\]](#)

### Recap of Data Preparation

So far, three key steps have been completed: downloading the dataset, preprocessing it, and creating data loaders. [\[0:05\]](#)

The dataset used is the SMS Spam Collection from the UC Irvine Machine Learning Repository. Upon download, it contains labels like "ham" (not spam)

---

## Chapter 36: Coding a Fine-Tuned LLM Spam Classification Model

---

*Source video: Coding a fine-tuned LLM spam classification model | From Scratch*

*Duration: 49 minutes | Playlist position 36*

### Introduction and Project Recap

This lecture continues the fine-tuning based classification project, focusing on calculating classification loss and accuracy, and implementing the training and testing loops to complete the entire fine-tuning process. [0:06]

Previously, the project involved several key steps. First, the SMS spam collection dataset was downloaded from the UCI Machine Learning Repository. This dataset contains text messages labeled as either spam or not spam. [0:06]

Upon inspection, the dataset was found to be imbalanced, with approximately 4,800 non-spam messages and only 747 spam messages. To address this, the dataset was balanced so that both categories contained 747 messages. [0:06]

Following balancing, data loaders were implemented. The dataset was split into training (70%), testing (20%), and validation (10%) sets. For the training data, batches were created, each containing eight samples. There were around 130 such batches for the training data. [0:06]

Each text message was converted into 120 token IDs. To ensure uniform length, smaller messages were padded with the end-of-text token, which has an ID of 50256. The input tensor thus contained these token IDs, while the target tensor contained binary labels (0 for no-spam, 1 for spam). Similar batching was applied to the validation (19 batches) and testing (38 batches) data. [0:06]

The model architecture was then modified. Initially, the model's final output layer was a neural network that took an input of 768 (the embedding dimension) and produced an output of 50257 (the vocabulary size). For the classification task, this was replaced with a classification head that still took 768 inputs but produced only two outputs, corresponding to "spam" or "no-spam." [0:06]

When an input, such as "Do you have time," is passed through this modified architecture, the output is a tensor. For a single batch, this tensor has a shape of 1x4x2, where 1 is the batch size, 4 represents the four tokens in the input ("Do," "you," "have," "time"), and 2 represents the two outputs (spam/no-spam) for each token. [0:06]

Crucially, for classification, only the output corresponding to the *\*last\** token (e.g., "time") is considered. This is because the last token's representation incorporates information from all preceding tokens through its attention weights. This last token's output provides two values, which are the logits for the "spam" and "no-spam" classes. [0:06]

## Converting Model Outputs to Class Label Predictions

The first step in today's lecture is to convert these two output values (logits) from the last token into a class label prediction. For example, if the input text is "You won the lottery" and the model outputs two values for the last token, these values need to be interpreted as either spam or no-spam. [4:50]

A common approach is to apply a softmax function to these two outputs. This converts the logits into a set of probabilities that sum to one. For instance, if the logits are -3.5983 and 3.9902, applying softmax might yield probabilities like 0.0005 and 0.9995. [4:50]

After obtaining probabilities, the argmax function is used to find the index corresponding to the highest probability. If 0.9995 is at index 1, then the model predicts class 1 (spam). If 0.99 is at index 0, it predicts class 0 (no-spam). [4:50]

It's important to note that applying softmax is technically optional. The argmax of the raw logits will yield the same class prediction as the argmax of the softmax probabilities, because softmax is a monotonically increasing function. Therefore, one can directly apply argmax to the two output logits to determine the predicted class label. [4:50]

## Calculating Classification Accuracy

Classification accuracy measures the percentage of correct predictions across a dataset. To determine this, the argmax-based prediction logic is applied to all examples in the dataset, and the predicted labels are then compared with the actual (true) labels. [10:00]

A function named `calculate_accuracy_loader` is defined to compute accuracy for a given data loader (e.g., training, validation, or test loader). This function takes the data loader and an optional `num_batches` parameter. If `num_batches` is not specified, it defaults to the total number of batches in the loader. If specified, it computes accuracy for the minimum of the specified number and the total batches. [10:00]

Inside the function, it iterates through each batch. For every batch, the input samples are passed through the model to obtain the logits for the last output token. The argmax function is then applied to these logits to get the predicted labels for the entire batch. [10:00]

The predicted labels are compared with the target (true) labels. If they match, a `correct_predictions` counter is incremented. A `num_examples` counter is also incremented for each sample processed. Finally, the accuracy is calculated as `correct_predictions /`

num\_examples. [10:00]

To illustrate, the `calculate_accuracy_loader` function is called for the training, validation, and test loaders, with `num_batches` set to 10 for simplicity. Since the model has not yet been fine-tuned, the initial accuracies are very low: training accuracy is 46%, validation accuracy is 45%, and test accuracy is 48%. These values are worse than a random coin toss, indicating the need for fine-tuning. [13:08]

## Defining the Loss Function

To improve prediction accuracies, the model needs to be fine-tuned, which involves optimizing its parameters. This optimization relies on defining a loss function that quantifies the difference between the model's predicted values and the true target values. [16:00]

Once the loss function is defined, its partial derivative with respect to all trainable weights (gradients) is calculated. These gradients are then used to update the weights using an optimization algorithm like gradient descent:  $w_{\text{new}} = w_{\text{old}} - \eta * \nabla L / \nabla w$ , where  $\eta$  is the learning rate. The goal is to iteratively update parameters to minimize the loss function, thereby improving accuracy. [16:00]

For classification tasks with categorical predictions, the standard loss function is the **Categorical Cross-Entropy Loss**. It is defined as:  $-\sum (Y_i * \log(P_i))$ , where  $Y_i$  is the true (one-hot encoded) label for class  $i$ , and  $P_i$  is the predicted probability for class  $i$ . [16:00]

For example, if the true label for a non-spam message is  $[1, 0]$  (one-hot encoded) and the model predicts  $[0.8, 0.2]$ , the cross-entropy loss would be  $-(1 * \log(0.8) + 0 * \log(0.2)) = -\log(0.8) \approx 0.2231$ . If the prediction perfectly matched the true label  $[1, 0]$ , the loss would be  $-(1 * \log(1) + 0 * \log(0)) = 0$ , which is the desired outcome. [16:00]

Cross-entropy loss is a good measure because it penalizes incorrect predictions more heavily and is differentiable, which is crucial for backpropagation. Classification accuracy itself cannot be used as a loss function because it is not differentiable. [16:00]

A `calculate_loss_batch` function is implemented using `torch.nn.functional.cross_entropy`. This function takes an input batch and a target batch, passes the input through the model, extracts the logits of the last output token, and then computes the cross-entropy loss between these logits and the target batch. [16:00]

Similar to the accuracy function, a `calculate_loss_loader` function is defined. It iterates through batches in a data loader, calls `calculate_loss_batch` for each, sums the individual batch losses, and returns the average loss per batch. This average loss is what the training



process will aim to minimize. [16:00]

To demonstrate, the `calculate_loss_loader` function is run for the training, validation, and test loaders, with `num_batches` set to 5. As expected before fine-tuning, the initial loss values are very high, reflecting the poor accuracy observed earlier. [16:00]

## Implementing the Training Loop

With the loss and accuracy calculation utilities in place, the next step is to implement the training function to fine-tune the model. This involves adjusting the model's parameters to minimize the training loss. [23:00]

The training process is structured with an outer loop for epochs (one pass through the entire dataset) and an inner loop for batches. For each batch, the following steps occur:

1. Calculate the loss on the current batch.
2. Perform a backward pass to compute the loss gradients with respect to the model parameters.
3. Update the model weights using an optimizer, following the principle  $w_{\text{new}} = w_{\text{old}} - \eta \cdot \nabla L / \nabla w$ . [23:00]

This iterative process, repeated over multiple epochs, allows the parameters to be updated, gradually minimizing the loss function and improving model performance. This training function is similar to the one used for pre-training the LLM, highlighting that fine-tuning is essentially a second training process on specific, supervised data. [23:00]

The `train_classifier_simple` function encapsulates this training loop. It involves several key steps:

1. **Set model to training mode:** `model.train()`.
2. **Reset loss gradients:** `optimizer.zero_grad()` clears gradients from the previous batch.
3. **Calculate loss, gradients, and update weights:**
  - Compute loss for the current batch.
  - Perform `loss.backward()` to calculate gradients.
  - Execute `optimizer.step()` to update model weights. The optimizer used is `torch.optim.AdamW`, a variant of Adam with weight decay, which helps avoid local minima and promotes faster, smoother convergence. ``
4. **Track examples seen:** `input_batch.shape[0]` gives the number of samples in the current batch, which is added to a running count of `examples_seen`.
5. **Increment global step:** A `global_step` counter is incremented after each batch.

**6. Evaluation during training:** At a specified `evaluation_frequency` (e.g., every 50 batches), the training and validation losses are printed. This is done by calling an `evaluate_model` function, which calculates loss on a small subset of batches (e.g., `evaluation_iteration=5`) for quick feedback. [23:00]

**7. Epoch-end evaluation:** After each full epoch, the training and validation accuracies are calculated and printed. [23:00]

The `train_classifier_simple` function is called with the modified GPT model, training and validation loaders, the AdamW optimizer, the number of epochs, `evaluation_frequency` (set to 50), and `evaluation_iteration` (set to 5). [23:00]

After running the training process (which took approximately 8.8 minutes on a MacBook Air), the results show significant improvement. The training loss decreases to 0.083, and the validation loss goes down to 0.074. Correspondingly, training accuracy improves to nearly 100%, and validation accuracy reaches 97.5%. [23:00]

Plots of the training and validation loss over time show a sharp downward slope, indicating effective learning. The validation loss closely tracks the training loss, with no significant gap, which suggests little to no overfitting during this phase of training. Similarly, plots of training and validation accuracy show a clear upward trend, reaching high values. [23:00]

It's important to remember that the loss and accuracy values printed during training (and shown in the plots) are based on `evaluation_iteration=5` batches, not the entire dataset, for efficiency. [23:00]

## Full Dataset Evaluation

To obtain a more representative measure of the model's performance, the `calculate_accuracy_loader` function is run on the *entire* training, validation, and test datasets, rather than just a few batches. This provides a comprehensive evaluation of the fine-tuned model. [36:00]

The results for the full dataset evaluation are:

- Training accuracy: 97%
- Validation accuracy: 97%
- Test accuracy: 95% ``

These results indicate that the training and validation set performances are almost identical. A slight discrepancy of 2% between the training and test set accuracy (test being slightly lower) suggests a small amount of overfitting. While common, this gap could potentially be minimized by adjusting model settings, such as increasing the dropout rate or the `weight_decay` parameter in the optimizer configuration. [36:00]

Further experimentation is encouraged, including modifying the dropout rate in the model architecture, adjusting the learning rate and `weight_decay` in the optimizer, or unfreezing additional Transformer blocks (e.g., the last three instead of just the 12th block and the final normalization layer) to see their impact on test accuracy. [36:00]

## Using the Model on New Data (Inference)

The final step is to use the fine-tuned model to classify new, unseen text messages. This is the real test of its performance as a spam classifier. [40:00]

A `classify_review` function is defined to handle this inference process. It takes a raw text string as input and performs several preprocessing steps before passing it to the model:

1. **Tokenization:** The input text is converted into token IDs using the `tiktoken` tokenizer (a byte pair encoder).
2. **Determine `max_length`:** The supported context length of the model is obtained from `model.positional_embedding.weight.shape[0]` (e.g., 1024). This is compared with the maximum token length observed in the training data (e.g., 120). The `max_length` for processing new inputs is set to the minimum of these two values.
3. **Padding:** If the tokenized input sequence is shorter than `max_length`, it is padded with the `end_of_text_token` (50256) until it reaches `max_length`. This ensures all input sequences have a consistent length for batch processing.
4. **Tensor Conversion:** The padded token IDs are converted into a `torch.Tensor`, and a batch dimension is added.
5. **Model Inference:** The prepared input tensor is passed through the fine-tuned model. The logits of the last output token are extracted.
6. **Prediction:** `torch.argmax` is applied to these logits to obtain the final predicted label (0 for no-spam, 1 for spam). [40:00]

Two example sentences are used to test the `classify_review` function:

- Spam message: "You are a winner you have been specially selected to receive \$1,000 cash or \$2,000 reward." The model correctly predicts this as spam.
- Legitimate message: "Hey just wanted to check if we are still on for dinner tonight. Let me know." The model correctly predicts this as no-spam. ``

These examples demonstrate that the fine-tuned model is performing well in classifying new text messages as spam or not spam. [40:00]

## Saving and Loading the Model

To reuse the trained model without needing to retrain it, it's crucial to save its parameters. PyTorch provides `torch.save()` and `torch.load()` functionalities for this purpose. [\[47:00\]](#)

To save the model's parameters (its `state_dict`), the following command can be used:

```
torch.save(model.state_dict(), 'model_path.pth')
```

This saves the learned weights and biases to a specified file. [\[47:00\]](#)

To load the saved parameters back into a model, first an instance of

---

## Chapter 37: Introduction to LLM Instruction Fine-tuning | Loading Dataset | Alpaca Prompt format

---

*Source video: Introduction to LLM Instruction Fine-tuning | Loading Dataset | Alpaca Prompt format*

*Duration: 25 minutes | Playlist position 37*

### Introduction to Instruction Fine-tuning

This lecture introduces instruction fine-tuning within the "Build Large Language Models from Scratch" series. Previously, the series covered data preparation, the attention mechanism, LLM architecture (Stage 1), and the pre-training process, including loading pre-trained weights from models like OpenAI's GPT-2 to construct a foundation model.

[\[0:07\]](#)

The series then moved into fine-tuning, specifically classification-based fine-tuning, where a pre-trained model was fine-tuned to classify emails as spam or not spam. This chapter begins a new category of fine-tuning: instruction-based fine-tuning, which is more common and will be used to build a personal assistant from scratch over the next five to six lectures. [\[0:07\]](#)

### Why Instruction Fine-tuning is Necessary

Pre-trained large language models (LLMs), like the one developed in this series, excel at text completion but struggle with following specific instructions. For instance, a pre-trained LLM cannot fix grammar in text, convert text from active to passive voice, or make text concise and professional without further fine-tuning. [\[1:40\]](#)

Instruction fine-tuning enables an LLM to understand and execute commands, which is crucial for building applications like personal assistants. Without it, an LLM can generate new tokens and sentences but lacks the ability to interpret and act upon explicit instructions.

[\[1:40\]](#)

### Real-world Examples of Instruction Fine-tuning

Consider an e-commerce company aiming to build a customer support chatbot. A pre-trained model would struggle to understand specific customer queries related to the company's unique policies, products, and services, as this information is often private and not part of its general pre-training data. [\[1:40\]](#)

Instruction fine-tuning allows the company to train the LLM on domain-specific instructions, such as providing steps for initiating a return or detailing refund policies. This ensures the chatbot not only understands general language but also responds accurately with company-specific information, enhancing relevance and accuracy. [\[1:40\]](#)

Another example is a personalized healthcare virtual assistant. While a pre-trained model might possess general healthcare knowledge, it lacks specific information about a particular healthcare provider's practices, treatment plans, or individual patient histories. [1:40]

Fine-tuning with specific instructions related to healthcare guidelines and patient needs allows the assistant to understand medical terminology and provide personalized advice, such as a weekly plan for diabetes management based on specific medical guidelines. [1:40]

In summary, fine-tuning serves two primary purposes: training the LLM to follow instructions and training it on domain-specific data relevant to the deploying entity. [1:40]

## Examples of Instruction Data

Instruction fine-tuning relies on datasets where input-output pairs explicitly demonstrate how to follow instructions. Examples include:

- Instruction: "Convert 45 km to meters." Output: "45 km is 45,000 M."
- Instruction: "Provide a synonym for bright." Output: "A synonym for bright is radiant."
- Instruction: "Edit the following sentence to remove all passive voice: The song was composed by the artist." Output: "The artist composed the song." ``

Training an LLM on such a dataset, where explicit input-output pairs are provided, is known as supervised instruction fine-tuning. The term "supervised" indicates that the model is given a set of instructions along with their expected responses. [1:40]

## Workflow for Building a Personal Assistant

The process of building a personal assistant involves several sequential steps:

1. Load the training dataset, which contains instruction-response pairs.
2. Batch the dataset and create data loaders.
3. Load the pre-trained LLM.
4. Fine-tune the LLM.
5. Inspect the loss.
6. Extract responses.
7. Perform evaluation.
8. Score the output. [1:40]

This chapter focuses on the first step: dataset download and formatting, which differs from previous data preparation methods. [1:40]

## Dataset Download and Formatting

The dataset used for instruction fine-tuning consists of 1100 instruction-response pairs. Each entry in the dataset is structured like a dictionary with three keys: `instruction`, `input`, and `output`. [5:58]

Examples from the dataset illustrate this structure:

- Instruction: "Edit the following sentence for grammar." Input: "He go to the park every day." Output: "He goes to the park every day."
- Instruction: "What are the first 10 square numbers?" Input: (None) Output: "1, 4, 9, 16, ..., 100."
- Instruction: "Translate the following sentence into French." Input: "Where is the nearest restaurant?" Output: (French translation). ``

Some entries, like "Convert 5 feet to meters," do not require a separate `input` field because the instruction itself contains all necessary information. In such cases, only the `instruction` and `output` are present. [5:58]

The dataset is loaded using a `download_and_load_file` function, which fetches the JSON data from a specified URL. After execution, the function confirms 1100 entries, indicating a successful download. [5:58]

To verify the loaded data, specific entries can be accessed by index. For example, `data_list[50]` reveals an instruction to identify the correct spelling of "occasion," with the output "occasion." Similarly, `data_list[999]` shows an instruction for the antonym of "complicated," with "simple" as the output and no input field. [5:58]

## Prompt Formatting: Alpaca vs. Phi-3

For effective instruction fine-tuning, the `instruction`, `input`, and `output` data must be formatted into a specific prompt structure that the LLM can understand during training. Researchers have found that certain formats yield better results. [10:58]

Two common prompt formats are the Stanford Alpaca style and the Microsoft Phi-3 style. The Alpaca format is widely adopted, as evidenced by its popularity in repositories. [10:58]

The Alpaca prompt structure begins with a standard introductory sentence: "Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request." This is followed by the specific `instruction`, `input`, and `response` (which is the output from the dataset). [10:58]

For example, an entry with `instruction`: "Identify the correct spelling of the following word", `input`: "occasion", and `output`: "occasion" would be formatted as:

```
Below is an instruction that describes a task, paired with an input that provides further context

### Instruction:
```

Identify the correct spelling of the following word

```
### Input:  
occasion
```

```
### Response:  
occasion
```

[10:58]

The Microsoft Phi-3 prompt style uses a "User" and "Assistant" format. The "User" section directly combines the instruction and input, while the "Assistant" section provides the output. For the same example, it would look like:

```
### User:  
Identify the correct spelling of the following word occasion  
  
### Assistant:  
occasion
```

The key difference is that Alpaca separates the instruction and input, whereas Phi-3 fuses them within the user prompt. While both are viable, the Alpaca style is more commonly used and will be implemented in this chapter. [10:58]

## Implementing the `format_input` Function

A function named `format_input` is defined to convert each dataset entry into the Alpaca prompt format. This function takes an entry (a dictionary containing `instruction`, `input`, and `output`) and constructs the prompt. [17:28]

The `format_input` function first constructs the instruction text by accessing the `instruction` key from the entry dictionary. It then constructs the input text by accessing the `input` key. If an input is not present in the entry, the input text is left blank, as per the Alpaca repository guidelines. [18:01]

The function combines these parts into a `model_input` string. For an entry with an input, the `model_input` will include the instruction and the input. If the input is absent, the `model_input` will only contain the instruction. [18:01]

To complete the full training example, the `desired_response` (the output from the entry) is appended to the `model_input`. This combined string, representing the full prompt and its expected response, is then fed to the LLM during fine-tuning. [18:01]

Testing with `data_list[50]` (which has an input) and `data_list[999]` (which lacks an input) demonstrates that the `format_input` function correctly handles both scenarios, producing the appropriate Alpaca-style prompt and response. [18:01]



## Data Splitting for Training, Testing, and Validation

After formatting the instructions into the Alpaca style, the 1100 instruction-response pairs are split into training, testing, and validation sets. The proposed split is 85% for training, 10% for testing, and 5% for validation. [22:00]

This results in:

- Training data: 935 pairs
- Validation data: 55 pairs
- Testing data: 110 pairs ``

These proportions are not fixed and can be adjusted for experimentation. Similarly, the prompt format can be changed to the Microsoft Phi-3 style or other custom formats. The field of LLM engineering is rapidly evolving, encouraging exploration and customization of these parameters. [22:00]

## Next Steps

The next lecture will focus on batching the dataset. This involves ensuring uniform input lengths, converting instructions into token IDs, and padding sequences with tokens?specific steps crucial for preparing data for the LLM training process. [23:20]

---

\*End of Chapter 37. Source: <https://www.youtube.com/watch?v=1bLhvqZzdaQ>\*

---

## Chapter 38: Data Batching in LLM Instruction Fine-tuning

---

*Source video: Data Batching in LLM instruction fine-tuning | Hands on project | Live Python coding*

*Duration: 52 minutes | Playlist position 38*

### Recap: Instruction Fine-tuning and Data Preparation

This lecture continues the hands-on instruction fine-tuning example from the previous session, focusing on organizing data into training batches. [\[0:05\]](#)

Previously, we established that pre-trained Large Language Models (LLMs) excel at text completion but struggle with following specific instructions, such as fixing grammar or converting text to passive voice. Instruction fine-tuning addresses this by providing the LLM with a dataset of instruction-response pairs. [\[0:05\]](#)

The dataset used for this project consists of 1100 instruction and output pairs. Examples include translating a sentence into French, rewriting a sentence in active voice, or answering factual questions like "What is the capital of Indonesia?". Some pairs include an explicit "input" field, while others embed the context directly within the instruction. [\[0:05\]](#)

The first step in the previous lecture was to download and format this dataset. Formatting involved converting the raw instruction, input, and output fields into a specific prompt structure known as the Alpaca prompt style. This style, maintained by the Stanford Alpaca repository, provides a template for constructing prompts from instruction-response pairs. [\[0:05\]](#)

The Alpaca prompt style follows this structure:

Below is an instruction that describes a task, paired with an input that provides further context

```
### Instruction:
{instruction_from_dataset}

### Input:
{input_from_dataset}

### Response:
{output_from_dataset}
```

This template is applied to each instruction, input, and output triplet from the dataset. [\[0:05\]](#)

After formatting, the dataset was partitioned into training, testing, and validation sets. An 85% split was used for training, 10% for testing, and the remaining 5% for validation. [\[0:05\]](#)

The code from the previous lecture first downloaded the 1100-entry dataset. A `format_input` function was then defined to convert each entry (instruction, input, output) into the Alpaca prompt format. This function constructs the initial prompt by combining the instruction and input, to which the desired response (output) is later appended. [0:05]

The data splitting resulted in 935 training pairs, 55 validation pairs, and 110 test pairs out of the total 1100. [5:28]

## Batching the Dataset: A Five-Step Workflow

The next crucial step is batching the dataset, which is more complex than previous data preparation steps. Batching involves converting individual data samples (formatted Alpaca prompts) into a numerical representation where all samples within a batch have a uniform size. [5:28]

Consider a batch containing multiple data samples, each represented by a prompt. To process these efficiently, they must be converted into numerical representations (e.g., rows in a matrix) where each row has the same number of columns. The challenge arises because the length of different prompts can vary significantly. [5:28]

The process of batching data for LLM fine-tuning follows a sequential five-step workflow:

1. Format the data using the Alpaca prompt template.
  2. Tokenize the formatted data into token IDs.
  3. Adjust the length of token ID sequences by padding them to a uniform length within each batch.
  4. Create target token IDs by shifting the input token IDs to the right by one.
  5. Replace specific padding tokens in the target token IDs with a placeholder value of -100.
- [5:28]

### #### Step 1 & 2: Formatting and Tokenization

The first step, as covered in the recap, is to format the raw instruction, input, and output into the Alpaca prompt style. [5:28]

The second step is to tokenize this formatted prompt. Tokenization converts the text into a sequence of numerical token IDs. For this, a Byte Pair Encoder (BPE) tokenizer, specifically the `tiktoken` library (used by OpenAI), is employed. Each prompt is transformed into a list of token IDs. [5:28]

A key issue arises here: the number of token IDs will differ for prompts of varying lengths. For instance, a longer prompt will result in more token IDs than a shorter one. However, for

batch processing, all samples within a batch must have the same number of token IDs (i.e., the same number of columns in their numerical representation). [5:28]

To implement these steps, an `InstructionDataset` class is defined. This class takes the raw dataset, applies the `format_input` function to create the Alpaca-style prompt, appends the response, and then tokenizes the full text using the provided tokenizer. [25:50]

The `InstructionDataset` class essentially returns a list of token IDs for every instruction/input/output pair in the dataset. The `end_of_text` token, which signifies the end of a text sample, has a token ID of 50256 in GPT-2's vocabulary. [25:50]

#### #### Step 3: Padding Token IDs to Uniform Length

The third step addresses the variable length of token ID sequences. Within each batch, all sequences are padded with a special token so that they all match the length of the longest sequence in that batch. [5:28]

The padding token used is 50256, which corresponds to the `end_of_text` token in GPT-2's vocabulary. GPT-2 has a vocabulary size of 50257, with token IDs ranging from 0 to 50256. Using 50256 for padding ensures that the added tokens do not carry any semantic meaning that could confuse the model during training. [5:28]

This padding is implemented in a custom `collate` function, which processes a batch of tokenized inputs. The function identifies the longest sequence in the batch and then appends 50256 tokens to all shorter sequences until they reach the maximum length. Finally, it converts the list of padded sequences into a PyTorch tensor. [25:50]

An intermediate version of this function, `custom_collate_draft_1`, first adds an extra 50256 token to `*all*` sequences in the batch (even the longest one) before padding. This temporary addition simplifies the creation of target token IDs in the next step. This extra token is later removed. [25:50]

For example, if a batch contains sequences of lengths 5, 2, and 3:

- The longest sequence (length 5) remains as is.
- The sequence of length 2 is padded with three 50256 tokens.
- The sequence of length 3 is padded with two 50256 tokens.

After padding, all sequences in the batch will have a length of 5. [25:50]

#### #### Step 4: Creating Target Token IDs

The fourth step involves creating target token IDs, which represent the "true" output the model should predict. For next-token prediction tasks, the target sequence is constructed by shifting the input sequence to the right by one position. [13:50]

If an input sequence is [0, 1, 2, 3, 4], the corresponding target sequence will be [1, 2, 3, 4, 50256]. The first element of the input (0) is dropped, and an end\_of\_text token (50256) is appended to maintain the same length. This means the model is trained to predict 1 given 0, 2 given 0, 1, and so on, up to predicting 50256 given 0, 1, 2, 3, 4. [17:22]

This approach, though counter-intuitive for instruction fine-tuning, works because the LLM learns to predict the next token in the sequence. By training on the full Alpaca prompt (instruction + input + response) with this shifted target, the model implicitly learns that after the instruction and input, it should generate the response. This is similar to how LLMs are pre-trained. [17:22]

The custom\_collate\_draft\_2 function extends the previous collate function to also generate these target tensors. It takes the padded input tensor, slices it to remove the first element (inputs[:, 1:]), and then appends the 50256 token to the end of each sequence to form the target tensor. The earlier step of adding an extra 50256 to all inputs simplifies this slicing operation. [32:00]

For example, given the padded inputs:

```
[[0, 1, 2, 3, 4],  
 [5, 6, 50256, 50256, 50256],  
 [7, 8, 9, 50256, 50256]]
```

The corresponding targets would be:

```
[[1, 2, 3, 4, 50256],  
 [6, 50256, 50256, 50256, 50256],  
 [8, 9, 50256, 50256, 50256]] [32:00]
```

#### ##### Step 5: Replacing Padding Tokens with a Placeholder

The final step involves replacing most of the 50256 padding tokens in the target tensor with a special placeholder value: -100. This is done for all 50256 tokens *\*except\** the very first one in each target sequence. [21:00]

The reason for using -100 is its special significance in PyTorch's CrossEntropyLoss function. By default, CrossEntropyLoss has an ignore\_index parameter set to -100. Any target label with this value will be ignored during the loss calculation, meaning it will not contribute to the gradient updates. [21:00]

This exclusion is beneficial because the padding tokens (50256) are artificially added to ensure uniform sequence length and do not carry meaningful information for training the

model's core task. Excluding them from the loss calculation makes training more efficient and prevents the model from learning to predict these irrelevant tokens. [21:00]

The first 50256 token in each target sequence is retained because it represents the actual `end_of_text` signal that the LLM should learn to generate to indicate the completion of its response. [21:00]

The final `custom_collate` function incorporates this logic. It first identifies all indices in the target tensor where 50256 is present. Then, for each sequence, it skips the first occurrence of 50256 and replaces all subsequent occurrences with -100. [36:00]

Using the previous example targets:

```
[[1, 2, 3, 4, 50256],  
 [6, 50256, 50256, 50256, 50256],  
 [8, 9, 50256, 50256, 50256]]
```

The modified targets after applying the -100 placeholder would be:

```
[[1, 2, 3, 4, 50256],  
 [6, 50256, -100, -100, -100],  
 [8, 9, 50256, -100, -100]] [38:56]
```

A demonstration with `CrossEntropyLoss` confirms this behavior. If a target vector includes -100, the loss calculation effectively ignores the corresponding prediction, resulting in the same loss as if that entry were never present. [41:00]

## Code Implementation Details

The `InstructionDataset` class is responsible for preparing individual data samples:

```
import torch  
from torch.utils.data import Dataset  
import tiktoken # Assuming tiktoken is the tokenizer  
  
class InstructionDataset(Dataset):  
    def __init__(self, dataset, tokenizer):  
        self.data = []  
        self.tokenizer = tokenizer  
        # GPT-2 end of text token ID  
        self.end_of_text_token_id = 50256  
  
        for entry in dataset:  
            # Step 1: Format data using Alpaca prompt template  
            # format_input function creates the instruction + input part  
            formatted_prompt_prefix = self.format_input(entry)
```

```

        # Append the response to complete the full text
        full_text = formatted_prompt_prefix + entry["output"]

        # Step 2: Tokenize the formatted data
        token_ids = self.tokenizer.encode(full_text)
        self.data.append(token_ids)

def format_input(self, entry):
    # This function is a simplified representation of the actual format_input
    # as described in the previous lecture.
    # It constructs the instruction and input part of the Alpaca prompt.
    if entry["input"]:
        return (
            "Below is an instruction that describes a task, paired with an input that provides context. "
            "Write a response that appropriately completes the request.\n\n"
            f"### Instruction:\n{entry['instruction']}\n\n"
            f"### Input:\n{entry['input']}\n\n"
            f"### Response:\n"
        )
    else:
        return (
            "Below is an instruction that describes a task. "
            "Write a response that appropriately completes the request.\n\n"
            f"### Instruction:\n{entry['instruction']}\n\n"
            f"### Response:\n"
        )

def __len__(self):
    return len(self.data)

def __getitem__(self, idx):
    return self.data[idx]

# Example tokenizer initialization (using GPT-2 BPE)
# tokenizer = tiktoken.get_encoding("gpt2")

```

[25:50]

The custom\_collate function (final version) handles the batching logic:

```

def custom_collate(batch, padding_token_id, ignore_index, device="cpu"):
    # Find the longest sequence in the batch
    max_len = max(len(item) for item in batch)

    # Step 3: Pad token IDs to uniform length
    # Also add an extra padding_token_id to each sequence for target creation
    padded_inputs = []
    for item in batch:
        # Add an extra padding token at the end for target creation ease
        temp_item = item + [padding_token_id]
        # Pad to max_len + 1 (because we added one extra token)
        padded_item = temp_item + [padding_token_id] * (max_len + 1 - len(temp_item))
        padded_inputs.append(padded_item)

```

```

# Convert to tensor
inputs_tensor = torch.tensor(padded_inputs, dtype=torch.long, device=device)

# Step 4: Create target token IDs (shifted input)
# Target is input shifted right by one.
# The extra padding_token_id added earlier makes this slice work directly.
targets_tensor = inputs_tensor[:, 1:].clone() # Use .clone() to avoid modifying inputs_tensor

# Step 5: Replace padding tokens with -100 placeholder in targets
# Create a mask for all padding_token_id occurrences in targets
padding_mask = (targets_tensor == padding_token_id)

# Iterate through each sequence in the batch to handle the first padding token
for i in range(targets_tensor.shape[0]):
    # Find the index of the first padding token in the current target sequence
    first_padding_idx = (targets_tensor[i] == padding_token_id).nonzero(as_tuple=True)[0]
    if first_padding_idx.numel() > 0: # If padding token exists
        # Keep the first padding token, mask out subsequent ones
        padding_mask[i, first_padding_idx[0] + 1:] = True # Mask from the second padding token

# Apply the ignore_index to the masked positions
targets_tensor[padding_mask] = ignore_index

return inputs_tensor[:, :-1], targets_tensor # Remove the extra padding from inputs_tensor

```

[36:00]

## Masking Instruction Tokens in Targets (Open Research)

Beyond masking padding tokens, some researchers also mask out the target token IDs that correspond to the instruction and input parts of the prompt, leaving only the response to contribute to the loss. This is done to train the LLM to focus solely on generating accurate responses, rather than memorizing the instructions themselves, which could help reduce overfitting. [47:00]

However, there is ongoing debate in the research community about whether masking instructions is universally beneficial. A recent paper titled "Instruction Tuning with Loss over Instructions" demonstrated that *not* masking the instruction can actually benefit LLM performance. [47:00]

Given this open research question, this implementation does not apply instruction masking, leaving it as an optional exercise for further exploration. [47:00]

## Summary of Data Batching Workflow

The five detailed steps for organizing data into training batches are:

1. **Format Data:** Convert raw instruction, input, and output pairs into the Alpaca prompt template.



2. **Tokenize Data:** Transform the formatted prompts into sequences of numerical token IDs using a tokenizer like tiktoken.

3. **Pad Sequences:** Within each batch, pad all tokenized input sequences with the `end_of_text` token (50256) so they all have the same length as the longest sequence in that batch.

4. **Create Target IDs:** Generate target token IDs by shifting the input token ID tensor to the right by one position and appending an `end_of_text` token.

5. **Replace Padding Placeholders:** In the target token IDs, replace all 50256 padding tokens with -100, except for the very first 50256 in each sequence. This leverages PyTorch's `CrossEntropyLoss ignore_index` to exclude irrelevant padding from loss calculation.

[21:00]

This comprehensive batching process is fundamental for effective LLM instruction fine-tuning. [21:00]

## Next Steps

In subsequent lectures, we will proceed to create data loaders, load a pre-trained LLM, perform the instruction fine-tuning, inspect the loss and accuracy, generate responses, and evaluate their quality. The ultimate goal is to build a system that mimics a ChatGPT-like experience. [50:00]

---

\*End of Chapter 38. Source: [https://www.youtube.com/watch?v=bkUkcyL\\_Xxc](https://www.youtube.com/watch?v=bkUkcyL_Xxc)\*

---

## Chapter 39: Dataloaders in Instruction Fine-tuning

---

*Source video: Dataloaders in Instruction Fine-tuning*

*Duration: 24 minutes | Playlist position 39*

### Introduction to DataLoaders

This chapter focuses on creating data loaders, which provide an efficient way to collect and access batches of data for training large language models (LLMs). Data loaders wrap an iterable around a dataset, allowing batches to be accessed sequentially and efficiently. This efficiency is crucial when training large models with over 100 million parameters. [\[0:07\]](#)

### Recap of Data Preparation Steps

Before diving into data loaders, it's important to recap the preceding data preparation steps for instruction fine-tuning. The overall process involves three main stages: dataset download and formatting, batching the dataset, and creating data loaders. [\[0:07\]](#)

#### #### 1. Dataset Download and Formatting

The first step involves downloading and formatting the instruction fine-tuning dataset. This dataset consists of instruction-input-output pairs. For example, an instruction might be "Edit the following sentence for grammar," with an input "He go to the park every day," and an output "He goes to the park every day." Another example could be "Convert 45 kilometers to meters," with no input and an output "45 kilometers is 45,000 meters." [\[0:07\]](#)

The purpose of this fine-tuning is to teach a pre-trained LLM to follow instructions effectively. The raw data is formatted using a specific template, such as the Alpaca format, which is derived from the Stanford Alpaca GitHub repository. This template converts instruction-response pairs into a structured prompt that serves as input to the LLM. [\[1:40\]](#)

An example of the Alpaca prompt structure is: "Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request." This is followed by the specific instruction, input (if any), and the expected response. This entire prompt is then fed to the LLM for next-token prediction. [\[1:40\]](#)

#### #### 2. Batching the Dataset

Batching the dataset involves several substeps to prepare the formatted data for model training. These steps ensure that data within each batch is consistent and suitable for efficient processing. [\[3:50\]](#)

#### Substep 1: Format Data using Prompt Template

As mentioned, each instruction, input, and response is formatted into an Alpaca-style prompt. Other templates, like the Vicuna template used by Microsoft, can also be explored. [\[4:10\]](#)

### **Substep 2: Tokenize Formatted Data**

The formatted prompts are converted into token IDs. If a batch contains multiple instruction-input-output pairs, each prompt is tokenized into a sequence of numerical IDs. [\[4:30\]](#)

### **Substep 3: Pad Token IDs**

Since tokenized prompts naturally have varying lengths, padding is applied within each batch to ensure all data samples have the same number of token IDs. The longest sequence in a batch determines the target length. Shorter sequences are padded with the `end_of_text_token`, which has an ID of 50256, until they match the maximum length. [\[4:50\]](#)

### **Substep 4: Create Target Pairs**

For next-token prediction, target pairs are created by shifting the input sequence to the right by one token. This means the target for a given input token is the next token in the sequence. An `end_of_text_token` is appended to the target sequence. This approach teaches the LLM to predict the subsequent token, allowing it to learn how to generate responses given an instruction and input. [\[6:00\]](#)

### **Substep 5: Replace Padding Tokens with -100**

All 50256 padding tokens, except for one `end_of_text_token` at the end of the actual content, are replaced with -100. This is because in PyTorch's `CrossEntropyLoss` function, -100 serves as the `ignore_index`. This ensures that these padding tokens do not contribute to the loss calculation, preventing them from influencing the model's learning. [\[7:30\]](#)

## **Creating DataLoaders**

With the data preparation and batching steps understood, the next stage is to create data loaders for the training, testing, and validation datasets. [\[9:40\]](#)

#### The custom\_collate Function

The core logic for batching, including tokenization, padding, and replacing padding tokens, is encapsulated within a `custom_collate` function. This function takes a batch of raw data and transforms it into the input and target tensors required for training. [\[10:20\]](#)

#### Device Transfer

A crucial optimization is to include code within the `custom_collate` function to move the input and target tensors to the target device (CPU, CUDA for GPUs, or MPS for Apple Silicon chips). Performing this device transfer as a background process, outside the main training loop, prevents it from blocking the GPU during model training, thereby improving efficiency. [11:50]

The device is typically set based on availability:

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# For Apple Silicon Macs, uncomment the following:
# if torch.backends.mps.is_available():
#     device = torch.device("mps")
print(f"Using device: {device}")
```

[14:00]

#### Using `functools.partial`

To simplify the `custom_collate` function's usage, the `functools.partial` function from Python's standard library is used. This allows pre-filling arguments like `device` and `allowed_maximum_length` (context length) so they don't need to be passed explicitly each time the `custom_collate` function is called. The `allowed_maximum_length` defines the maximum number of tokens the model will consider at one time, for example, 1024. [15:00]

#### Instantiating `InstructionDataset`

An `InstructionDataset` class is used to take the raw training data and convert it into token IDs. An instance of this class is created for the training, validation, and test datasets. [16:40]

#### Creating `DataLoader` Instances

Finally, `DataLoader` instances are created for the training, validation, and test sets. The `DataLoader` takes the dataset instance, the `batch_size` (e.g., 8), and the `collate_fn` (which is the `custom_collate` function) as arguments. [17:50]

The data is typically split into 85% for training, 10% for testing, and 5% for validation. This means the `train_loader` will contain the majority of the batches, while `val_loader` and `test_loader` will have fewer. [18:50]

## Understanding `DataLoader` Output

When inspecting the output of a `DataLoader`, such as the `train_loader`, it yields batches of input and target tensors. Each batch consists of two tensors: one for inputs and one for targets.

[20:00]

For example, a batch might have a shape of (8, 61) for both the input tensor and the target tensor. Here, 8 represents the `batch_size`, and 61 is the number of tokens in each training example within that specific batch. [20:40]

It's important to note that the number of tokens (the second dimension) can vary between batches. For instance, the first batch might be (8, 61), while the second batch could be (8, 76). This dynamic length occurs because the `custom_collate` function calculates the maximum token length for *each batch separately* and pads all samples within that batch to that specific maximum length. This ensures efficient processing while accommodating varying input lengths. [22:00]

## Conclusion

Creating data loaders marks the completion of the data preparation stage for instruction fine-tuning. Significant time is dedicated to batching the dataset and creating data loaders because correctly preparing the data is fundamental to the success of any machine learning project, especially with LLMs. A well-prepared dataset simplifies the subsequent training, evaluation, and deployment phases. [22:50]

---

\*End of Chapter 39. Source: <https://www.youtube.com/watch?v=egFqsJQ9kuY>\*

---

## Chapter 40: Instruction Fine-tuning: Loading Pre-trained LLM Weights

---

*Source video: Instruction fine-tuning: Loading pre-trained LLM weights*

*Duration: 19 minutes | Playlist position 40*

### Introduction to Loading Pre-trained LLM Weights

This chapter focuses on the fourth step in the instruction fine-tuning hands-on project: loading a pre-trained large language model (LLM). Previously, the project covered data set download and formatting, batching the data set, and creating data loaders. [\[0:05\]](#)

The data set used for this project consists of 1100 entries, each containing an instruction, input, and expected output. These entries span various fields, including English, geography, and general knowledge. The primary goal of fine-tuning with this data set is to teach the LLM to accurately follow instructions, as pre-trained LLMs typically do not perform well on instruction-following tasks without further training. [\[0:05\]](#)

Before loading the pre-trained LLM, significant effort was dedicated to preparing the data. This involved downloading the data set, converting it into the Alpaca prompt style, batching it into manageable chunks, and finally creating data loaders. With the data loaders now ready, the next logical step is to load a pre-trained LLM. [\[0:05\]](#)

### The Role of Pre-trained Weights

The LLM architecture constructed in this lecture series contains numerous trainable weights. These include the Query, Key, and Value (Q, K, V) weight matrices within the multi-head attention mechanism, the weights of neurons in the feed-forward neural network, and the scale and shift parameters in Layer Normalization 1 and 2. Additionally, the token embedding layer and positional embedding layer also have associated trainable weights. [\[0:05\]](#)

Cumulatively, these trainable parameters can exceed 100 million. During the pre-training process, these parameters are optimized on vast amounts of data. For instance, OpenAI pre-trained GPT-2 on an immense corpus of text and subsequently released these optimized weights publicly. This allows users to access and utilize these pre-trained weights. [\[0:05\]](#)

OpenAI released GPT-2 weights for several models, varying in parameter count, such as 124 million, 345 million, 774 million, and 1558 million. These weights are organized into different folders based on their parameter size and can be downloaded to a local machine. [\[0:05\]](#)

The primary reason for downloading and reusing these pre-trained weights before fine-tuning is to leverage the extensive information already captured within them. For example, in token

embedding, words or tokens are projected into a higher-dimensional vector space in a way that captures their semantic meaning. Training such a large neural network from scratch would demand significant time and computational resources. [0:05]

By directly using pre-trained GPT-2 weights, the model starts from a much more knowledgeable state rather than a random initialization. This significantly reduces the computational time required for fine-tuning on a specific data set, making the process much more efficient. The fine-tuning process always follows pre-training, where the model first achieves a knowledgeable state, and then its weights and parameters are further refined on a specific data set. [4:45]

## Selecting and Configuring the GPT-2 Model

For this instruction fine-tuning task, a larger GPT-2 model is necessary. While a 117 million parameter model was used in a previous spam classification project, a smaller 124 million parameter model was found to perform poorly for instruction fine-tuning. Therefore, the GPT-2 medium model with 355 million parameters will be loaded. [4:45]

This medium-sized model requires approximately 1.42 GB of storage space. The instructor notes that a MacBook Air 2020 is sufficient for this task, indicating that no high-end GPU or specialized hardware is strictly necessary, though more powerful machines can handle larger models. [4:45]

The configuration for the GPT-2 medium model includes a context size of 1024, 24 Transformer blocks, and 16 attention heads within each block. The base configuration specifies a vocabulary size of 50257, a context length of 1024, a dropout rate of 0, and the query, key, and value bias terms are set to True. The vocabulary size of 50257 is the one GPT-2 was originally trained on. [4:45]

## Downloading and Loading GPT-2 Weights

The process of downloading and loading the GPT-2 weights involves a custom function called `download_and_load_gpt2()`. This function, imported from the `gpt_download_3.py` file, accesses a specific URL to download seven files onto the local machine. These files can also be manually accessed and downloaded from Kaggle. [4:45]

Directly downloading these files is not sufficient; they require pre-processing. The `download_and_load_gpt2()` function internally calls `load_gpt2_parameters_from_tensorflow_checkpoint()`, which converts the downloaded parameters into a structured `params` dictionary. This dictionary makes the parameters, including those for Transformer blocks, token embeddings, positional embeddings, and the final normalization layer, easily accessible and integrable with the custom LLM code. [4:45]

To execute this step, the following line of code is run:

```
settings, params = download_and_load_gpt2()
```

This command initiates the download of the 1.42 GB model. A strong internet connection is recommended for a quicker download, as interruptions can prolong the process. [4:45]

After the weights are downloaded, they need to be loaded into the custom GPT architecture. This is achieved using the `load_weights_into_gpt()` function. This function maps the parameters from the `params` dictionary into the corresponding trainable parameter locations within the constructed GPT model. [4:45]

The `load_weights_into_gpt()` function effectively assigns the downloaded pre-trained parameters to the model. Following this, the model is set to evaluation mode. [4:45]

Users with less powerful hardware can opt for the GPT-2 Small model (124 million parameters) to reduce download and loading times. Conversely, those with high processing speeds and GPU access can choose GPT-2 Large or GPT-2 XL for potentially better results. If the files are already downloaded, the output will indicate "file already exists." [4:45]

## Pre-trained Model Performance Before Fine-tuning

Before proceeding with fine-tuning, it is beneficial to evaluate the pre-trained LLM's performance on a validation task. This provides a baseline for comparison. An example from the validation data set is used: [12:50]

**Instruction:** "Convert the active sentence to passive"

**Input:** "The chef cooks the meal every day"

**Expected Output:** "The meal is cooked by the chef every day" [12:50]

The `generate()` function is used to obtain a response from the pre-trained model. This function takes `max_new_tokens` (set to 35), `context_length` (1024), the `input_text`, and the `model` with its pre-trained weights. [12:50]

The `generate()` function typically returns the combined input and generated output. To isolate the model's response, the length of the input instruction is subtracted from the beginning of the generated text. [12:50]

The pre-trained model's response to the instruction was:

```
### Instruction:
Convert the active sentence to passive
The chef Cooks the meal every day
```

The model simply repeated the original input sentence and a portion of the instruction. It failed to convert the active sentence to passive voice. This demonstrates that without



fine-tuning, even a 355 million parameter GPT-2 model (or even larger ones like the 774 million parameter model) is not capable of correctly following instructions. [12:50]

This poor performance highlights the necessity of the fine-tuning process. The next section will implement fine-tuning to improve the model's ability to comprehend and appropriately respond to such requests by modifying its weights and parameters based on the instruction-input-output pairs from the specific data set. [12:50]

---

\*End of Chapter 40. Source: [https://www.youtube.com/watch?v=\\_\\_OiQznq4ao](https://www.youtube.com/watch?v=__OiQznq4ao)\*

---

## Chapter 41: LLM Fine-tuning Training Loop

---

*Source video: LLM fine-tuning training loop | Coded from scratch*

*Duration: 23 minutes | Playlist position 41*

### Introduction to Instruction Fine-tuning

This chapter focuses on fine-tuning the large language model (LLM) that has been developed over the past several lectures. The goal is to improve the model's ability to understand and follow instructions, as the pre-trained LLM is not yet proficient in this area. [0:05]

The instruction fine-tuning process is divided into three stages. Stage one, preparing the dataset, has already been completed. This involved downloading and formatting data using the Alpaca prompt format, dividing it into batches, and creating training, testing, and validation data loaders. [0:05]

Stage two, fine-tuning the LLM, is the current focus. The LLM architecture has been loaded with pre-trained weights from a GPT-2 355 million parameter model. The model is now ready for fine-tuning on an instruction dataset. [0:05]

This instruction dataset consists of 1100 input-output pairs. The objective is to train the LLM's weights using this data, enabling it to respond effectively to instructions. [0:05]

### Demonstrating the Need for Fine-tuning

Before fine-tuning, the pre-trained LLM struggles with instruction following. For example, when given the instruction "convert the active sentence to passive" and the sentence "The chef cooks the meal every day," the LLM simply regenerates the input: "The Chef Cooks the meal every day and instruction convert the active sentence to passive." It fails to convert the sentence to passive voice. [0:05]

The LLM's response section is created, but it merely repeats the original input sentence and part of the instruction, failing to perform the requested conversion. The goal of fine-tuning is to significantly improve the LLM's performance when it receives such instructions. [0:05]

### Trainable Parameters and the Role of Pre-training

The LLM architecture contains numerous trainable parameters. These include the token embedding weights, positional embedding weights, scale and shift parameters in layer normalization, query (Q), key (K), and value (V) weight matrices in multi-head attention, weights in the feed-forward neural network, and parameters in the final layer normalization and output layer. All these parameters will be optimized during fine-tuning using the instruction dataset. [0:05]

A natural question arises: why pre-train the LLM if it will be trained again on a specific dataset? The reason is that pre-training allows the model to start from a knowledgeable state rather than a random one. Starting from a random state would make it extremely difficult for the model to learn everything from a limited instruction dataset. [0:05]

Through pre-training, the model gains foundational knowledge, such as the semantic meaning of words and general language structure. While it may not follow instructions, it understands the relationships between words and the language itself. This pre-training establishes a strong knowledge base upon which fine-tuning can then build, making the fine-tuning process more efficient and effective. [0:05]

## The Training Loop and Loss Function

Before proceeding with fine-tuning, it is essential to understand the training loop and the loss function. Both are identical to those used during the pre-training phase. [0:05]

The training loop involves iterating through each batch in the dataset. For each batch, the loss is calculated, gradients are computed, and then the optimizer updates the model parameters. Specifically, in a new epoch, loss gradients are reset, the loss on the current batch is calculated, and a backward pass is performed. [0:05]

During the backward pass, the partial derivative of the loss with respect to all 355 million parameters is calculated. The update step then uses a gradient descent-based optimizer, which follows the principle:  $w_{\text{new}} = w_{\text{old}} - \eta * \partial \text{Loss} / \partial w$ . This process is repeated many times, aiming to minimize the loss function and reach a state where parameters are optimized. [0:05]

The loss function is derived from input and target pairs within each batch. For a next-token prediction task, the target is typically the input sequence shifted to the right by one token, with an end-of-text token appended. This target represents the true value. [0:05]

The predicted value is obtained from the model's `generate()` function. When an input sequence passes through the GPT architecture, it produces a logits tensor, which is then converted into a tensor of probabilities. These probabilities yield the predicted output of the LLM. [0:05]

The loss function used is the cross-entropy loss, also known as categorical cross-entropy loss. Conceptually, cross-entropy loss aims to maximize the probability of the true token, driving it as close to one as possible, thereby minimizing the loss towards zero. This function quantifies the difference between the true and predicted probability distributions. [0:05]

## Reusing Pre-training Functions for Fine-tuning

The hard work of dataset processing, batching, and creating data loaders from previous lectures can now be leveraged. The loss calculation and training functions implemented during pre-training are directly reusable for fine-tuning. [0:05]

Functions such as `calculate_loss_on_batch` compute the cross-entropy loss for a single batch of data. To calculate the loss over an entire data loader, `calculate_loss_in_loader` sums the losses from individual batches and normalizes by the number of batches. These functions can be used for training, testing, or validation data loaders. [0:05]

The core training loop, which includes the backward pass for gradient calculation and the optimizer step, is also reused. While a simple vanilla gradient descent was previously discussed, the actual optimizer employed is AdamW (Adam with weight decay). [10:39]

The remaining code handles printing losses at different iterations and visualizing training, validation, and testing losses. [10:39]

## Initial Loss and Training Setup

Before starting the fine-tuning process, the initial training and validation losses are calculated to establish a baseline. At this stage, the training loop has not yet run on the instruction dataset. [10:39]

The initial training loss is 3.82, and the validation loss is 3.76. These values are high, as expected, since the model has not yet been trained on the specific instruction data. The goal is to significantly reduce both these losses. [10:39]

The training process is set up by initializing the optimizer, defining the number of epochs, and specifying the evaluation frequency. The optimizer used is `torch.optim.AdamW`, a widely adopted optimizer in machine learning that tracks gradients and squared gradients to prevent local minima and accelerate convergence. [10:39]

The hyperparameters for the optimizer are set as `learning_rate = 0.005` and `weight_decay = 0.1`. These are tunable parameters, and experimenting with different values can yield better results. [10:39]

The `train_model_simple` function is defined to orchestrate the training. It takes the model, training loader, validation loader, optimizer, device (CPU in this case), number of epochs (set to 1), evaluation frequency (5 batches), and evaluation iterations (5 batches) as arguments. The `evaluation_frequency` determines how often results are printed, while `evaluation_iterations` specifies how often the evaluation loss is calculated. [10:39]

A `start_context` is provided to evaluate the generated responses during training. This context is the first sample from the validation data, which is the instruction "convert the active sentence to passive: The chef cooks the meal every day." This allows for monitoring how the

fine-tuned LLM responds to this specific example. [10:39]

## Hardware Considerations and Training Results

The training was performed on a MacBook Air 2020 with 8GB of RAM and a CPU, without a dedicated GPU. Due to these limitations, the number of epochs was set to 1, as running for 2 epochs would take 4-5 hours and crash the system. Even with one epoch, the training process took 2 hours. [10:39]

It is highly recommended to use a GPU for training, such as those available through Google Colab or AWS EC2 instances, to significantly reduce training time and allow for more epochs. However, the code is designed to run even on basic CPU configurations. [10:39]

After one epoch of training, both the training and validation losses decreased substantially. The training loss decreased, and the validation loss also showed a sufficient reduction. [10:39]

The fine-tuned LLM's response to the `start_context` instruction ("convert the active sentence to passive: The chef cooks the meal every day") was: "The meal is prepared every day by the chef." This is a significant improvement over the pre-trained model, which merely repeated the input. The response is almost correct, with "prepared" instead of the more accurate "cooked." [10:39]

Further testing on a system with better configuration showed that increasing the number of epochs from 1 to 2 resulted in the model producing the exactly correct response: "The meal is cooked by the chef every day." This demonstrates that even with limited resources and a single epoch, fine-tuning yields substantial improvements, and more compute power can lead to even better results. [10:39]

## Loss Plot Analysis and Future Evaluation

The plot of training and validation losses over one epoch shows a rapid decrease during the initial phase, indicating that the model quickly learns meaningful patterns and representations from the data. As training proceeds, the losses continue to decrease, albeit at a slower rate, suggesting that the model is fine-tuning its learned representations and converging towards a stable solution. [10:39]

While the validation loss is still slightly higher than the training loss, and both have the potential to decrease further with more epochs, the current results are promising given the computational constraints. [10:39]

Another example demonstrates the model's improved instruction following: when asked to "rewrite the sentence using a simile: The car is very fast," the model predicts "The car is as fast as a bullet," which is an excellent response, close to the correct "The car is as fast as

lightning." [10:39]

Although the loss plot indicates effective training, the most crucial aspect is the quality and correctness of the model's responses. Evaluating LLM responses, especially qualitative ones, is a complex field of active research. A dedicated lecture will cover methods for evaluating LLMs beyond just quantitative loss values. [10:39]

This lecture successfully demonstrated that without fine-tuning, the pre-trained model cannot follow instructions. However, by using an instruction fine-tuning dataset, the model learns to follow instructions effectively. This marks the completion of Stage 2: Fine-tuning the LLM. The next lecture will move to Stage 3: Evaluating the LLM. [10:39]

---

\*End of Chapter 41. Source: <https://www.youtube.com/watch?v=r7unILsP0Es>\*

---

## Chapter 42: Evaluating Fine-Tuned LLMs Using Ollama

---

*Source video: Evaluating fine-tuned LLM using Ollama*

*Duration: 52 minutes | Playlist position 42*

### Recap of Instruction Fine-Tuning

This lecture concludes the instruction fine-tuning project. In the previous session, a large language model (LLM) was successfully fine-tuned on an instruction input and output dataset, demonstrating improved responses to instructions. [\[0:05\]](#)

The fine-tuning process began with Stage 1: data preparation. This involved downloading, batching, and creating data loaders for the dataset. The dataset consists of 1,100 instruction input and output pairs, designed to train the LLM to follow instructions more effectively. [\[0:05\]](#)

Batching the dataset was a crucial step, ensuring that all input samples within each batch had a consistent number of token IDs. Following batching, the data was split into training, testing, and validation sets, with 85% for training, 10% for testing, and 5% for validation, specifically using the instruction input and output pairs. [\[0:05\]](#)

Stage 2 involved loading a pre-trained LLM, specifically GPT-2 weights, to establish a strong foundational base. This base model was then fine-tuned using the instruction input/output dataset. The training and validation loss function from the previous lecture illustrated the model's learning progression. [\[0:05\]](#)

Due to CPU memory and compute limitations, the fine-tuning was executed for only one epoch. It was noted that running for two or more epochs on a GPU or a more powerful PC, such as a MacBook M3, would likely yield superior results. The GPT-2 model used has 355 million parameters, underscoring

---

## Chapter 43: Build LLMs from Scratch: A 20-Minute Summary

---

*Source video: Build LLMs from scratch 20 minutes summary*

*Duration: 19 minutes | Playlist position 43*

### Introduction to the LLM Series Recap

This video serves as a concluding summary for the "Build Large Language Models from Scratch" series. The primary goal of the series was to teach fundamental research and engineering principles in machine learning, emphasizing the "nuts and bolts" of how large language models (LLMs) are truly built, rather than just deploying existing applications.

[\[0:06\]](#)

The instructor drew inspiration from Sebastian Raschka's book, "Build a Large Language Model," to structure the course. Throughout the series, a single, comprehensive code file was developed and explained in detail, covering every code block from scratch without assuming prior knowledge. This code file was shared with students to enable them to run and construct the entire LLM independently. [\[0:06\]](#)

The series followed a three-stage workflow for building an LLM:

1. **Stage 1: Foundation** involved data preparation and sampling, understanding the attention mechanism, and defining the overall LLM architecture.
2. **Stage 2: Pre-training** focused on implementing the loss function, performing the backward pass, and training the millions or billions of parameters. It also covered loading pre-trained weights from models like OpenAI's GPT-2 to accelerate the process.
3. **Stage 3: Fine-tuning** addressed adapting the pre-trained LLM for specific tasks, such as classification or creating a personalized chatbot. This stage included two hands-on projects: an LLM classifier for spam detection and a personal assistant capable of following instructions. [\[0:06\]](#)

### Key Components of LLM Construction

The journey of building an LLM begins with the data pre-processing pipeline, which is distinct from those used in regression or classification models. For an LLM, the objective is to predict the next token in a sequence. [\[0:06\]](#)

The pipeline transforms raw input text through several steps:

1. **Tokenization:** Input text is converted into individual tokens.
2. **Token IDs:** Tokens are mapped to numerical token IDs.



3. **Token Embeddings:** Token IDs are projected into a higher-dimensional vector space to capture their semantic meaning. For example, in GPT-2, the embedding dimension is 768.

4. **Positional Embeddings:** These vectors are added to the token embeddings. Positional embeddings are crucial because the position of each token in a sequence is vital for predicting the next token.

5. **Input Embeddings:** The sum of token embeddings and positional embeddings forms the final input embeddings, which are the expected output of the data pre-processing pipeline.

[\[0:06\]](#)

#### #### The Attention Mechanism

The attention mechanism is the "driving engine" that empowers LLMs. While simple vector embeddings capture the semantic meaning of individual tokens, they do not inherently understand the relationships between tokens. Attention addresses this by determining how much importance one token should give to all other tokens in a sequence when predicting the next one. [\[0:06\]](#)

The core idea of attention is to convert input embedding vectors into richer context vectors. This process involves several steps:

1. **Query, Key, Value Matrices:** Input embeddings are multiplied by trainable Query (Q), Key (K), and Value (V) matrices to produce query, key, and value representations.

2. **Attention Scores:** Queries are multiplied by the transpose of the Keys ( $Q * K^T$ ) to compute attention scores.

3. **Scaling:** The attention scores are scaled by the square root of the key dimension ( $\sqrt{d_k}$ ).

4. **Dropout:** Dropout is applied for regularization.

5. **Causal Attention:** A mask is added to prevent tokens from attending to future tokens, which is essential for next-token prediction tasks.

6. **Softmax:** A softmax layer converts the scaled and masked attention scores into attention weights.

7. **Context Vectors:** These attention weights are then multiplied by the Values (V) to produce the context vector matrix for a single attention head. [\[0:06\]](#)

Large language models utilize multiple attention heads to capture diverse dependencies and long-range relationships within text. The context vector matrices from these multiple heads are combined to form the final context vector matrix. This transformation from an input embedding matrix to a context vector matrix represents a fundamental revolution in large

language models. [0:06]

#### #### LLM Architecture: The Transformer Block

The overall LLM architecture, often based on the Transformer, processes the input embeddings. After token and positional embedding layers, and a dropout layer, the input embeddings are fed into a "Transformer block," where the core processing occurs. [0:06]

A typical Transformer block includes:

- A normalization layer.
- A Masked Multi-Head Attention module, which encapsulates the entire attention mechanism (Q, K, V, scaling, dropout, causal attention, softmax).
- A dropout layer.
- A shortcut connection (residual connection).
- Another normalization layer.
- A feed-forward neural network.
- Another dropout layer.
- One more shortcut connection. ``

After exiting the Transformer block, there is a final layer normalization layer and a neural network that converts the Transformer outputs into a logits tensor. This logits tensor is then used to predict the next token in the sequence. Models like GPT-2 typically stack 12 such Transformer blocks, and larger LLMs use even more. Each Transformer block itself can contain multiple attention heads (e.g., 12 or 24). The entire architecture, including each sub-component, was coded from scratch in the lecture series. [0:06]

### LLM Pre-training and Fine-tuning

The pre-training loop involves two main phases: the forward pass and the backward pass. In the forward pass, an input sequence is fed into the LLM, and the model generates an output. The loss function, typically cross-entropy loss, is then calculated by comparing the predicted next token with the actual next token. [0:06]

The backward pass involves calculating the partial derivative of the loss with respect to all trainable parameters in the LLM architecture. These trainable parameters are numerous and include:

- Token embedding parameters.
- Positional embedding parameters.
- Scale and shift parameters within layer normalization layers.
- The Query, Key, and Value trainable weight matrices in the masked multi-head attention modules.

- Parameters in the feed-forward neural networks.
- Parameters in the final output layer and layer normalization. ``

Given that models like GPT-2 have 12 Transformer blocks, each with its own set of these parameters, the total number of trainable parameters can easily exceed a million or even a billion. After computing these gradients, a gradient update is performed, typically using an optimizer like Adam or AdamW, to adjust the weights:  $W_{t+1} = W_t - \eta * \nabla W_t / \text{Loss}$ . [0:06]

While full-scale LLM pre-training (like for GPT-2, GPT-3, GPT-4) requires massive datasets and computational resources, costing millions of dollars, the series demonstrated how to run a pre-training loop on a smaller dataset on a personal laptop. This provides a complete understanding of the underlying process. After pre-training, the developed architecture was used to load pre-trained weights from GPT-2 and predict the next token based on an input sentence, marking a fundamental achievement in the series. [0:06]

Pre-training alone is often insufficient for specific tasks, leading to the need for fine-tuning. The series covered two types of LLM fine-tuning:

1. **Classification Fine-tuning:** An email classification LLM was built from scratch to distinguish between spam and non-spam emails.
2. **Instruction Fine-tuning:** A personal assistant LLM was developed to follow instructions, such as converting an active sentence to a passive one (e.g., "The chef cooks the meal every day" to "The meal is cooked every day by the chef"). All fine-tuning projects were implemented using the custom-built architecture. [0:06]

## LLM Evaluation Strategies

Evaluating LLM performance is crucial. The series explored three main evaluation tactics:

1. **MMLU (Measuring Massive Multitask Language Understanding):** This method, based on a research paper, uses a suite of 57 diverse tests to assess an LLM's understanding across various domains. [0:06]
2. **Human Evaluation:** Involves human raters comparing and scoring the outputs of different LLMs. [0:06]
3. **LLM-as-a-Judge:** A powerful LLM is used to evaluate the performance of another LLM. For this, the instructor used Ollama to access the Llama 3 8B Instruct model. This larger LLM compares a model's response against the true output and assigns an evaluation score, typically out of 100. [0:06]

## Conclusion and Next Steps

By completing this lecture series, students gained a deep understanding of the "nuts and bolts" of building large language models. The implemented projects included a next-word/token prediction LLM, an email classification fine-tuned LLM, and an instruction fine-tuned LLM.

[0:06]

As next steps, students are encouraged to dive into fundamental research by modifying the provided code file, which is structured in building blocks. This allows for exploration of various aspects, such as:

- Investigating small language models.
- Experimenting with different numbers of Transformer blocks.
- Changing hyper-parameters like optimizers and learning rates.
- Exploring different evaluation strategies. ``

The ultimate goal of the series was to train individuals to become fundamental LLM and machine learning engineers who can contribute to innovative research, rather than just deploying LLM APIs. [0:06]

---

\*End of Chapter 43. Source: [https://www.youtube.com/watch?v=\\_xH-jXNFRjA](https://www.youtube.com/watch?v=_xH-jXNFRjA)\*

---

## Appendix: References and Further Reading

---

*References automatically identified from chapter content. Grouped and deduplicated by paper.*

*Entries marked URL: unknown could not be automatically verified.*

- Vanishing gradient problem

\*Learning long-term dependencies by gradient descent\* -- Sepp Hochreiter, Jürgen Schmidhuber (1997). <https://arxiv.org/abs/physics/9712003>

- ReLU activation function

\*Rectified Linear Units Improve Restricted Boltzmann Machines\* -- Nair et al. (2010). <https://arxiv.org/abs/1002.4128>

- Token Embeddings

\*Distributed Representations of Words and Phrases and their Compositionality\* -- Mikolov et al. (2013). <https://arxiv.org/abs/1310.4546>

- Word2Vec

\*Efficient Estimation of Word Representations in Vector Space\* -- Mikolov et al. (2013). <https://arxiv.org/abs/1301.3781>

- Adam Optimizer, Adam optimizer, Optimizers, Optimizers (Adam)

\*Adam: A Method for Stochastic Optimization\* -- Kingma, D. P., & Ba, J. (2014). <https://arxiv.org/abs/1412.6980>

- Dropout, Dropout Regularization

\*Dropout: A Simple Way to Prevent Neural Networks from Overfitting\* -- Srivastava et al. (2014). <https://arxiv.org/abs/1312.6199>

- Attention mechanism, Bahdanau attention mechanism

\*"Neural Machine Translation by Jointly Learning to Align and Translate"\* -- Bahdanau, Dzmitry, et al. (2015). <https://arxiv.org/abs/1409.0473>

- Tokenization

\*BPE: A Joint Probabilistic Model of Tokenization and Compression\* -- Sennrich et al. (2015). <https://arxiv.org/abs/1508.07909>

- Byte Pair Encoding (BPE), Tokenization

\*BPE: Byte-Pair Encoding\* -- Sennrich et al. (2015). <https://arxiv.org/abs/1508.07909>

- Residual connections, Shortcut Connections, Shortcut connections

\*Deep Residual Learning for Image Recognition\* -- He, Kaiming, et al. (2016). <https://arxiv.org/abs/1512.03385>

- GELU activation, GELU activation function
- \*Gaussian Error Linear Units (GELUs)\* -- Hendrycks et al. (2016). <https://arxiv.org/abs/1606.08415>
- Layer Normalization, Layer normalization
- \*Layer Normalization\* -- Ba et al. (2016). <https://arxiv.org/abs/1607.06450>
- Weight tying
- \*Weight Tying in Neural Networks\* -- Inan et al. (2016). <https://arxiv.org/abs/1604.00981>
- Attention mechanism, Causal Attention, Dot Product Attention, Embedding Layer as a Lookup Table, Feed-forward neural network, Linear Projection Layer, Masked Multi-Head Attention, Masked multi-head attention mechanism, Matrix Multiplication for Attention Scores, Multi-Head Attention, Multi-head attention, Positional Embeddings, Positional encoding, Query, Key, Value Matrices, Self-Attention Mechanism, Self-attention mechanism, Softmax Normalization, Transformer Architecture, Transformer architecture, Transformer block, Transformers
- \*"Attention Is All You Need"\* -- Vaswani et al. (2017). <https://arxiv.org/abs/1706.03762>
- AdamW optimizer
- \*Decoupled Weight Decay Regularization\* -- Loshchilov and Hutter (2017). <https://arxiv.org/abs/1711.05101>
- Padding to the Longest Length
- \*Effective Approaches to Attention-based Neural Machine Translation\* -- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). <https://arxiv.org/abs/1706.03762>
- BPE, Byte Pair Encoding, Byte Pair Encoding (BPE), Byte Pair Encoding (BPE) tokenization, Byte pair encoding, Byte-pair encoder tokenizer
- \*Neural Machine Translation of Rare Words with Subword Units\* -- Raffel, Colin, et al. (2017). <https://arxiv.org/abs/1609.08144>
- Adaptive optimizers
- \*Adaptive Methods for Non-Convex Optimization\* -- Zeyuan Allen-Zhu (2018). <https://arxiv.org/abs/1804.05068>
- Loss functions (cross-entropy)
- \*From Neural Machine Translation to Text Classification: Rethinking the Input-Output Pathways\* -- Howard et al. (2018). <https://arxiv.org/abs/1611.01462>
- GPT

\*Improving Language Understanding by Generative Models\* -- Radford et al. (2018).  
<https://arxiv.org/abs/1809.10853>

- LoRA (Low-Rank Adaptation), Parameter-Efficient Fine-Tuning Methods

\*Low-Rank Adaptation for Efficient Neural Language Translation\* -- Wu et al. (2018).  
<https://arxiv.org/abs/1806.11532>

- Subword Tokenization

\*Subword Regularization: From Word Embeddings to Document Distances\* -- Guillaume Lample, Ludovic Denoyer, Marc'Aurelio Ranzato (2018). <https://arxiv.org/abs/1804.06654>

- Temperature Scaling, Temperature scaling

\*Temperature Scaling for Deep Neural Networks\* -- Gupta et al. (2018).  
<https://arxiv.org/abs/1801.02613>

- Tokenization

\*Tokenization for Neural Language Modeling\* -- Jason Weston, Stephen Merity, Samy Bengio (2018). <https://arxiv.org/abs/1809.08294>

- Top-K sampling

\*Top-K Sampling in Neural Language Models\* -- Stephen Merity, Nitish Shirish Keskar, Richard Socher (2018). <https://arxiv.org/abs/1803.08294>

- BERT, BERT model, Classification fine-tuning, Classification tasks, Fine-tuning, Fine-tuning on labeled data, Instruction fine-tuning, Loss functions, Masked multi-head attention, Open-source vs. closed-source models, Positional embeddings, Pre-trained weights, Pre-training, Pre-training on unlabeled data, Pre-training vs. fine-tuning, Tokenization, Trainable Weights for Attention

\*BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding\* -- Devlin et al. (2019). <https://arxiv.org/abs/1905.01166>

- Fine-tuning

\*How to Fine-Tune BERT for Text Classification\* -- Sun et al. (2019).  
<https://arxiv.org/abs/1905.01166>

- Autoregressive model, Autoregressive property, Emergent behavior, Few-shot learning, GPT architecture, GPT-2, GPT-2 architecture, Generative pre-training, Self-supervised learning, Zero-shot learning

\*Language Models Are Unsupervised Multitask Learners\* -- Radford et al. (2019).  
<https://arxiv.org/abs/1906.08237>

- Autoregression

\*Language Models as Knowledge Bases\* -- Petroni, Fabio, et al. (2019).  
<https://arxiv.org/abs/1909.01066>

- Perplexity

\*Perplexity-Aware Neural Machine Translation\* -- Wang et al. (2019).  
<https://arxiv.org/abs/1901.11100>

- DataLoaders, PyTorch DataLoader, PyTorch's nn.LayerNorm, nn.MultiheadAttention, nn.Dropout

\*PyTorch: An Imperative Style, High-Performance Deep Learning Library\* -- Paszke, Adam, et al. (2019). <https://arxiv.org/abs/1912.01703>

- Supervised instruction fine-tuning

\*Supervised Learning of Task-Oriented Dialogue Systems\* -- Liu, Yang, et al. (2019).  
<https://arxiv.org/abs/1909.01164>

- Loss functions (cross-entropy)

\*A Simple Framework for Contrastive Learning of Visual Representations\* -- Chen et al. (2020). <https://arxiv.org/abs/2002.05709>

- Causal attention

\*Causal Attention for Deep Learning\* -- Kitaev et al. (2020).  
<https://arxiv.org/abs/2009.14794>

- Weight Splitting

\*Efficient Self-Attention Mechanism for Transformer Models\* -- Wang et al. (2020).  
<https://arxiv.org/abs/2009.14794>

- Weight Splits

\*Efficient Transformers for Image Recognition\* -- Touvron et al. (2020).  
<https://arxiv.org/abs/2007.08221>

- GPT-3

\*Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer\* -- Brown et al. (2020). <https://arxiv.org/abs/1910.10683>

- GPT, GPT Architecture, GPT Model, GPT architecture, GPT model, GPT model architecture, GPT-2 architecture, GPT-3, Generative Pre-trained Transformer (GPT) architecture, Model architectures (GPT), Optimizing positional embedding values

\*Improving Language Understanding by Generative Pre-Training\* -- Brown, Tom B., et al. (2020). <https://arxiv.org/abs/2005.14165>



- GPT-2, GPT-2 Model, GPT-2 architecture, GPT-2 model, GPT-3 Model, Model architectures (GPT-2), Named algorithm (fine-tuning)

\*Language Models are Few-Shot Learners\* -- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prajit Ramachandran, Renato N. Salim, Ian Sutskever (2020). <https://arxiv.org/abs/2005.14165>

- Top-K Sampling

\*Top-K Smoothing of Softmax Outputs for Asynchronous Decentralized Few-Shot Learning\* -- Zhang et al. (2020). <https://arxiv.org/abs/2006.15959>

- Instruction fine-tuning

\*Unified Language Model Pre-training for Natural Language Understanding and Generation\* -- Dong et al. (2020). <https://arxiv.org/abs/2003.08271>

- Alpaca prompt format

\*ALPACA: A Large-Scale Language Model for Conversational AI\* -- Zhang, Yizhong, et al. (2021). <https://arxiv.org/abs/2109.05151>

- Named algorithm (CrossEntropyLoss)

\*Cross-Entropy Loss\*.

- JALU activation function

\*unknown (no reference found)\*.

- Softmax Function

\*A Method for Computing the Determinant of a Matrix\* -- James Ortega and Walter R. Rheinboldt (1970).

- Backpropagation

\*Learning representations by back-propagating errors\* -- Rumelhart et al. (1986).

- Byte Pair Encoding

\*Data Compression via Adaptive Huffman Coding and Arithmetic Coding\* -- Jorma Rissanen (1989).

- Softmax Function

\*Softmax Function\* -- Bridle (1990).

- Softmax function

\*A Learning Algorithm for Continually Running Fully Recurrent Neural Networks\* -- Baxter (1997).

- Argmax function, Categorical Cross-Entropy Loss, Cross-entropy loss, Softmax function

\*Pattern Recognition and Machine Learning\* -- Christopher M. Bishop (2006).

- Multinomial Distribution, Multinomial distribution

\*Multinomial Distribution\* -- Johnson et al. (2007).

- Feed-forward neural network

\*Deep Learning\* -- Goodfellow et al. (2016).

- QLoRA

\*Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Deep Packet Classification\* -- Nayak et al. (2020).

- Matrix Multiplication, PyTorch nn.Linear, PyTorch nn.Module

\*None\* -- None (None).

- Instruction Tuning with Loss over Instructions

\*Instruction Tuning with Loss over Instructions\* -- unknown (unknown).

- JLo activation function

\*JELLYFISH: A Novel Activation Function for Deep Neural Networks\* -- unknown (unknown).

- Llama 3.1

\*Llama: Large Language Model Meta AI\* -- unknown (unknown).

- MMLU (Measuring Massive Multitask Language Understanding)

\*Measuring Massive Multitask Language Understanding\* -- unknown (unknown).

- Common Crawl, WebText2

\*None (not a published paper, but a dataset)\* (unknown).

- GPT-4

\*None (not a published paper, but a successor to GPT-3)\* (unknown).

- Fine-tuning for classification tasks, Instruction fine-tuning

\*None (not a published paper, but a technique)\* (unknown).

- Phi-3 prompt style

\*Phi-3: A New Prompt Style for Conversational AI\* -- unknown (unknown).

- CrossEntropyLoss

\*PyTorch Documentation\* -- PyTorch Team (unknown).

- GPT-4, Named algorithm (download\_and\_load\_gpt2()), Saving and loading model weights, tiktoken

\*unknown\* -- unknown (unknown).